



Základy počítačové grafiky

IZG

Studijní opora

Autor: Ing. Přemysl Kršek, Ph.D.

verze 0.9.1

Tento učební text vznikl za podpory projektu „Zvýšení konkurenceschopnosti IT odborníků – absolventů pro Evropský trh práce“, reg. č. CZ.04.1.03/3.2.15.1/0003. Tento projekt je spolufinancován Evropským sociálním fondem a státním rozpočtem České republiky.

Obsah

1	Úvod	8
1.1	Slovo autorů	8
1.2	Cíle předmětu a struktura opory	8
2	Základní principy Počítačové grafiky	10
2.1	Co je to Počítačová grafika?	10
2.2	Rastrová grafika	11
2.3	Vektorová grafika	13
2.4	Princip zobrazení v počítačové grafice	14
3	Barvy a barevné modely	16
3.1	Barva	16
3.1.1	Co je to barva?	16
3.1.2	Míchání barev	18
3.2	Barevné modely	19
3.2.1	RGB	19
3.2.2	CMY (CMYK)	19
3.2.3	Chromatický diagram, CIE	19
3.2.4	HLS	19
3.2.5	HSV	21
3.3	Redukce barevného prostoru	21
3.3.1	GrayScale	22
3.3.2	Dithering/Halftoning	22
3.3.3	Prahování	22
3.3.4	Náhodné rozptýlení	23
3.3.5	Maticové rozptýlení	24
3.3.6	Distribuce chyby	24
3.3.7	Paleta 332	24
4	Rasterizace ve 2D	28
4.1	Rasterizace úsečky	28
4.1.1	DDA algoritmus	29
4.1.2	Vykreslování úsečky s hlídáním chyby	30
4.1.3	Bresenhamův algoritmus	30
4.2	Rasterizace kružnice	32
4.2.1	Vykreslení kružnice po bodech	32
4.2.2	Vykreslení kružnice jako n-úhelník	33
4.2.3	Midpoint algoritmus	33
4.3	Rasterizace elipsy	35
4.3.1	Midpoint algoritmus	35

5	Vyplňování 2D oblastí	39
5.1	Řádkové vyplňování	41
5.1.1	Lichý počet průsečíků řádku	42
5.1.2	Optimalizace, seznam aktivních hran	42
5.2	Inverzní řádkové vyplňování	43
5.3	Pinedův algoritmus	44
5.4	Semínkové vyplňování	46
5.4.1	Řádkové semínkové vyplňování	47
6	Ořezávání vektorových 2D objektů	50
6.1	Ořezávání úseček	51
6.1.1	Cohen-Sutherland	52
6.1.2	Liang-Barsky	53
6.2	Ořezávání polygonů	54
6.2.1	Sutherland-Hodgman	55
6.2.2	Weiler-Atherton	56
7	Geometrické transformace	60
7.1	Homogenní souřadnice	60
7.2	Základní geometrické transformace ve 2D	61
7.2.1	Posunutí	61
7.2.2	Otáčení	63
7.2.3	Změna měřítka	63
7.2.4	Zkosení	64
7.3	Transformace ve 3D	64
7.4	Skládání transformací	65
7.4.1	Rotace ve 3D kolem obecné osy	66
7.5	Projekce	67
7.5.1	Paralelní	67
7.5.2	Perspektivní projekce	70
7.5.3	Pohledový objem	71
8	2D křivky	74
8.1	Křivky v CG	74
8.2	Spline křivky, spojitost křivek	74
8.3	Interpolační křivky	74
8.3.1	Fergusonova kubika	74
8.3.2	Cochanek/Bartels	74
8.3.3	Catmul-Room	74
8.4	Aproximační křivky	74
8.4.1	Bezierovy křivky	74
8.4.2	NURBS křivky	74
9	Datová reprezentace 3D objektů	75
9.1	Požadavky na popis 3D dat	75
9.2	CSG	75
9.3	B-rep	75
9.4	Voxel modely	75
9.5	Implicitní modely	75
9.6	LOD	75
9.7	Subdivision	75
9.8	3D spline plochy	75

10 Řešení viditelnosti 3D objektů	77
10.1 Základní myšlenka řešení viditelnosti	77
10.2 Vektorové algoritmy	77
10.2.1 Robertsův algoritmus	77
10.2.2 Metoda plovoucího horizontu	77
10.3 Rastrové algoritmy	77
10.3.1 Malířův algoritmus	77
10.3.2 Dělení obrazu	77
10.3.3 Z-buffer	77
10.3.4 Ray-casting	77
11 Osvětlení s stínováním 3D objektů	78
11.1 Osvětlovací modely	78
11.1.1 Lambertův osvětlovací model	78
11.1.2 Phongův osvětlovací model	78
11.2 Stínování	78
11.2.1 Konstantní stínování	78
11.2.2 Gourandovo stínování	78
11.2.3 Phongovo stínování	78
12 Texturování 3D objektů	79
12.1 Mapování textur	81
12.1.1 Inverzní mapovací funkce	83
12.1.2 Texturovací souřadnice	84
12.1.3 Promítání textur	85
12.1.4 Mapování 3D textur	86
12.2 Aplikace textur	86
12.2.1 Bump mapping	86
12.2.2 Displacement mapping	88
12.2.3 Environmental mapping	88
12.3 MIP mapping	90
12.4 Perspektivní korekce textur	90

Seznam obrázků

2.1	Schéma počítačové grafiky	10
2.2	Ilustrace interpretace textové a grafické informace	11
2.3	Vizuální klamy, šachovnice a pohár	12
2.4	Schéma principu rastrové grafiky	12
2.5	Schéma principu vektorové grafiky	13
2.6	Detail zobrazení rastrové a vektorové grafiky	14
2.7	Schéma procesu zobrazení v počítačové grafice	15
3.1	Rozsah frekvencí elektromagnetického záření	16
3.2	Barevné spektrum viditelného světla	17
3.3	Schéma lidského oka	18
3.4	Skládání barev	18
3.5	Barevný model RGB	19
3.6	Barevný model CMY	20
3.7	Chromatický diagram, CIE	20
3.8	Barevný model HLS	20
3.9	Barevný model HSV	21
3.10	Testovací obrázek pro redukci barevného prostoru	21
3.11	Redukce barevného prostoru na 256 stupňů šedi	22
3.12	Redukce barevného prostoru na černou a bílou prahováním	23
3.13	Redukce barevného prostoru na černou a bílou náhodným rozptýlením	23
3.14	Redukce barevného prostoru na černou a bílou maticovým rozptýlením jako dithering	24
3.15	Redukce barevného prostoru na černou a bílou maticovým rozptýlením jako halftoning	25
3.16	Redukce barevného prostoru na černou a bílou rozptýlením s distribucí chyby	25
3.17	Floyd-Steinberg schéma distribuce chyby	25
3.18	Redukce barevného prostoru barevného RGB obrazu zmenšením počtu barev paletou 332	26
4.1	Definice úsečky ve 2D prostoru	29
4.2	Algoritmus DDA pro vykreslení úsečky	30
4.3	Algoritmus vykreslení úsečky s hlídáním chyby	30
4.4	Bresenhamův algoritmus pro vykreslení úsečky	32
4.5	Definice kružnice ve 2D prostoru	32
4.6	Algoritmus pro vykreslení kružnice po bodech	33
4.7	Algoritmus pro vykreslení kružnice jako n-úhelník	34
4.8	Midpoint algoritmus pro vykreslení kružnice	35
4.9	Definice kružnice ve 2D prostoru	36

4.10	Midpoint algoritmus pro vykreslení kružnice	37
5.1	Polygonální model lebky	40
5.2	Geometrie oblastí.	40
5.3	Pravidla vyplňování oblastí.	40
5.4	Druhy výplně oblastí.	40
5.5	Řádkové vyplnění oblasti.	41
5.6	Inverzní řádkové vyplnění oblasti.	43
5.7	Schéma Pinedova algoritmu.	45
5.8	Strategie procházení řádků pro Pinedův algoritmus.	45
5.9	Druhy oblastí a vyplňování pro semínkové vyplňování.	47
6.1	Primitivní ořezání úsečky.	51
6.2	Určování průsečíků úsečky s hranicí okna.	51
6.3	Ořezání úsečky algoritmem Cohen-Sutherland.	52
6.4	Ořezání úsečky algoritmem Liang-Barsky.	53
6.5	Orientace úsečky vůči oknu při ořezávání algoritmem Liang-Barsky.	54
6.6	Postup ořezání polygonu algoritmem Sutherland-Hodgman.	55
6.7	Postup ořezání polygonu jednou stranou okna algoritmem Sutherland-Hodgman.	55
6.8	Vznik překrývajících se hran při ořezání polygonu algoritmem Sutherland-Hodgman.	56
6.9	Postup ořezání polygonu algoritmem Weiler-Atherton	57
7.1	Geometrická představa homogenních souřadnic ve 2D.	61
7.2	Schéma základních lineárních geometrických transformací.	62
7.3	Skládání transformací.	65
7.4	Schéma rotace okolo obecné osy.	66
7.5	Paralelní a perspektivní projekce.	67
7.6	Mongeova projekce.	68
7.7	Axonometrická projekce.	68
7.8	Kosoúhlá projekce.	69
7.9	Kosoúhlá projekce, realizace.	69
7.10	Druhy perspektivní projekce.	70
7.11	Schéma odvození perspektivní projekce.	71
7.12	Pohledový objem pro paralelní a perspektivní projekci.	72
12.1	Zobrazení scény interiéru bodovy.	79
12.2	Použití textury s průhledností.	81
12.3	Ilustrace mapování textur jako tapetování	82
12.4	Mapování textur.	82
12.5	Inverzní mapování na válec.	83
12.6	Inverzní mapování na kouli.	84
12.7	Mapování textur s použitím texturovacích souřadnic.	85
12.8	Způsoby zobrazení povrchů při promítání textur.	86
12.9	Příklady mapování textury promítáním.	87
12.10	Mapování 3D textury.	88
12.11	Výsledný efekt aplikace bump textury.	89
12.12	Způsob úpravy povrchu objektu při bump mappingu.	89
12.13	Srovnání výsledného efektu pro bump a displacement mapping.	89
12.14	Displacement mapping efekt na povrchu koule.	90
12.15	Aplikace displacement mappingu na povrch objektu.	91
12.16	Environmental mapping efekt na povrchu objektu.	92

12.17Schéma aplikace environmental mappingu.	92
12.18Aplikace environmental mappingu na povrch objektu.	92
12.19Schéma hierarchie MIP textury.	93
12.20Zobrazení matice MIP textury.	93
12.21Srovnání zobrazení textury.	93
12.22Zobrazení problematiky perspektivní korekce mapování textur. . .	94
12.23Zobrazení textur v perspektivní projekci.	94

Kapitola 1

Úvod



0:15

1.1 Slovo autorů

Vážení studenti, dostává se vám do rukou studijní opora k předmětu Základy počítačové grafiky, který je vyučován ve druhém ročníku bakalářského studijního programu Informační technologie na Fakultě informačních technologií Vysokého učení technického v Brně.

Studijní opory jsou obecně určeny pro distanční a kombinované studium a slouží jako náhrada přednášek kvůli specificky omezeným možnostem studia zejména pracujících studentů. Domníváme se však, že studijní opora může být užitečnou pomůckou i pro studenty prezenční formy studia — v ideálním případě by měla prezenčním studentům sloužit pro zopakování látky před zkouškou, v méně ideálních případech pak pro dostudování látky při nezbytné absenci na přednášce nebo, což vzhledem k charakteru předmětu bude spíše výjimečné, při nepochopení látky z přednášky. Věříme, že studenti v prezenční formě studia budou navštěvovat přednášky v hojném počtu i přesto, že se jim tato studijní opora dostala do rukou.

Následující text je výběrem těch nejdůležitějších částí našich přednášek, které jsme doplnili o podrobné komentáře i o prvky vyžadované pro samostudium. Jedná se především o zavedení grafických symbolů (tzv. piktogramů) pro označení specifických částí textu. Význam jednotlivých piktogramů je v tabulce 1.1. Oproti klasickým skriptům zde navíc najdete otázky, které vám pomohou ověřit, že jste probíranou látku dobře zvládli.

Tato studijní opora vznikla v průběhu roku 20065 a vy jste jejími prvními čtenáři. Prosíme vás proto o jistou dávku shovívavosti a také o vaše připomínky a komentáře, například k časům potřebným pro studium jednotlivých kapitol.

Autor

1.2 Cíle předmětu a struktura opory

Cílem předmětu je, aby studenti získali přehled o základních metodách a algoritmech v oblasti *Počítačové grafiky* především z pohledu programátora s důrazem na vektorovou 2D i 3D grafiku.



Udává se, že maximální rozsah studijní opory je 60 stran, protože větší rozsah značně snižuje odhodlání studentů se studijní oporou *prokousat*. Protože tato studijní opora bude sloužit i (a možná především) studentům prezenční formy studia, dovolili jsme si tento rozsah mírně překročit. I tak jsme nemohli do této opory zařadit řadu témat, kterým se v přednáškách věnujeme. Na jednu stranu jsme tím čtenáře ochudili o jistě zajímavá témata, na druhou stranu jsme vybrali pouze to, co považujeme za opravdu podstatné a co se proto **pravidelně objevuje u zkoušek**.

	Čas potřebný ke studium		Kontrolní otázky
	Cíl		Počítačové cvičení
	Definice		příklad
	Důležitá část		reference, odkazy
	Rozšiřující látka		Správné odpovědi, řešení
	Obtížná část, stupeň 1		Shrnutí
	Obtížná část, stupeň 2		komentář, slovo autora
	Obtížná část, stupeň 3		Zajímavé místo

Tabulka 1.1: Význam použitých piktogramů

Probíraná látka je organizována následovně. Na začátku se seznámíme se základními principy počítačové grafiky, rastrovou a vektorovou grafikou. Jako vstup do oboru je uvedena problematika barev, barevných modelů a redukce barevného prostoru rastrových obrazů. Další ucelená část je pak věnována 2D vektorové grafice: rasterizaci úseček, kružnic, elips, křivek a vyplňování uzavřených oblastí uzavřené ořezáním zobrazení pohledovým oknem. Přechod mezi 2D a 3D grafikou zajišťují geometrické transformace spolu s projekcemi (paralelní a perspektivní). Oporu pak uzavírá problematika 3D vektorové grafiky: reprezentace 3D objektů, viditelnost, osvětlení, texturování a realistické zobrazení.

Kapitola 2

Základní principy Počítačové grafiky



0:30



Cílem této kapitoly je seznámit se s základními dvěma principy počítačové grafiky z pohledu programátora, z hlediska popisu (reprezentace) zobrazovaných dat, což jsou rastrová a vektorová grafika.

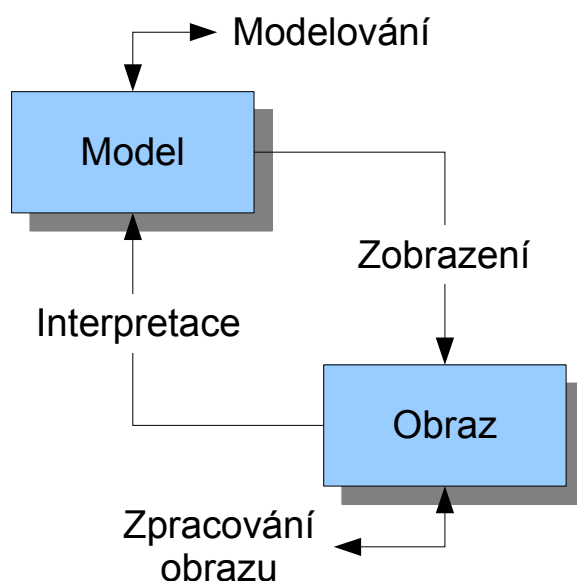
2.1 Co je to Počítačová grafika?

Počítačová grafika (Computer graphics - CG) je součástí oboru *Informatika*, která se zabývá analýzou (interpretací) nebo tvorbou (syntézou, generováním) grafické obrazové informace (viz. obr. 2.1).

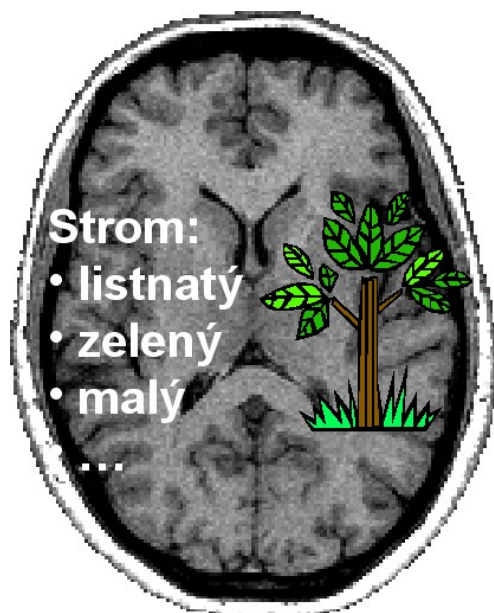


Máme-li již k dispozici nějakou obrazovou informaci, získanou snímáním nebo měřením (kamerou atd.), potřebujeme ji většinou dále zpracovat (upravit, vylepšit) nebo analyzovat (interpretovat a rozpoznat objekty). Jedná se o aplikace typu *Počítačové vidění*.

Máme-li naopak k dispozici virtuální počítačový (matematický) popis (model) objektů, potřebujeme ho převést na grafickou obrazovou informaci (syntéza) a zobrazit.



Obrázek 2.1: Schéma počítačové grafiky



Obrázek 2.2: Ilustrace interpretace textové a grafické informace

Ve srovnání s např. textovým popisem nebo zvukovým vjemem je pro nás grafické vyjádření informace (její zobrazení) nejvíce informačně obsažné (viz. obr. 2.2). Náš mozek je schopen grafickou informaci velmi rychle a detailně analyzovat a interpretovat. Při zpracování jiné formy vyjádření informace máme sklony k různým dezinterpretacím nebo klamům. Klamy se však samozřejmě nevyhnou ani našemu vidění (viz. obr. 2.3).

Počítačovou grafiku si můžeme rozdělit z různých hledisek:

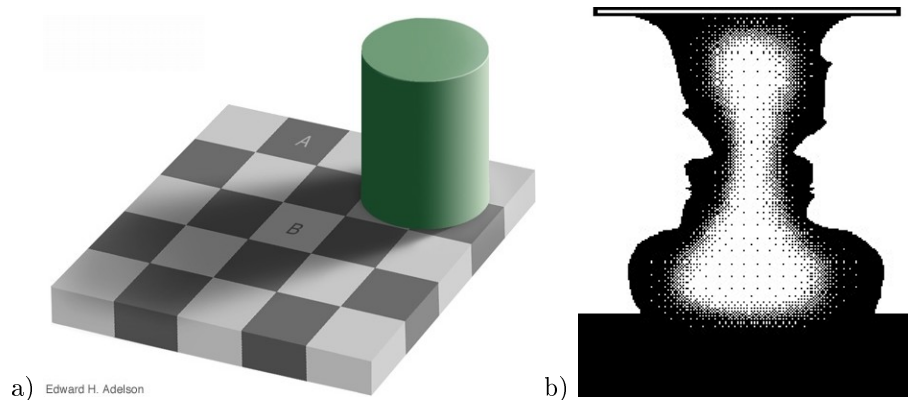
- Podle dimenze zpracovávané informace například na 2D a 3D.
- Podle oblasti použití například na zpracování fotek, videa, malování nebo CAD, vizualizaci, animaci atd.
- Z hlediska uživatele grafických programů nebo z hlediska jejich programátora.

My se zaměříme na počítačovou grafiku z pohledu programátora, z hlediska popisu (reprezentace) zobrazovaných dat. Z tohoto úhlu pohledu rozlišujeme *rastrovou* grafiku a *vektorovou* grafiku.

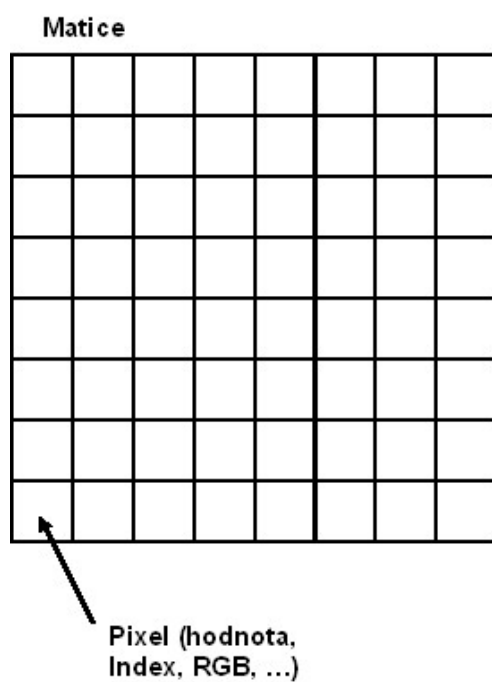
2.2 Rastrová grafika

V *Rastrové grafice* popisujeme (ukládáme, definujeme) zpracovávané a zobrazované informace ve formě rastrové matice (2D / 3D) (viz. obr. 2.4), diskrétně. Tento popis dat získáme: manuálně, syntézou (generováním nebo převodem z jiného popisu) nebo snímáním (kamerou atd.). Jeden prvek matice nazýváme *pixel*.

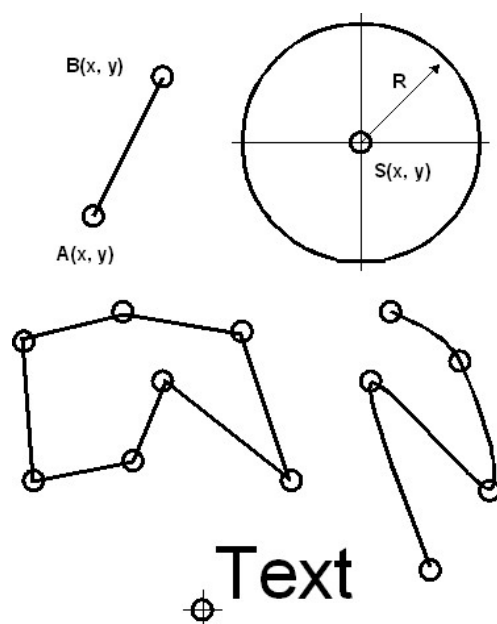
DEF



Obrázek 2.3: Vizuální klam: a) políčko šachovnice A a B mají ve skutečnosti stejnou barvu, porovnejte, b) co vidíte, tvář nebo pohár?



Obrázek 2.4: Schéma principu rastrové grafiky



Obrázek 2.5: Schéma principu vektorové grafiky

V rastrové grafice uchováváme informaci (v rastrové matici) pouze o podobě zobrazených objektů, ne o samotných objektech. Jakmile je objektu jednou zobrazen, jako takový přestává existovat a uloženou informaci lze změnit pouze na úrovni jednotlivých pixelů obrazu.

Rozlišení rastrové matice vůči zobrazovaným objektům je dopředu dané a nelze je jednoduše měnit. Jedinou možností směny rozlišení je převzorkování, které sice změní fyzické rozlišení, ale nepřidá chybějící informaci. Rozlišení obrazu (úroveň detailu) se při tom buď zhorší (podvzorkování) nebo zlepší (interpolace, vyhlazení). Informace je tedy uložena s konečnou úrovní detailu. Když se podíváme na detail dané matice, můžeme vidět až jednotlivé pixely, ale úroveň detailu zobrazené informace se tím nezmění (viz. obr. 2.6).

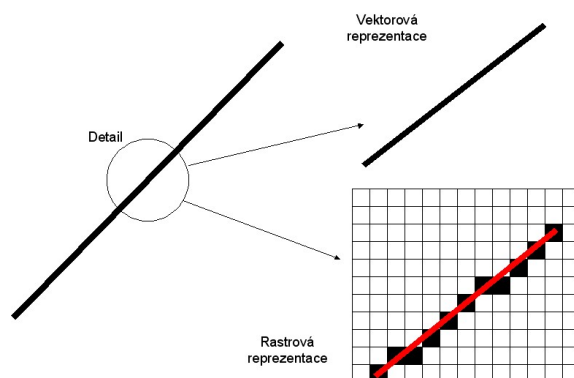
2.3 Vektorová grafika

Ve Vektorové grafice popisujeme (ukládáme, definujeme) zpracovávané a zobrazované informace ve formě skupiny vektorových entit (úsečky, kružnice, křivky, polygony, atd.) (viz. obr. 2.5), analyticky. Tento popis dat získáme: manuálně nebo syntézou (generováním nebo převodem z jiného popisu).

DEF

Všechny objekty definované ve vektorové grafice jsou tvořeny seznamem základních entit. Ve 2D to jsou: úsečka, kružnice, křivka, polygon (vyplněná oblast). Uchováváme tedy informaci o popisu zobrazených objektů. Při každém zobrazení se potom popis objektů převádí na jejich zobrazení (rastrové). Jednotlivé objekty a jejich tvořící entity stále existují a proto můžeme podle potřeby měnit jejich vlastnosti (barva, styl, atd.) a parametry (polohu, velikost, orientaci, atd.).

Jednotlivé entity jsou definovány přesně matematicky (geometrie entit, souřadnice bodů, atd.) a parametricky (barvy, styl a tloušťka čar, atd.). Úsečka je definována koncovými body a přímkou, kružnice středem a poloměrem, atd. Entity



Obrázek 2.6: Detail zobrazení rastrové a vektorové grafiky

jsou tedy popsány nezávisle na rozlišení jejich zobrazení, jako v rastrové grafice. Když se podíváme na detail zobrazení vektorových objektů, je jejich zobrazení provedeno pro aktuální výstupní rastrové rozlišení s maximální úrovní detailu (viz. obr. 2.6).

2.4 Princip zobrazení v počítačové grafice

Problematika počítačové grafiky však není omezena pouze na popis dat, které zpracováváme a zobrazujeme. Je potřeba zajistit také jejich zobrazení na příslušném zobrazovacím zařízení, které je k dispozici.

Z hlediska základního principu (podle našeho dělení na rastrovou a vektorovou grafiku) jsou všechna (absolutně většina) současná zobrazovací zařízení založena na rastrovém principu. Monitory, displeje, projektory i tiskárny zajišťují zobrazení prostřednictvím množiny relativně malých bodů (vůči rozlišení našich očí). Naše vidění (očí) je také založeno na rastrovém principu. Oči jsou však napojeny na mozek, který provádí interpretaci získané vizuální informace, takže ji vnímá zpětně jako objekty.

Většina počítačových programů (minimálně polovina) pro tvorbu, zpracování a zobrazení grafické informace je však založena na vektorovém principu.

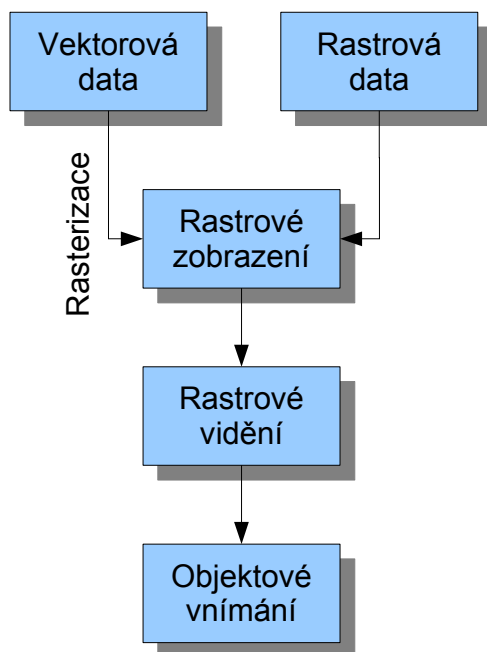
Dostáváme se tedy do rozporu, kdy máme data reprezentovány vektorově, ale zobrazujeme je rastrově. Při zobrazení vektorových se proto provádí jejich *rasterizace* (viz. obr. 2.7). Opačným procesem je *vektORIZACE*.

Rasterizace je proces převodu vektorových entit na jejich odpovídající zobrazení ve formě rastrové grafiky. Provádí se především pro potřeby zobrazení. Tento proces probíhá automaticky se snahou o jeho maximální rychlost, kvůli rychlosti zobrazení. Pro všechny druhy 2D i 3D entit je vektorizace bez problému řešitelná.

Vektorizace je proces převodu odpovídajícího rastrového zobrazení objektů na jejich vektorový popis. Provádí se především pro potřeby zpracování rastrového dat (obrazů). Tento proces není triviální a v mnoha případech je nejednoznačný. Neprobíhá proto automaticky, ale vyžaduje manuální korekci.

DEF

DEF



Obrázek 2.7: Schéma procesu zobrazení v počítačové grafice

Shrnutí

Z programátorského hlediska, z pohledu reprezentace dat rozdělujeme počítačovou grafiku na rastrovou a vektorovou. Rastrová grafika je založena na diskrétním popisu prostřednictvím rastrové matice. Vektorová grafika je založena na analytickém popisu množinou základních entit (úsečky, kružnice, křivky, polygony, atd.).



Kontrolní otázky

1. Co je to Počítačová grafika?
2. Jaké dva základní druhy počítačové grafiky rozlišujeme, z hlediska reprezentace dat?
3. Co je podstatou rastrové počítačové grafiky?
4. Co je podstatou vektorové počítačové grafiky?
5. Která z těchto dvou základních druhů počítačové grafiky je lepší?
6. Jakým způsobem probíhá zobrazení v současné počítačové grafice?



Kapitola 3

Barvy a barevné modely



1:30



Cílem této kapitoly je seznámit se s problematikou barev v kontextu počítačové grafiky. Nejprve se budeme zabývat definicí a popisem barev prostřednictvím několika základních barevných modelů. V druhé části se podíváme na základní metody a algoritmy redukce barevného prostoru, zmenšení počtu barev.

Svět, ve kterém žijeme je barevný, respektive mi jej vnímáme jako barevný. Éra němého černobílého filmu je již dávno pryč a proto musíme i v oblasti počítačové grafiky umět smysluplně a efektivně pracovat s barvami (popsat, definovat a převádět).

3.1 Barva

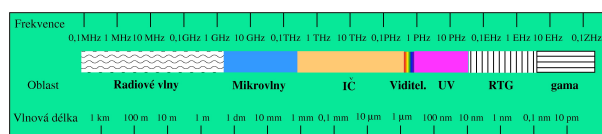
3.1.1 Co je to barva?

Abychom mohli vnímat barvy, musí být v prostoru kolem nás světlo, které osvětluje objekty, jejichž barvu vnímáme, a které dopadá do našich očí, abychom mohli vůbec něco vnímat. Světlo je ve své podstatě elektromagnetické záření o určitém rozsahu vlnových délek nebo frekvencí (viz. obr. 3.1). Z tohoto rozsahu jsme schopni vnímat pouze malou část, viditelné světlo (viz. obr. 3.2). Fyziologicky to znamená, že naše oči jsou citlivé o elektromagnetické záření právě v rozsahu vlnových délek viditelného světla.

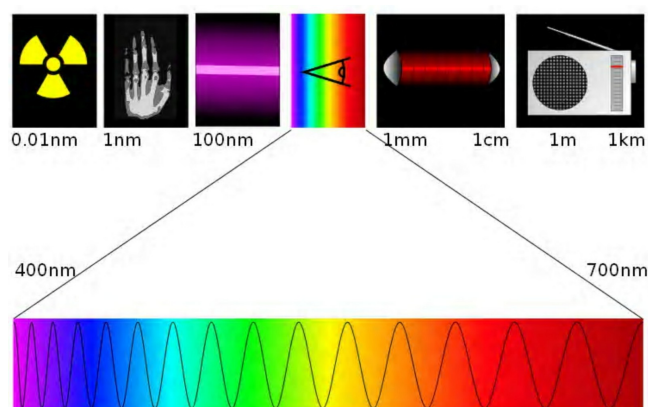
Ze světelného zdroje (slunce, žárovka atd.) se šíří světlo s určitým spektrem (zastoupením, které nemusí být úplně a rovnoměrné) vlnových délek, barevným spektrem, barvou. Podle toho rozlišujeme světlo bílé a barevné. Rozklad bílého (slunečního) světla na jednotlivé barevné složky (spektrum) provedl poprvé Sir Isaac Newton roku 1666 pomocí optického hranolu.

Bílé světlo (achromatické) má rovnoměrné zastoupení elektromagnetického záření všech vlnových délek ve viditelné oblasti. **Barevné světlo** (chromatické) má nerovnoměrné zastoupení elektromagnetického záření pouze některých vlnových délek z viditelné oblasti. Výsledná barva světla je pak dána smícháním obsažených složek.

Dopadá-li bílé světlo na povrch objektu, je část světla objektem odražena a část pohlcena, podle objektivních vlastností povrchu objektu. Spektrum vlnových délek odraženého světla vytváří efekt, který vnímáme jako barvu daného objektu.



Obrázek 3.1: Rozsah frekvencí elektromagnetického záření



Obrázek 3.2: Barevné spektrum viditelného světla

Dopadá-li na povrch objektu barevné světlo, odrazí se opět část spektra původního dopadajícího světla, podle vlastností povrchu objektu. Výsledný barevný efekt je však samozřejmě ovlivněn spektrem původního dopadajícího světla. Barvu objektu pak samozřejmě vnímáme jinak, přestože se objektivní vlastnosti jeho povrchu vůbec nezměnily.

Abychom vůbec mohli vnímat světlo a tím i jeho barvu, musíme k tomuto účelu mít vhodné detektory (senzory), v našem případě oči (viz. obr. 3.3). Na sítnici oka jsou světlocitlivé buňky, tyčinky (rods) a čípky (cones). Tyčinky ležící více na okraji sítnice reagují na intenzitu dopadajícího světla, Tyčinky nám tak kromě jiného umožňuje noční vidění, kdy je málo světla.

Čípky ležící ve středu sítnice jsou trojího druhu a reagují na světlo červené, zelené a modré vlnové délky. Tak nám právě umožňují barevné vidění se základními barvami červená, zelená a modrá. Největší citlivost máme na světlo zelené vlnové délky (nejvíce čípků), potom na červenou a nejméně na modrou.

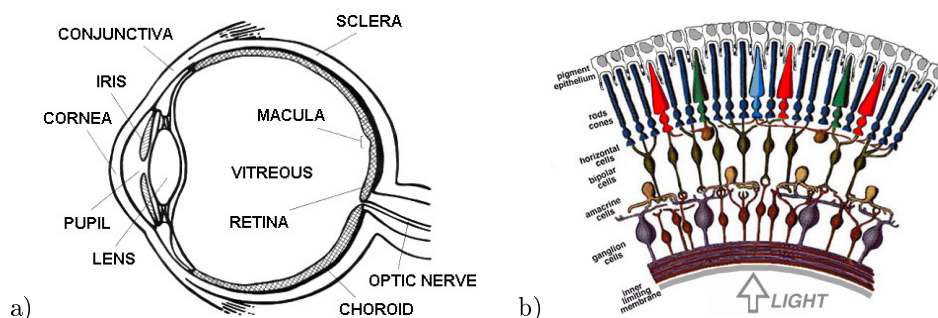
Kromě barvy tedy můžeme vnímat také další charakteristiky světla, jako jsou:

- Jas (svítivost) – intenzita světla, světelného zdroje.
- Sytost – čistota barvy světla, šířka barevného spektra.
- Světlost – velikost achromatické složky světla určité (hlavní) barvy.
- Odstín – dominantní vlnová délka světla (hue).

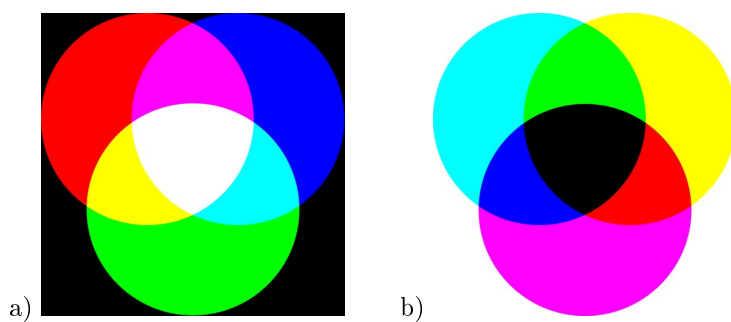
Z předchozích skutečností můžeme konečně přistoupit k definici pojmu *barva*. Víme, že závisí na světle, které dopadá na objekt. Dále pak na objektivních vlastnostech povrchu objektu, který určitou část vlnových délek dopadajícího světla odrazí nebo pohlcuje. Nakonec pak na detektoru (našem oku), do kterého odražené světlo dopadá a který toto světlo interpretuje jako určité barvy. Vnímané barvy přitom odpovídají určitým rozsahům vlnových délek světla, pro které má detektor odpovídající citlivost. Barvu tedy můžeme definovat jako:

Barva je subjektivní vjem vyvolaný působením světla daného spektra šířeného ze zdroje, které se odrazí od povrchu objektu (podle jeho objektivních vlastností) a dopadá na sítnici našeho oka. Oko pak předává získanou informaci do mozku, který ji interpretuje jako barvu odpovídající charakteristickým rozsahům vlnových délek světla odraženého od povrchu objektu.

DEF



Obrázek 3.3: Schéma lidského oka: a) schéma oka, b) schéma sítnice s tyčinkami (rods) a čípkami (cones)



Obrázek 3.4: Skládání barev: a) aditivní, b) subtraktivní

3.1.2 Míchání barev

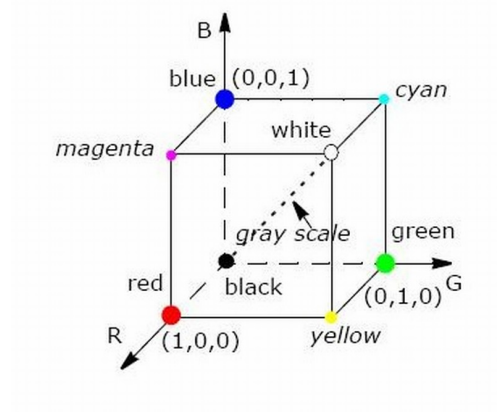
Kombinacemi různých barev, mícháním světla různých barevných spekter a intenzit z různých zdrojů světla, můžeme získat velké množství různých barev. Z toho plyne, že můžeme vhodně zvolit několik základních barev (dominantních vlnových délek), jejichž kombinací získáme všechny ostatní potřebné barvy. Pro vytvoření všech teoretických barev (celého barevného viditelného spektra) bychom však potřebovali nekonečně mnoho takovýchto základních barev. Pro praxi nám však úplně stačí tři základní barvy, kterými nejčastěji jsou: červená (Red), zelená (Green) a modrá (Blue) (RGB) nebo tyrkysová (Cyan), fialová (Magenta), žlutá (Yellow) (CMY). Podle výběru a způsobu práce se základními barvami rozlišujeme dva základní druhy míchání barev: aditivní a subtraktivní.

Aditivní míchání je charakteristické pro práci se světlem, u zařízení které vyzařují světlo (monitory, projektory, displeje, kamery, atd.) (viz. obr. 3.4a). Složením všech tří základních barev při jejich maximální intenzitě dostáváme bílou barvu. Aditivnímu skládání barev odpovídají základní barvy červená (Red), zelená (Green) a modrá (Blue) a barevný model RGB.

Subtraktivní míchání barev je charakteristické pro práci s pigmenty, u zařízení které tisknou barvy (tiskárny, plotry, atd.) (viz. obr. 3.4b). Složením všech tří základních barev při jejich maximální intenzitě dostáváme černou barvu. Aditivnímu skládání barev odpovídají základní barvy tyrkysová (Cyan), fialová (Magenta), žlutá (Yellow) a barevný model CMY.

DEF

DEF



Obrázek 3.5: Barevný model RGB

3.2 Barevné modely

Barevné modely umožňují pracovat s barvami, míchat barev. Je jich více druhů a jsou přizpůsobeny účelu použití, pro různé druhy uživatelů (programátor, grafik), různé oblasti použití (tisk, televizní technika).

3.2.1 RGB

Odpovídá aditivnímu skládání barev (viz. obr. 3.5). Jde o základní barevný model v oblasti počítačové techniky.

3.2.2 CMY (CMYK)

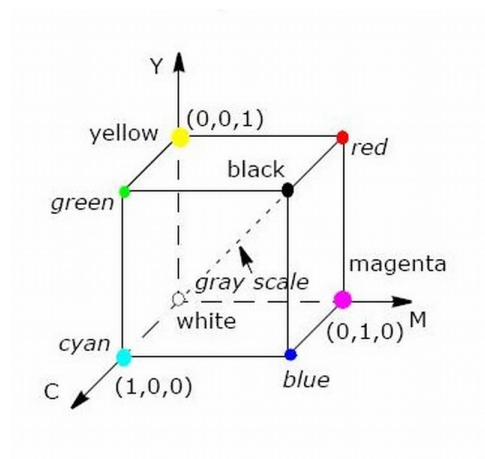
Odpovídá subtraktivnímu skládání barev (viz. obr. 3.6). Jde o specifický barevný model v oblasti počítačové techniky pro tiskový výstup. Prakticky se v realizaci barev používá model CMYK, kde K na konci představuje černou barvu (black). Fyzickým mícháním CMY pigmentů totiž nezískáme čistou černou barvu a proto ji musíme přidat.

3.2.3 Chromatický diagram, CIE

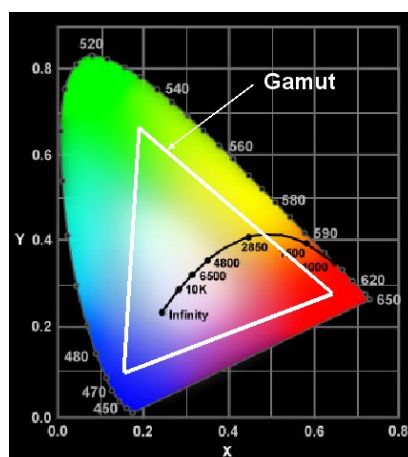
Mezinárodní standard pro porovnávání barev, kalibraci barevných zařízení (viz. obr. 3.7). Ostatní barevné modely můžeme brát jako podmnožinu chromatického diagramu nebo se dají takto zobrazit.

3.2.4 HLS

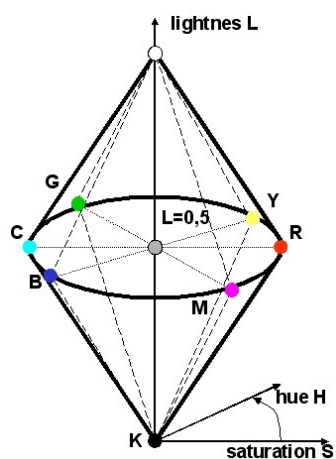
Jedná se o uživatelsky orientovaný barevný model, který umožňuje intuitivně pracovat s barvou, vybírat sytost (saturation), barevný tón (hue) a světlost (value) (viz. obr. 3.8). Přímou ovládat tyto vlastnosti barvy (RGB, CMY) v jiných modelech není možné.



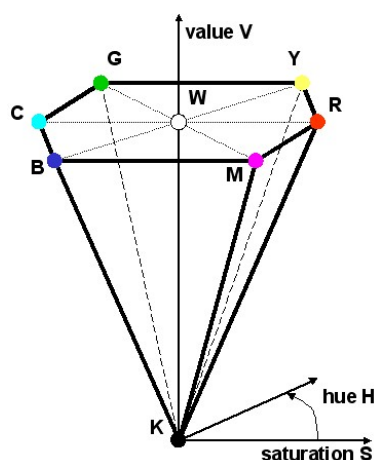
Obrázek 3.6: Barevný model CMY



Obrázek 3.7: Chromatický diagram, CIE



Obrázek 3.8: Barevný model HLS



Obrázek 3.9: Barevný model HSV



Obrázek 3.10: Testovací obrázek pro redukci barevného prostoru

3.2.5 HSV

Podobně jako HLS model se jedná se o uživatelsky orientovaný barevný model, který umožňuje intuitivně pracovat s barvou, vybírat sytost (saturation), barevný tón (hue) a jas (lightnes) (viz. obr. 3.9). Přímou ovládat tyto vlastnosti barvy (RGB, CMY) v jiných modelech není možné.

3.3 Redukce barevného prostoru

Máme-li tří prvkový barevný model (většina předchozích) a přidělíme-li pro každý barevný kanál (prvek) 8 bitů, pak máme k dispozici pro jednu barvu 24 bitů, což je ~ 16 miliónů barev (viz. obr. 3.10). V mnoha případech však nemá výstupní zařízení k dispozici takový barevný rozsah (tiskárny, projektory, displeje mobilních zařízení, atd.). Další možností je záměrné omezení barevného rozsahu z důvodu zmenšení velikosti dat (komprese). Z těchto důvodů potřebujeme provést požadovanou redukci barevného prostoru.



Obrázek 3.11: Redukce barevného prostoru na 256 stupňů šedi

3.3.1 GrayScale

První možností je redukce plně barevného 24 bit modelu na stupně šedi. Stupně šedi jsou většinou vzorkovány na 8 bitů, což dává standardních 256 úrovní šedi (viz. obr. 3.11). Pro převod se používá empirický vztah 3.1.

$$I = 0.299R + 0.587G + 0.114B \quad (3.1)$$

3.3.2 Dithering/Halftoning

Další možností (stupněm) je redukce 256 stupňů šedi, na dvě barvy, černou a bílou (black&white). Tento převod je možné v zásadě provést dvěma způsoby, jako Dithering nebo Halftoning. Většina následujících metod je založena na ditheringu. Halftoning je možné realizovat metodou maticového rozptýlení (viz. kapitola 3.3.5).

Dithering je způsob převodu stupňů šedi (256) na černou a bílou barvu nahrazením původního rozsahu hodnot pixelů obrazu vhodnou distribucí černých a bílých bodů, které mají integrálně vizuálně maximálně odpovídající podobu. Nedochozí při tom ke zvětšení rozlišení obrazu. Význam ditheringu je v při zobrazení na monitorech a displejích, které mají omezené rozlišení.

Halftoning je způsob převodu stupňů šedi (256) na černou a bílou barvu nahrazením každého původního pixelu s rozsahem hodnot vhodnou distribucí černých a bílých bodů, které mají integrálně vizuálně maximálně odpovídající podobu. Dochází při tom ke zvětšení rozlišení obrazu. Význam halftoningu je především při tisku, kde je ve srovnání s monitory mnohem větší rozlišení. Můžeme proto kompenzovat ztrátu barevné informace zvětšením rozlišení.

3.3.3 Prahování

Jedná se o metodu převodu obrazu ve stupních šedi na černou a bílou. Tato metoda je založena na porovnání všech pixelů obrazu s danou hodnotou prahem

DEF

DEF



Obrázek 3.12: Redukce barevného prostoru na černou a bílou prahováním

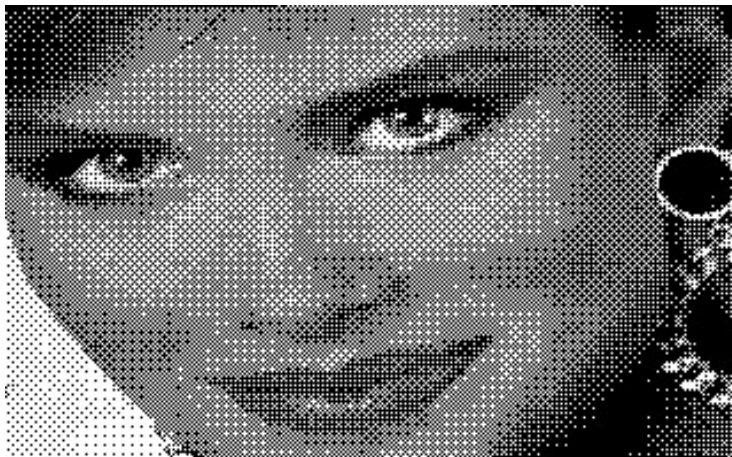


Obrázek 3.13: Redukce barevného prostoru na černou a bílou náhodným rozptýlením

(threshold). Všechny pixely, které mají hodnotu pod daným prahem, budou černé, ostatní bílé (viz. obr. 3.12). Je to metoda jednoduchá a rychlá, ale relativně destruktivní pro výsledný obraz. Pro určité typy obrazů, s výrazným kontrastem oblastí, však může dávat uspokojivé výsledky.

3.3.4 Náhodné rozptýlení

Jedná se opět o metodu převodu obrazu ve stupních šedi na černou a bílou. Tato metoda je založena na porovnání všech pixelů obrazu s hodnotou prahu, která je pro každý pixel vždy znova náhodně generována. Rozsah náhodného generátoru odpovídá rozsahu hodnot obrazu. Pixely, které mají hodnotu pod daným náhodným prahem, budou opět černé, ostatní bílé (viz. obr. 3.13). Je to opět metoda jednoduchá a rychlá, která se velmi osvědčuje pro obrazy s velkými konstantními plochami nebo pozvolnými barevnými přechody.



Obrázek 3.14: Redukce barevného prostoru na černou a bílou maticovým rozptýlením jako dithering

3.3.5 Maticové rozptýlení

Jedná se o další metodu převodu obrazu ve stupních šedi na černou a bílou. Tato metoda je založena na porovnání pixelů obrazu s odpovídajícími hodnotami vhodné distribuční matice (viz. vztahy 3.2). Pixely, které mají hodnotu pod příslušnou hodnotou matice budou černé, ostatní bílé. Je možné tuto metodu aplikovat jako dithering nebo halftoning.

V případě halftoningu probíhá aplikace maticového rozptýlení "přiložením" distribuční matice do každého pixelu obrazu. Každý pixel je tak rozdělen na více pixelů podle rozlišení použité distribuční matice (viz. obr. 3.15).

V případě ditheringu probíhá aplikace maticového rozptýlení "opakovaným přiložením" (dlažba) distribuční matice na celý obraz. Rozlišení obrazu se tak nemění (viz. obr. 3.15).

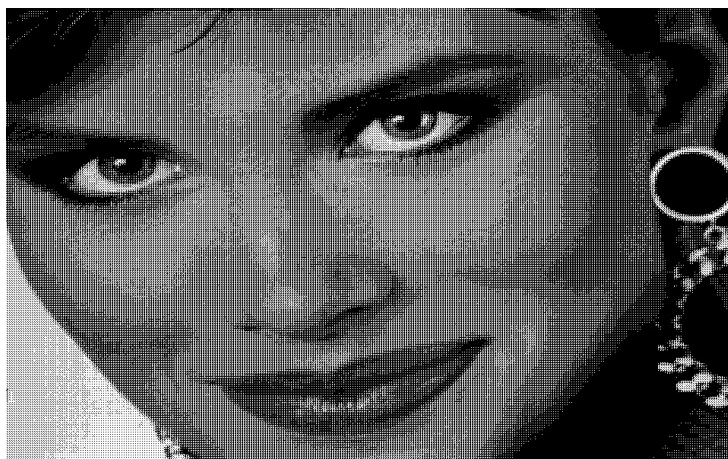
$$M_D = \begin{pmatrix} 0 & 12 & 3 & 15 \\ 4 & 8 & 11 & 7 \\ 2 & 14 & 1 & 13 \\ 10 & 6 & 9 & 5 \end{pmatrix}, \quad M_P = \begin{pmatrix} 1 & 5 & 9 & 2 \\ 8 & 12 & 13 & 6 \\ 4 & 15 & 14 & 10 \\ 0 & 11 & 7 & 3 \end{pmatrix} \quad (3.2)$$

3.3.6 Distribuce chyby

Jedná se o poslední uvedenou metodu převodu obrazu ve stupních šedi na černou a bílou (viz. obr. 3.16). Tato metoda je v základu založena na metodě prahování. K tomu je přidána distribuce vznikajících chyb okolním, dosud nezpracovaným pixelům podle různých schémat (Floyd-Steinberg, viz. obr. 3.17). Je-li aktuální pixel převeden na černý, pak se vzniklá chyba rovná původní hodnotě pixelu. Je-li aktuální pixel převeden na bílou, pak se vzniklá chyba rovná rozdílu maximální hodnoty obrazu a původní hodnoty pixelu.

3.3.7 Paleta 332

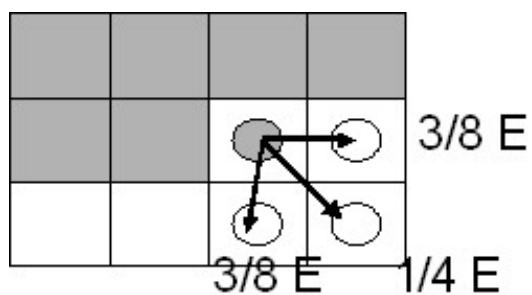
Paleta 332 je jedinou uvedenou metodou pro redukci počtu barev plně barevného RGB obrazu. Tato metoda je založena na převzorkování standardního RGB 24 bit rozsahu, kdy máme na každý barevný kanál 8 bitů. V paletě 332 máme 3 bity na



Obrázek 3.15: Redukce barevného prostoru na černou a bílou maticovým rozptýlením jako halftoning, s rozměrem upraveným na původní velikost



Obrázek 3.16: Redukce barevného prostoru na černou a bílou rozptýlením s distribucí chyby



Obrázek 3.17: Floyd-Steinberg schéma distribuce chyby



Obrázek 3.18: Redukce barevného prostoru barevného RGB obrazu zmenšením počtu barev paletou 332

zelenou a červenou barvu a 2 bity na modrou barvu (viz. obr. 3.18). Celkově tak dostaneme 8 bitový rozsah celého obrazu. Při relativně rovnoměrném rozložení barev v obrazu dává tato metoda poměrně dobré výsledky.

Shrnutí

Náš svět je barevný a proto v oblasti počítačové grafiky musíme umět s barvou pracovat (definovat, popisovat, převádět, analyzovat, atd.). Barva je při tom náš subjektivní vjem vyvolaný působením světla šířeného ze zdroje, které se odráží od povrchu objektu a dopadá na sítnici našeho oka. V mozku je pak informace interpretována jako barva odpovídající charakteristickým rozsahům vlnových délek světla.

Naopak barvu můžeme syntetizovat mícháním světla určité vlnové délky (barvy) z různých zdrojů. Probíhá-li syntéza barev skládáním světla, jedná se o aditivní míchání barev. Jde-li o skládání pigmentů, jedná se o subtraktivní míchání barev.

Pro práci s barvou máme k dispozici barevné modely. RGB model je základním interním barevným modelem v oblasti počítačové grafiky, který odpovídá aditivnímu míchání barev. Model CMY představuje subtraktivní míchání barev a používá se hlavně pro oblast tisku. Mezinárodní model CIE (chromatický diagram) je kalibračním nástrojem, který umožňuje skládat a srovnávat vlastnosti různých zařízení a výstupů. Pro intuitivní uživatelské použití jsou pak určeny modely jako HSV a HLS.

RGB model se používá v rozsahu 8 bitů na každý barevný kanál, což dává celkově 24 bitů (16 miliónů barev). V mnoha případech však použité výstupní zařízení nemá k dispozici takový barevný rozsah. Proto musíme provést redukci barevného prostoru. Prvním stupněm této redukce je převod RGB plně barevných dat na stupně šedi (většinou 256 úrovní) přepočtem na intenzitu barvy. Dalším stupněm je převod stupňů šedi na dvě úrovně, černou a bílou. V případě požadavku na zachování barevnosti obrazu je možné omezit počet barev, podvzorkováním nebo shlukováním barev.



Kontrolní otázky



1. Co je to barva?
2. Co je podstatou aditivního míchání barev?
3. Co je podstatou subtraktivního míchání barev?
4. Jaká barva vznikne smícháním maximálních intenzit základních složek při aditivním míchání barev?
5. Jaká barva vznikne smícháním maximálních intenzit základních složek při subtraktivním míchání barev?
6. Jaký je vztah barevných modelů RGB a CMY?
7. Jaký je vztah barevných modelů CMY a CMYK?
8. Jaký význam má použití barevných modelů HSV a HLS?
9. Nakreslete schéma barevných modelů RGB a CMY, v čem se liší?
10. Nakreslete schéma barevného modelu HSV.
11. Nakreslete schéma barevného modelu HLS.
12. Jaký význam má chromatický diagram (CIE)?
13. Proč je potřeba provádět redukci barevného modelu?
14. Jakým způsobem probíhá redukce barevného obrazu na stupně šedi?
15. Co je podstatou ditheringu?
16. Co je podstatou halftoningu?
17. Co je můžeme realizovat halftoning?
18. Jak pracuje redukce barevného prostoru prahováním?
19. Jak pracuje redukce barevného prostoru náhodnou distribucí?
20. Jak pracuje redukce barevného prostoru maticovou distribucí?
21. Jak pracuje redukce barevného prostoru distribucí chyby?
22. Co je podstatou redukce barevného prostoru paletou 332?

Kapitola 4

Rasterizace vektorových entit ve 2D



1:00



Cílem této kapitoly je seznámit se s hlavními algoritmy používanými pro převod základních vektorových prvků (entit) na rastrové zobrazení. Konkrétně se budeme zabývat úsečkou, kružnicí a elipsou.

Jak již bylo řečeno dříve (viz. kapitola XXX), takřka všechny druhy v současné době běžných výstupních grafických zařízení pracují na principu rastrové grafiky (monitory, displeje, tiskárny, ...). Většina grafických aplikací je však založena na principu vektorové grafiky (zobrazení textů, uživatelská rozhraní, velká část grafických aplikací, ...).

Jsme tedy v situaci, kdy chceme (potřebujeme) zobrazovat vektorovou grafickou informaci na rastrovém grafickém zobrazovacím zařízení. Proto musíme převést vektorový popis naší scény (obrazu) na rastrovou informaci, které je již dále bez problému přímo zobrazitelná.

Logickým požadavkem při tom je, aby výsledný rastrový obraz vypadal pokud možno stejně, jako původní vektorová data. Dosažení tohoto požadavku samozřejmě závisí na fyzickém rozlišení výstupního rastrového grafického zařízení a na rozlišení našeho oka. V ideálním případě požadujeme, aby nebylo možné prostým okem rozeznat použitý princip zobrazení nebo datové reprezentace.

Druhým logickým požadavkem je maximální možná rychlost zobrazení. Abychom mohli realizovat interaktivní grafické aplikace pracující interně s vektorovými daty, musí proces rasterizace probíhat v reálném čase. Pro dosažení tohoto požadavku se používají celočíselné algoritmy. To plyne již z diskrétní podstaty rastrové grafiky. S tím úzce souvisí také implementace použitým algoritmy přímo do hardware grafických karet. Hardwarová implementace je pro celočíselné výpočty samozřejmě mnohem jednodušší a rychlejší.

Rasterizace je proces převodu vektorové reprezentace dat na jejich rastrovou formu s cílem dosáhnout maximální možnou kvalitu a zároveň rychlost výsledného zobrazení.

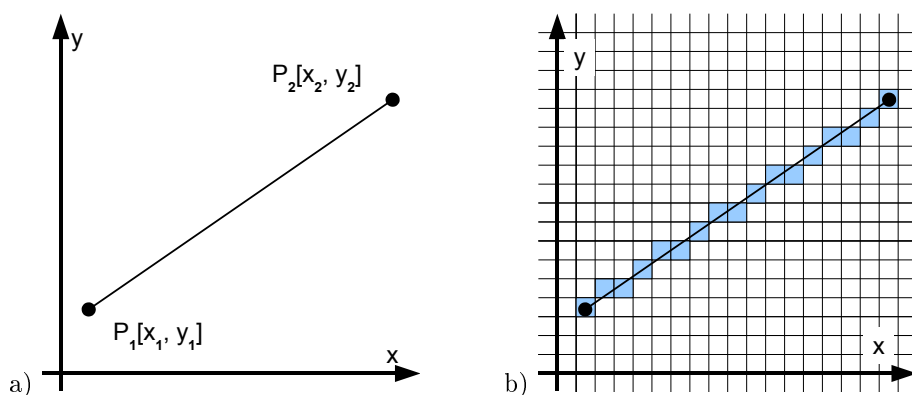


Nejčastěji používaným typem vektorových entit jsou bezesporu úsečky nebo objekty z úseček složené (lomené čáry ...), ~90%. Dále jsou to kružnice nebo elipsy, ~5%. Zbývající podíl připadá na ostatní typy entit, křivky, polygony atd. Z tohoto důvodu se budeme v této kapitole dále zabývat pouze rasterizací úseček, kružnic a elips. Neřešíme při tom tloušťky čar, jejich napojení a tvar ukončení.

4.1 Rasterizace úsečky

Úsečka je základní geometrická vektorová entita definovaná souřadnicemi dvou koncových bodů (viz. obr. 4.1). Geometrie úsečky je mate-





Obrázek 4.1: Definice úsečky ve 2D prostoru: a) v kartézském souřadném systému, b) rasterizace téže úsečky.

maticky popsána rovnicí přímky, která může být podle potřeby v obecném (viz. rovnice 4.1), parametrickém (viz. rovnice 4.2) nebo směrnicovém (viz. rovnice 4.3) tvaru.

$$Ax + By + C = 0, \quad A = (y_2 - y_1), \quad B = (x_2 - x_1) \quad (4.1)$$

$$x = x_1 + t(x_2 - x_1), \quad y = y_1 + t(y_2 - y_1) \quad (4.2)$$

$$y = kx + q, \quad k = \frac{\Delta y}{\Delta x} = \frac{(y_2 - y_1)}{(x_2 - x_1)} \quad (4.3)$$

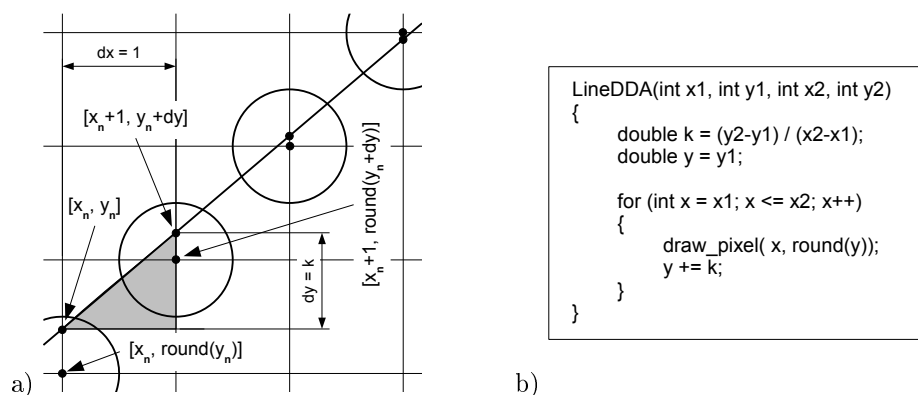
Následující popisované algoritmy pro vykreslení úsečky jsou odvozeny pro případ, kdy úsečka leží v prvním kvadrantu, je rostoucí od počátečního bodu P_1 ke koncovému P_2 a nejrychleji roste ve směru osy X. Ostatní případy polohy úsečky je potřeba převést na tento případ (výměna souřadnic, os ...)



4.1.1 DDA algoritmus

Jedná se o jeden z prvních a nejjednodušších algoritmů na rasterizaci úsečky. Pro výpočty používá výpočty s desetinnými čísly (floating-point aritmetiku) a proto není příliš efektivní a snadno implementovatelný v hardware. DDA (digital differential analyser) je historický název elektronického zařízení pro diferenciální výpočty. Rasterizace úsečky tímto algoritmem je pak speciálním případem DDA interpolátoru. Následující krok výpočtu provádíme na základě předchozích výsledků. Pohybujeme se při tom po malých krocích, diferencích.

Úsečku budeme vykreslovat postupně po jednotlivých pixelech ve směru od počátečního bodu P_1 ke koncovému P_2 podle rekurentních vztahů 4.4 a 4.5. Ve směru osy X postupujeme po jednotlivých pixelech s přírůstkem $dx = 1$. Ve směru osy Y je přírůstek dán velikostí směrnice úsečky, která je obecně reálné číslo. Pro vykreslení jednotlivých pixelů však potřebujeme celočíselné souřadnice. Výslednou souřadnici v ose Y musíme proto zaokrouhlit na nejbližší celé číslo. Schéma principu algoritmu DDA a jeho zjednodušenou implementaci najdeme na obrázku 4.2.



Obrázek 4.2: Algoritmus DDA pro vykreslení úsečky: a) schéma principu algoritmu, b) zjednodušená implementace.

```

LineEC(int x1, int y1, int x2, int y2)
{
    double k = (y2-y1) / (x2-x1);
    double E = 0;
    int y = y1;

    for (int x = x1; x <= x2; x++)
    {
        draw_pixel( x, y);
        E += k;
        if (E >= 0.5) { y++; E -= 1; }
    }
}

```

Obrázek 4.3: Zjednodušená implementace algoritmu pro vykreslení úsečky s hlídáním chyby.

$$x_{n+1} = x_n + dx, \quad dx = 1 \quad (4.4)$$

$$y_{n+1} = y_n + dy, \quad dy = k = \frac{(y_2 - y_1)}{(x_2 - x_1)} \quad (4.5)$$

4.1.2 Vykreslování úsečky s hlídáním chyby

Algoritmus pro vykreslení úsečky s hlídáním chyby je variací na dříve uvedený algoritmus DDA. Rozdíl je pouze v tom, že se neprovádí výpočet skutečné souřadnice v ose Y, kterou posléze zaokrouhlíme. Pracujeme s hodnotou relativní odchylky (chyby) aktuální celočíselné souřadnice od skutečné souřadnice v ose Y. Jestliže tato chyba překročí hodnotu 0.5, přesuneme se na další řádek, na další celočíselnou souřadnici v ose Y. Po každém posunu na nový řádek je hodnota chyby snížena o hodnotu 1.0, abychom stále pracovali s relativní odchylkou od aktuálního řádku. Zjednodušenou implementaci tohoto algoritmu najdeme na obrázku 4.3.

4.1.3 Bresenhamův algoritmus

Jde o dnes nepoužívanější a nejefektivnější algoritmus pro rasterizaci úsečky publikovaný J.E. Bresenhamem v roce 1965. Pracuje s celými čísly a používá při výpo-

čtech pouze operace sčítání, odčítání a porovnání. Jeho implementace v hardware je relativně snadná a proto ho nejdeme prakticky ve všech rastrových grafických zařízeních.

Úsečku budeme opět vykreslovat postupně po jednotlivých pixelech ve směru od počátečního bodu P_1 ke koncovému P_2 . Ve směru osy X postupujeme po jednotlivých pixelech s přírůstkem $dx = 1$. O posunu ve směru osy Y rozhodujeme podle znaménka, nikoli hodnoty, prediktoru p_i .

Při odvození prediktoru vycházíme ze směrnice tvaru rovnice úsečky (viz. rovnice 4.3). Předpokládejme, že jsme v celočíselné pozici $[x_i, y_i]$ a rozhodujeme, jestli následující pixel leží na pozici $[x_i + 1, y_i]$ nebo $[x_i + 1, y_i + 1]$. Samozřejmě nás zajímá ta pozice, která je nejbližší skutečné pozici přímky $[x_i + 1, y_{i+1}]$ (viz. rovnice 4.6). Proto si vyjádříme vzdálenosti uvažovaných pozic d_1 a d_2 $[x_i + 1, y_i]$ a $[x_i + 1, y_i + 1]$ od skutečné pozice přímky $[x_i + 1, y_{i+1}]$ (viz. rovnice 4.7 a 4.8). Abychom nemuseli porovnávat jejich hodnoty, vyjádříme si jejich rozdíl Δd , u kterého bude rozhodující pouze znaménko (viz. rovnice 4.9). Hodnota Δd je však stále reálné číslo, kvůli hodnotě směrnice přímky úsečky k . Reálné hodnoty k se proto zbavíme vynásobením celého vztahu pro Δd hodnotou Δx ($k = \Delta y / \Delta x$) (viz. rovnice 4.10).

$$y_{i+1} = kx_{i+1} + q = k(x_i + 1) + q \quad (4.6)$$

$$d_1 = y_{i+1} - y_i = k(x_i + 1) + q - y_i \quad (4.7)$$

$$d_2 = y_i + 1 - y_{i+1} = y_i + 1 - k(x_i + 1) - q \quad (4.8)$$

$$\Delta d = d_1 - d_2 = 2k(x_i + 1) - 2y_i + 2q - 1 \quad (4.9)$$

$$p_i = \Delta d \Delta x = 2\Delta y x_i - 2\Delta x y_i + 2\Delta y + \Delta x(2q - 1) \quad (4.10)$$

Jestliže je hodnota prediktoru $p_i < 0$, pak je hodnota vzdálenosti $d_2 > d_1$ a nový pixel leží na aktuální úrovni y_i . Pokud je však hodnota prediktoru $p_i \geq 0$, pak je hodnota vzdálenosti $d_2 \leq d_1$ a nový pixel se posune na další úroveň $y_i + 1$ (viz. rovnice 4.11 a 4.12).

$$p_i < 0 \implies y_{i+1} = y_i \quad (4.11)$$

$$p_i \geq 0 \implies y_{i+1} = y_i + 1 \quad (4.12)$$

Pro průběžné výpočty během vykreslování úseček bude výhodnější vyjádřit prediktor rekurentně, tedy na základě hodnoty z předchozího kroku (viz. rovnice 4.13). Výsledný rekurentní vztah pro prediktor bude samozřejmě záležet nejen na hodnotě předchozího prediktoru, ale také na jeho znaménku (viz. rovnice 4.14 a 4.15).

$$p_{i+1} = 2\Delta y x_{i+1} - 2\Delta x y_{i+1} + 2\Delta y + \Delta x(2q - 1) \quad (4.13)$$

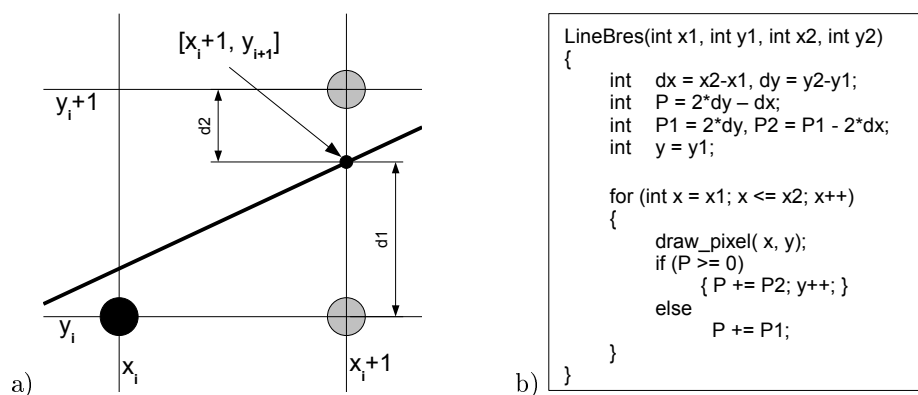
$$p_{i+1} = p_i + 2\Delta y \iff p_i < 0 \quad (4.14)$$

$$p_{i+1} = p_i + 2\Delta y - 2\Delta x \iff p_i \geq 0 \quad (4.15)$$

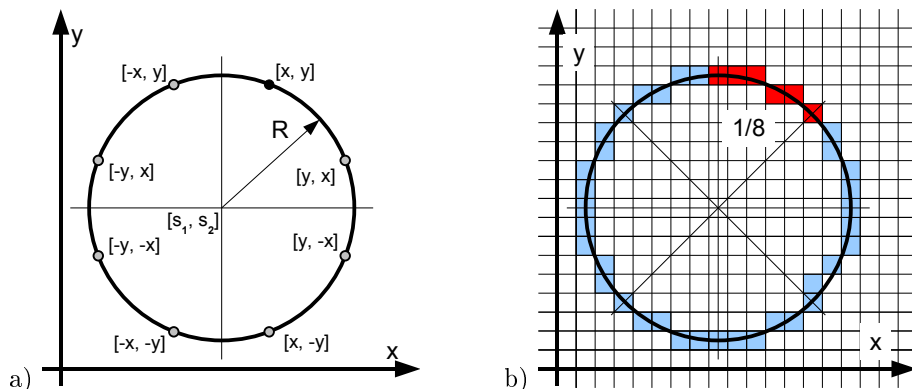
Pro vlastní výpočty během rasterizace úsečky jsou rozhodující rozhodovací vztahy 4.11 a 4.12 a vztahy pro rekurentní výpočet prediktoru 4.14 a 4.15. Startovací hodnota prediktoru je:

$$p_i = 2\Delta y - \Delta x$$

Schéma principu tohoto algoritmu a jeho zjednodušenou implementaci najdeme na obrázku 4.4.



Obrázek 4.4: Bresenhamův algoritmus pro vykreslení úsečky: a) schéma principu algoritmu, b) zjednodušená implementace.



Obrázek 4.5: Definice kružnice ve 2D prostoru: a) v kartézském souřadném systému, b) rasterizace těže kružnice.

4.2 Rasterizace kružnice

Kružnice je základní geometrická vektorová entita definovaná souřadnicemi středu a hodnotou poloměru (viz. obr. 4.5). Geometrie kružnice je matematicky popsána rovnicí kružnice (viz. rovnice 4.16).

$$F(x, y) : (x - s_1)^2 + (y - s_2)^2 = R^2 \quad (4.16)$$

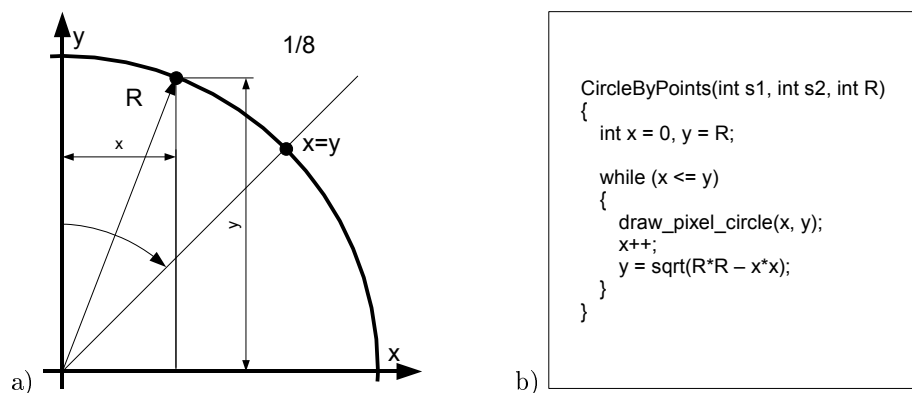
Kružnice je osminásobně symetrická. Stačí tedy vypočítat pouze její 1/8 bodů v 1/2 prvního kvadrantu a zbývající body získat záměnou souřadnic. Výpočty je výhodnější provádět pro kružnici se středem v počátku $[0, 0]$. Následující popisované algoritmy jsou proto odvozeny pro tento případ.

4.2.1 Vykreslení kružnice po bodech

Jedná se o naivní, nejjednodušší způsob vykreslení kružnice, který pracuje s desetinnými čísly a je výpočetně relativně náročný (odmocnina). Vztah 4.17 pro výpočet souřadnic v ose Y můžeme odvodit z rovnice kružnice 4.16. V ose X se pohybujeme ve směru hodinových ručiček po jenom pixelu od hodnoty 0 do okamžiku, kdy se souřadnice $x = y$ (pro úhel 45° , kdy je možné použít symetrie).

DEF





Obrázek 4.6: Algoritmus pro vykreslení kružnice po bodech: a) schéma principu algoritmu, b) zjednodušená implementace.

Vypočtené souřadnice v ose Y se samozřejmě musejí zaokrouhlovat na nejbližší celá čísla.

$$y = \sqrt{R^2 - x^2} \quad (4.17)$$

Schéma principu tohoto algoritmu a jeho zjednodušenou implementaci najdeme na obrázku 4.6.

4.2.2 Vykreslení kružnice jako n-úhelník

Jde o variantu algoritmu typu DDA (viz. kapitola 4.1.1) pro kružnici. Pracuje tedy s desetinnými čísly a není proto příliš efektivní a snadno implementovatelný v hardware.

Kružnici budeme vykreslovat jako N-úhelník pomocí úseček. Vycházíme při tom z rotačních transformací bodů (viz. kapitola 7). Prostřednictvím rekurentních vztahů 4.18 a 4.19 budeme počítat souřadnice X, Y pro daný konstantní přírůstek úhlu α ($\sim 1/R$). Hodnoty $\cos \alpha$ a $\sin \alpha$ mohou být předpočítány dopředu pouze jednou jako konstanty. Výsledné souřadnice musíme opět zaokrouhlit na nejbližší celá čísla.

Algoritmus, tak jak je popsán, je citlivý na kumulované numerické chyby výpočtu funkcí $\sin$ a $\cos$. Pro odstranění tohoto vlivu je možné počítat pouze $1/8$ kružnice a vykreslovat s využitím symetrie nebo použít "High precision" variantu algoritmu, která pro výpočet $\sin$ a $\cos$ využívá Taylorova rozvoje.

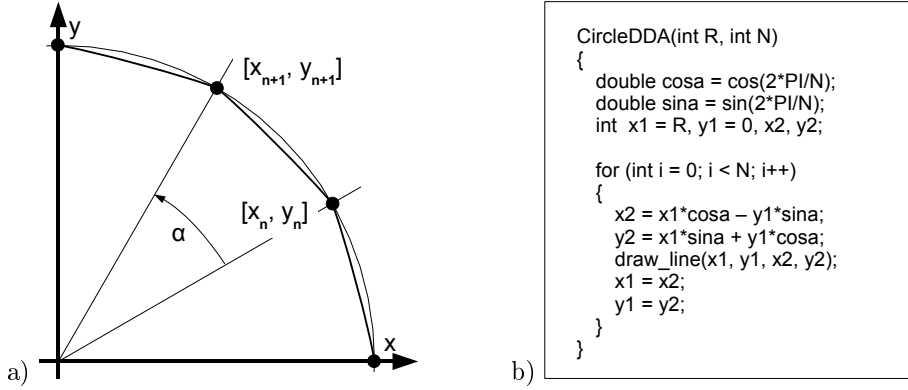
$$x_{n+1} = x_n \cos \alpha - y_n \sin \alpha \quad (4.18)$$

$$y_{n+1} = x_n \sin \alpha + y_n \cos \alpha \quad (4.19)$$

Schéma principu tohoto algoritmu a jeho zjednodušenou implementaci najdeme na obrázku 4.7.

4.2.3 Midpoint algoritmus

Pro vykreslování kružnice je možné odvodit Bresenhamův algoritmus stejně jako pro úsečku (viz. kapitola 4.1.3). Prakticky je však pro vykreslení kružnice více používán "Midpoint" algoritmus. Jeho princip je v podstatě stejný s Bresenhamovým algoritmem. Rozdíl je ve stanovení kritéria pro určení prediktora.



Obrázek 4.7: Algoritmus pro vykreslení kružnice jako n-úhelník: a) schéma principu algoritmu, b) zjednodušená implementace.

Vlastnosti tohoto algoritmu jsou stejné jako u Bresenhama, pracuje tedy s celými čísly a používá při výpočtech pouze operace sčítání, odčítání a porovnání. Jeho implementace v hardware je proto relativně snadná a často používaná.

Kružnici budeme vykreslovat postupně po jednotlivých pixelech ve směru hodinových ručiček pouze pro její 1/8. Zbývající body získáme symetrii. Ve směru osy X se pohybujeme od hodnoty 0 do okamžiku, kdy se souřadnice $x = y$ (pro úhel 45°), s přírůstkem $dx = 1$. Ve směru osy Y klesáme od hodnoty R. O posunu ve směru osy Y rozhodujeme opět podle znaménka, nikoli hodnoty, prediktoru p_i . Jako prediktor použijeme rovnici kružnice 4.16 v jejím homogenním tvaru 4.20.

$$F(x, y) : x^2 + y^2 - R^2 = 0 \quad (4.20)$$

Předpokládejme, že jsme v celočíselné pozici $[x_i, y_i]$ a rozhodujeme, jestli následující pixel leží na pozici $[x_i + 1, y_i]$ nebo $[x_i + 1, y_i - 1]$. Toto rozhodnutí provedeme podle polohy středního bodu $[x_i + 1, y_i - 1/2]$ (midpointu) vůči kružnici (uvnitř nebo vně). Midpoint dosadíme do vztahu pro prediktor 4.21 4.22.

$$p_i = F(x_i + 1, y_i - \frac{1}{2}) \quad (4.21)$$

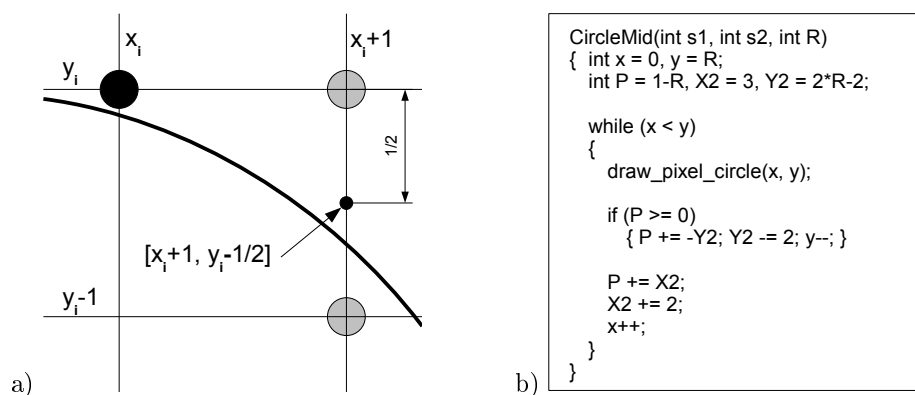
$$p_i = (x_i + 1)^2 + (y_i - \frac{1}{2})^2 - R^2 \quad (4.22)$$

Jestliže je hodnota prediktoru $p_i < 0$, pak je midpoint uvnitř kružnice a nový pixel leží na aktuální úrovni y_i . Pokud je však hodnota prediktoru $p_i \geq 0$, pak je midpoint vně kružnice a nový pixel se posune na další úroveň $y_i - 1$ (viz. rovnice 4.23 a 4.24).

$$p_i < 0 \implies y_{i+1} = y_i \quad (4.23)$$

$$p_i \geq 0 \implies y_{i+1} = y_i - 1 \quad (4.24)$$

Pro průběžné výpočty během vykreslování kružnice bude opět výhodnější vyjádřit prediktor rekurentně, tedy na základě hodnoty z předchozího kroku (viz. rovnice 4.25). Výsledný rekurentní vztah pro prediktor bude opět samozřejmě záležet nejen na hodnotě předchozího prediktoru, ale také na jeho znaménku (viz. rovnice 4.26 a 4.27).



Obrázek 4.8: Midpoint algoritmus pro vykreslení kružnice: a) schéma principu algoritmu, b) zjednodušená implementace.

$$p_{i+1} = (x_{i+1} + 1)^2 + (y_{i+1} - \frac{1}{2})^2 - R^2 \quad (4.25)$$

$$p_{i+1} = p_i + 2x_i + 3 \quad \Leftarrow \quad p_i < 0 \quad (4.26)$$

$$p_{i+1} = p_i + 2x_i - 2y_i + 5 \quad \Leftarrow \quad p_i \geq 0 \quad (4.27)$$

Pro vlastní výpočty během rasterizace kružnice jsou rozhodující rozhodovací vztahy 4.23 a 4.24 a vztahy pro rekurentní výpočet prediktoru 4.26 a 4.27.

Startovací hodnota prediktoru je:

$$p_i = 1 - R \quad (4.28)$$

Schéma principu tohoto algoritmu a jeho zjednodušenou implementaci najdeme na obrázku 4.8.

4.3 Rasterizace elipsy

Elipsa je základní geometrická vektorová entita definovaná souřadnicemi středu a délkami hlavní a vedlejší poloosy, případně jejich natočením (viz. obr. 4.9). Geometrie elipsy je matematicky popsána rovnicí elipsy (viz. rovnice 4.29), která platí pro elipsu s poloosami rovnoběžnými se souřadnými osami.

$$F(x, y) : b^2(x - s_1)^2 + a^2(y - s_2)^2 = a^2b^2 \quad (4.29)$$

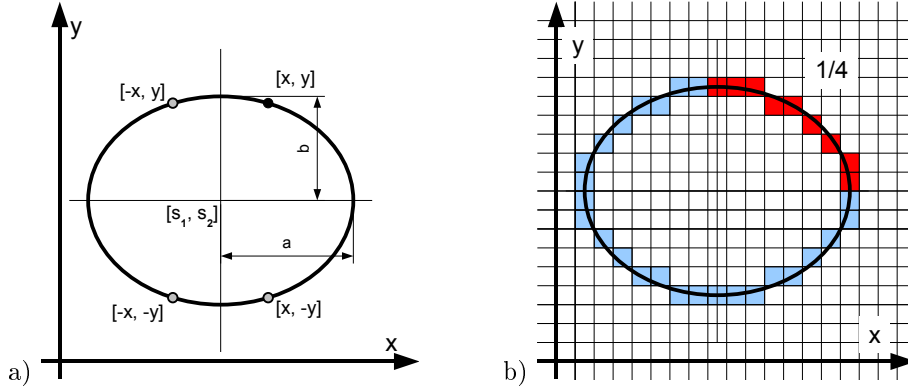
Elipsa je čtyřnásobně symetrická. Stačí tedy vypočítat pouze její 1/4 bodů prvního kvadrantu a zbývající body získat záměnou souřadnic. Výpočty je výhodnější provádět pro elipsu se středem v počátku [0, 0] a poloosami rovnoběžnými se souřadnými osami. Následující popisované algoritmy jsou proto odvozeny pro tento případ.

4.3.1 Midpoint algoritmus

Na elipsu je možné teoreticky aplikovat stejné metody vykreslování jako na kružnici. V případě elipsy se však budeme zabývat pouze "Midpoint"algoritmem. Princip a vlastnosti tohoto algoritmu pro vykreslení elipsy jsou v podstatě stejné jako

DEF





Obrázek 4.9: Definice elipsy ve 2D prostoru: a) v kartézském souřadném systému, b) rasterizace téže elipsy.

pro kružnici. Zjednodušenou implementaci tohoto algoritmu najdeme na obrázku 4.10 b.

Protože je elipsa pouze 4x symetrická, musíme vypočítat body pro celou její 1/4. Zbývající body získáme symetrií. Během výpočtů bodů 1/4 se však mění velikost parciálních derivací v ose X a v ose Y (viz. vztahy 4.30). Proto rozdělíme 1/4 elipsy na dvě oblasti (viz. obr. 4.10 a). V oblasti I je větší parciální derivace v ose Y a řídící osou proto bude osa X ($x_{i+1} = x_i + 1$). V oblasti II je větší parciální derivace v ose X a řídící osou proto bude osa Y ($y_{i+1} = y_i - 1$). Přechod nastává v okamžiku rovnosti parciálních derivací, kdy má směrnice normály elipsy hodnotu 1 (úhel 45°).

$$\frac{\partial F(x, y)}{\partial x} = 2b^2x \quad , \quad \frac{\partial F(x, y)}{\partial y} = 2a^2y \quad (4.30)$$

Odvození vztahů pro vykreslení elipsy odpovídá odvození vztahů pro kružnici. Vycházíme při tom však z homogenní rovnice elipsy 4.31. Vztahy 4.32 až 4.38 platí pro oblast I.

$$F(x, y) : \quad b^2x^2 + a^2y^2 - a^2b^2 = 0 \quad (4.31)$$

$$p_i = F(x_i + 1, y_i - \frac{1}{2}) \quad (4.32)$$

$$p_i = b^2(x_i + 1)^2 + a^2(y_i - \frac{1}{2})^2 - a^2b^2 \quad (4.33)$$

$$p_i < 0 \implies y_{i+1} = y_i \quad (4.34)$$

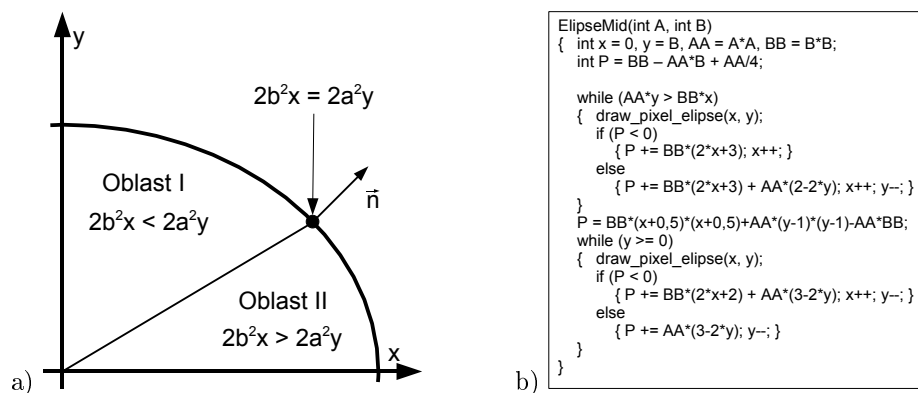
$$p_i \geq 0 \implies y_{i+1} = y_i - 1 \quad (4.35)$$

$$p_{i+1} = b^2(x_{i+1} + 1)^2 + a^2(y_{i+1} - \frac{1}{2})^2 - a^2b^2 \quad (4.36)$$

$$p_{i+1} = p_i + b^2(2x_i + 3) \iff p_i < 0 \quad (4.37)$$

$$p_{i+1} = p_i + b^2(2x_i + 3) + a^2(2 - 2y_i) \iff p_i \geq 0 \quad (4.38)$$

Vztahy 4.39 až 4.45 platí pro oblast II.



Obrázek 4.10: Midpoint algoritmus pro vykreslení elipsy: a) schéma oblastí elipsy, b) zjednodušená implementace.

$$p_i = F\left(x_i + \frac{1}{2}, y_i - 1\right) \quad (4.39)$$

$$p_i = b^2\left(x_i + \frac{1}{2}\right)^2 + a^2(y_i - 1)^2 - a^2b^2 \quad (4.40)$$

$$p_i \geq 0 \implies x_{i+1} = x_i \quad (4.41)$$

$$p_i < 0 \implies x_{i+1} = x_i + 1 \quad (4.42)$$

$$p_{i+1} = b^2\left(x_{i+1} + \frac{1}{2}\right)^2 + a^2(y_{i+1} - 1)^2 - a^2b^2 \quad (4.43)$$

$$p_{i+1} = p_i + a^2(3 - 2y_i) \iff p_i \geq 0 \quad (4.44)$$

$$p_{i+1} = p_i + b^2(2x_i + 2) + a^2(3 - 2y_i) \iff p_i < 0 \quad (4.45)$$

Startovací hodnota prediktoru je:

$$p_i = b^2 - a^2b + \frac{1}{4}a^2 \quad (4.46)$$

Shrnutí

Rasterizace je jeden z nejdůležitějších kroků v procesu zobrazení vektorových entit na rastrových zobrazovacích zařízeních. Rastrová zařízení dnes představují absolutní většinu. Stejně tak je většina grafických aplikací založena na vektorovém principu. Nejvíce používané vektorové entity jsou: úsečky (~90%), kružnice a elipsy (~5%). Proto musí být jejich rastrovací algoritmy maximálně efektivní a pracující s celočíselnou aritmetikou, především kvůli implementaci do hardware počítačů. Nejefektivnějším algoritmem pro rasterizaci úsečky je Bresenhamův algoritmus. U kružnice je to Midpoint algoritmus. Seznámili jsme se také s dalšími algoritmy, které však pracují s reálnými čísly (DDA pro úsečku a kružnici, ...).



Kontrolní otázky

1. Co je podstatou procesu rasterizace vektorových entit?



2. Proč je důležité, aby rasterizační metody pracovaly maximálně efektivně a s použitím celočíselné aritmetiky?
3. Jaký je základní princip algoritmu DDA pro rasterizaci úsečky?
4. Jaký je rozdíl mezi algoritmem DDA pro úsečku a Vykreslováním úsečky s hlídáním chyby?
5. Jaký je základní princip Bresenhamova algoritmu pro rasterizaci úsečky?
6. Proveďte kompletní odvození vztahů pro Bresenhamův algoritmu rasterizace úsečky.
7. Jaký je základní princip algoritmu pro rasterizaci kružnice po bodech?
8. Jaký je základní princip algoritmu pro vykreslení kružnice jako N-úhelník?
9. Jaký je základní princip Midpoint algoritmu pro rasterizaci kružnice?
10. Proveďte kompletní odvození vztahů pro Midpoint algoritmu rasterizace kružnice.
11. Jaký je základní princip Midpoint algoritmu pro rasterizaci elipsy?
12. Proveďte kompletní odvození vztahů pro Midpoint algoritmu rasterizace elipsy.

Kapitola 5

Vyplňování 2D oblastí



1:00



Cílem této kapitoly je seznámit se s hlavními algoritmy a metodami používanými pro vyplňování uzavřených rovinných oblastí s hranicí definovanou rastrově nebo vektorově. U vektorových oblastí se omezíme na oblasti definované po částech lineárně, tedy na mnohoúhelníky. Vyplňovat budeme konstantní barvou.

Interaktivní zobrazení především ve 3D je v současné době založeno na *polygonálních modelech* (viz. kapitola 9). Objekty jsou v tom případě popsány množinou polygonů, mnohoúhelníků. Většinou se používají sítě trojúhelníků (viz. obr. 5.1). Trojúhelník je při tom nejjednodušší polygon, který je vždy konvexní a rovinný. Proto potřebujeme umět rychle a efektivně zobrazovat vyplněné plošné objekty (oblasti, polygony). Stejně jako u čárových entit máme na tyto metody požadavek rychlosti a kvality zobrazení.

Oblasti máme popsány většinou svou hranicí a řešíme jejich *vyplnění* požadovaným způsobem. Podle způsobu definice hranice oblasti rozlišujeme:

- Rastrové oblasti – hranice je popsána rastrově, v rastrové matici nastavením bodů hranice nebo barvou vyplňované oblasti (viz. obr. ??).
- Vektorové oblasti – hranice je popsána seznamem vektorových entit (úseček, kruhových oblouků, křivek atd. viz. obr. 5.2).

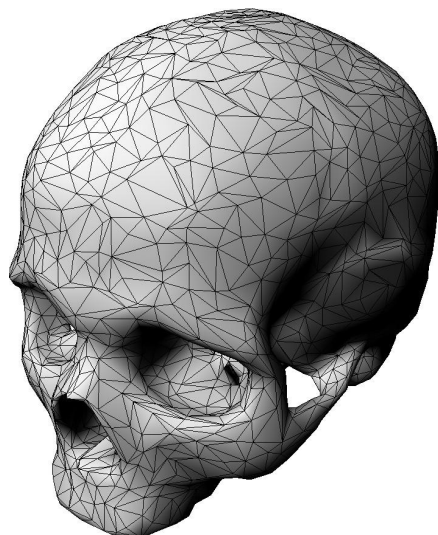
Vyplňování oblastí je proces nalezení a označení (obarvení) všech vnitřních bodů výsledného rastru pro danou oblast. Vstupní informací je popis hranice oblasti. Výstupem je rastrový popis vyplněné oblasti. Jde tedy v podstatě o rasterizaci dané oblasti.



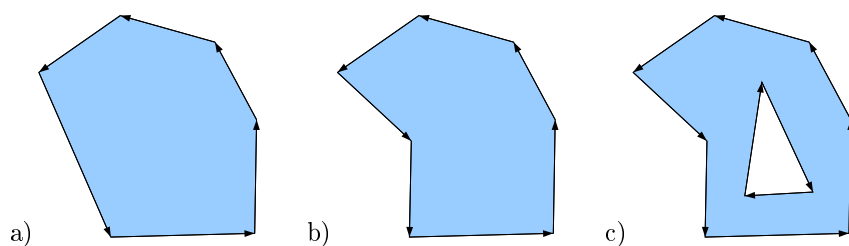
Podle geometrie rozlišujeme oblasti:

- Konvexní (vypouklý, vydutý) – pro libovolné dva body oblasti platí, že jejich spojnice je součástí oblasti, neprotíná její hranice (viz. obr. 5.2a).
- Konkávní (nekonvexní, prohnutý, dutý) – oblast, která není konvexní (viz. obr. 5.2b).
- S vnitřními otvory – hranice je tvořena více nezávislými smyčkami, obrysem a otvory (viz. obr. 5.2c).

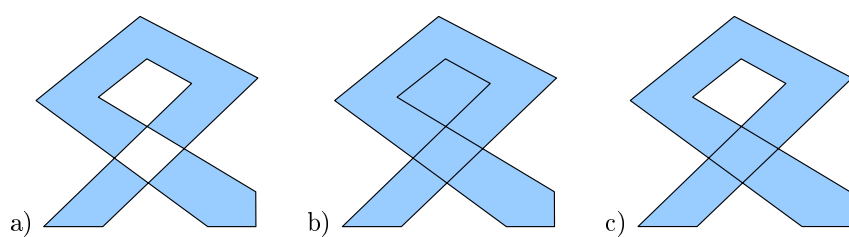
My se z praktických důvodů omezíme na obecné mnohoúhelníky, které jsou popsány množinou hraničních úseček, po částech lineárně. To je zároveň nejčastější způsob definice oblastí. Tyto mnohoúhelníky je možné vyplňovat přímo vhodným algoritmem nebo je můžeme rozdělit na trojúhelníky. Pro trojúhelníky jakožto nejjednodušší polygony pak existují modifikované (optimalizované a zjednodušené) vyplňovací algoritmy (viz. kapitola 5.3).



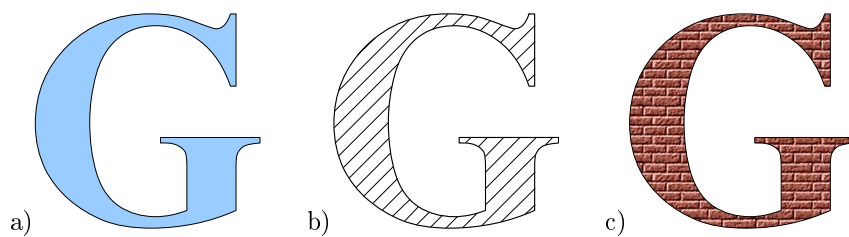
Obrázek 5.1: Polygonální model lebky.



Obrázek 5.2: Geometrie oblastí: a) konvexní, b) konkávní, c) konkávní s otvorem.



Obrázek 5.3: Pravidla vyplňování oblastí: a) paritní, b) vnitřní, c) nenulové.



Obrázek 5.4: Druhy výplně oblastí: a) konstantní barvou, b) šrafováním, c) grafickým vzorem.



Pro korektní definici vektorových oblastí musí být seznam hraničních entit vyplňované oblasti orientovaný a spojitý. Jinak bychom nemohli určit kde je uvnitř a kde vně oblasti. Většinou se pro rozlišení vnitřních a vnějších částí používá *pravidlo pravé ruky*. Prsty ukazují po směru hranice oblasti a dlaň směřuje dovnitř oblasti. Vnitřní díry mají opačnou orientaci.

U jednodušších oblastí, jejíž hranice se neprotínají, je způsob vyplnění většinou jednoznačný. U složitějších oblastí, jejíž hranice protínají samy sebe, však může dojít k určitým nejednoznačnostem. Tyto nejednoznačnosti je možné vyřešit některým z následujících pravidel vyplňování:

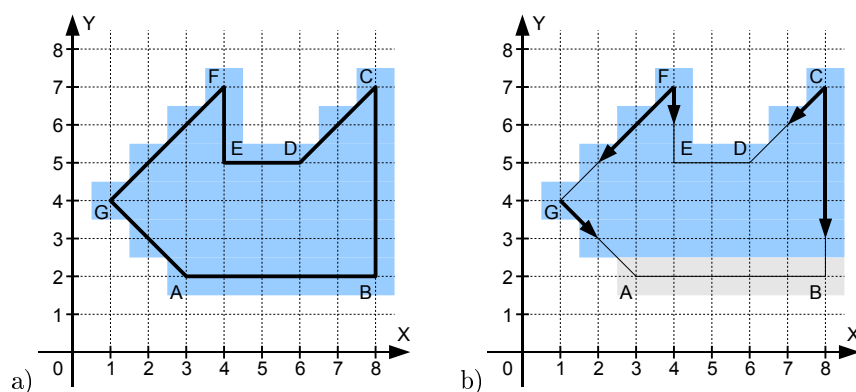
- Paritní vyplňování – hranice odděluje vyplněný a nevyplněný prostor, nejběžnější způsob (viz. obr. 5.3a).
- Vnitřní vyplňování – jsou vyplněny všechny body, které nejsou vně oblasti (viz. obr. 5.3b).
- Nenulové vyplňování – nenulové objetí bodu uzavřenou smyčkou při vyřešení sebeprotínání (viz. obr. 5.3c).

Oblasti můžeme podle potřeby vyplňovat různými druhy výplní:

- Danou konstantní barvou (viz. obr. 5.4a).
- Jednoduchým čárovým vzorem (šrafou) (viz. obr. 5.4b).
- Složitějším obecným grafickým vzorem (viz. obr. 5.4c).

5.1 Řádkové vyplňování

Jedná se o základní algoritmus pro vyplňování obecných mnohoúhelníků (viz. obr. 5.5a). Vyplňovaná oblast Ω je popsána seznamem hraničních entit (hran) $\Omega = \{e_1, e_2, \dots, e_n\}$. V případě po částech lineárních hranic (úseček) nám stačí orientovaný seznam vrcholů úseček.



Obrázek 5.5: Řádkové vyplňování oblasti: a) korektně vyplněná oblast, b) oblast se zkrácenými hranami, šedá část chybně nevyplněna.

DEF

Základní myšlenkou algoritmu *Řádkového vyplňování* je procházení dané oblasti Ω po jednotlivých řádcích výsledného rastru komplexně přes celou oblast. Úseky řádků ležící uvnitř oblasti jsou pak vyplněny barvou nebo vzorem oblasti (viz. obr. 5.5).

Každý řádek $Y_j \in \langle Y_{min}, Y_{max} \rangle$ oblasti Ω je zpracován následujícím způsobem (pro paritní vyplňování):

- Vytvoření seznamu P_j souřadnic x_i průsečíků řádku Y_j se všemi hraničními entitami e_i oblasti Ω . Vodorovné hrany se vynechávají.
- Setřídění seznamu průsečíků P_j podle souřadnic x_i .
- Vykreslení vodorovných úseků řádku Y_j vždy mezi lichými a sudými průsečíky seznamu P_j .

5.1.1 Lichý počet průsečíků řádku

Algoritmus Řádkového vyplňování předpokládá, že počet průsečíků každého řádku je vždy sudý, abychom mohli vyplňovat vždy mezi lichým a sudým. To však nemusí platit, pokud řádek protíná některý z vrcholů hranic oblasti.

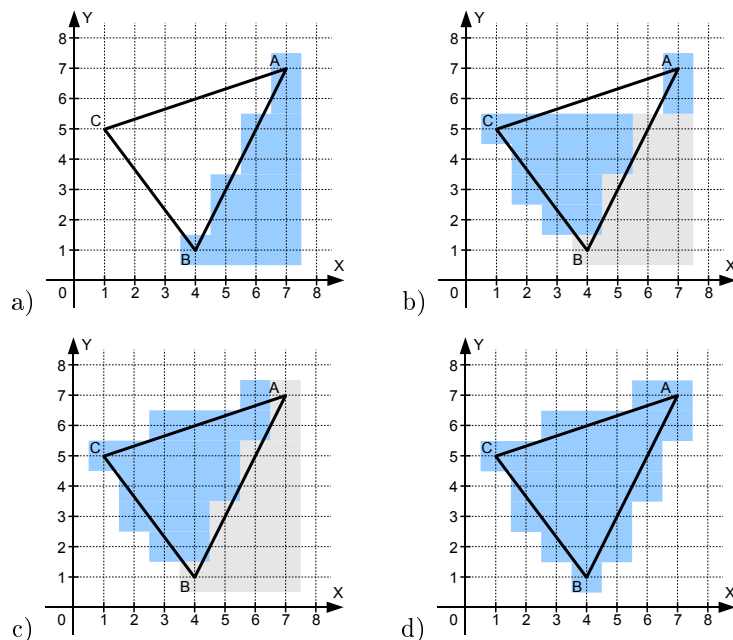
Například ve vrcholu G (viz. obr. 5.5a) bude přidán průsečík hrany FG, GA a BC, což jsou tři a to neodpovídá požadavku algoritmu. Ve vrcholech F a G to naopak nebude problém, protože se zde budou přidávat průsečíky pro hrany BC, CD, EF a FG, což jsou čtyři.

Obecně platí, že ve vrcholech, které jsou lokálními extrémy v ose Y (např. vrcholy F a C) se přidávají průsečíky obou hran. Ve vrcholech, které jsou lokálními extrémy v ose X (např. vrchol G) se přidává průsečík pouze jednou.

Pro řešení problému lichého počtu průsečíků v řádku při protínání vrcholu hrany máme k dispozici dvě řešení:

- Zkrácení dolního okraje všech hran hranice oblasti o 1 ve směru osy Y. Vodorovné hrany se vypouštějí (viz. obr. 5.5b). Při tom může dojít ke zkrácení dolního okraje oblasti (hrana AB). *Proto je nutné nakonec zvlášť vykreslit obrys oblasti rasterizací hranic.*
- Analýza typu extrémů protínaných vrcholů během generování průsečíků řádků a následné generování správného počtu průsečíků. Tím se však implementace algoritmu komplikuje a zpomaluje.

5.1.2 Optimalizace, seznam aktivních hran



Obrázek 5.6: Inverzní řádkové vyplnění oblasti (vyplněné oblasti modře, přemazané šedě): a) inverzní vyplnění od hrany AB, b) inverzní vyplnění od hrany BC, c) inverzní vyplnění od hrany CA, d) správně vyplněná oblast ABC.

5.2 Inverzní řádkové vyplňování

Hlavní nevýhodou *Řádkového vyplňovacího algoritmu* je nutnost třídění průsečíků pro každý vyplňovaný řádek (viz. kapitola 5.1), což je největší brzda výkonu. Různé optimalizační postupy mohou algoritmus urychlovat, ale třídění neodstraní.

Alternativou Řádkového vyplňování bez nutnosti třídění je *Inverzní řádkové vyplňování*. Tato metoda pracuje, stejně jako předchozí metoda, s obecnými mnohoúhelníky. Vyplňovaná oblast Ω je opět popsána seznamem hraničních entit (hran) $\Omega = \{e_1, e_2, \dots, e_n\}$.

Základní myšlenkou algoritmu *Inverzní řádkové vyplňování* je procházení dané oblasti Ω po jednotlivých hranách $e_i \in \Omega$. Od každé hrany jenapravo po řádcích provedeno vyplnění s inverzí aktuálních hodnot pixelů (viz. obr. 5.6).

DEF

Pro oblast Ω nejprve nalezneme maximální souřadnici X_{max} v ose X. Každá hrana $e_i \in \Omega$ je pak zpracována následujícím způsobem (pro paritní vyplňování):

- Získání Y_{min} a Y_{max} pro danou hranu e_i .
- Pro každý řádek $Y_j \in \langle Y_{min}, Y_{max} \rangle$, kromě vodorovných hran, proved':
 - Najdi průsečík $P_{ij} = (x_{ij}, y_{ij})$ řádku Y_j a aktuální hranou e_i .
 - Invertuj hodnoty pixelů v řádku Y_j od průsečíku P_{ij} po maximální souřadnici oblasti X_{max} v intervalu $\langle x_{ij}, X_{max} \rangle$.

Inverzí hodnot pixelů při vyplňování jednotlivých řádků hran oblasti se rozumí změna (přepnutí) jejich barvy z obarveného na neobarvený nebo naopak, podle

jejich aktuálního stavu. Tato operace může být realizována testováním hodnoty pixelů nebo bitovou operací XOR. Použití inverze pixelů odstraňuje nutnost řazení průsečíků hran s aktuálním řádkem (viz. kapitola 5.1). Tím je dosaženo lineární složitosti v závislosti na počtu hran.

První nevýhodou metody je zkrácení hranic vyplněné oblasti. Při prvním vyplnění od aktuální hrany s inverzí přes nevyplněné pozadí se hranice vykreslí. Při vyplnění od aktuální hrany s inverzí přes již vyplněné pozadí se hranice naopak smaže (viz. obr. 5.6).

Pro vyplnění celé oblasti je proto potřeba provést vykreslení obrysu oblasti rasterizací jeho hranic.

Druhou nevýhodou metody je ovlivňování okolí vyplněné oblasti napravo od ní až do její maximální souřadnice X_{max} (viz. obr. 5.6 – šedé přemazané části). Pokud provádíme vyplňování přímo do obrazové paměti (framebufferu), pak může dojít k porušení již dříve existující obrazové informace.

Proto je vhodnější provést vyplnění oblasti do pomocné paměti (lokálního framebufferu) a teprve potom přenést výsledek do vlastní obrazové paměti. Z důvodů inverze hodnot pixelů by měla být vyplňovaná oblast (obdélník opsaný dané oblastí) na začátku prázdná, nastavená na barvu pozadí.



5.3 Pinedův algoritmus

Pinedův algoritmus (J. Pineda, 1988) je další z metod pro vyplňování vektorových oblastí. Vzhledem k jeho principu pracuje pouze s *konvexními mnohoúhelníky*. Vyplňovaná oblast Ω je tedy opět popsána seznamem hraničních entit (hran) $\Omega = \{e_1, e_2, \dots, e_n\}$.

Základní myšlenkou Pinedova algoritmu vyplňování je rozdělení roviny dané oblasti Ω na poloroviny hran $e_i \in \Omega$. Všechny body roviny oblasti Ω , které leží na kladné straně všech polorovin hran e_i jsou pak uvnitř oblasti Ω (viz. obr. 5.7b).



Vzhledem k tomu, že je Pinedův algoritmus orientován na konvexní polygony, je nejčastěji používán pro vyplňování trojúhelníků. Trojúhelníky, jakožto nejjednodušší polygon, jsou vždycky konvexní, takže je na konvexnost nemusíme testovat.

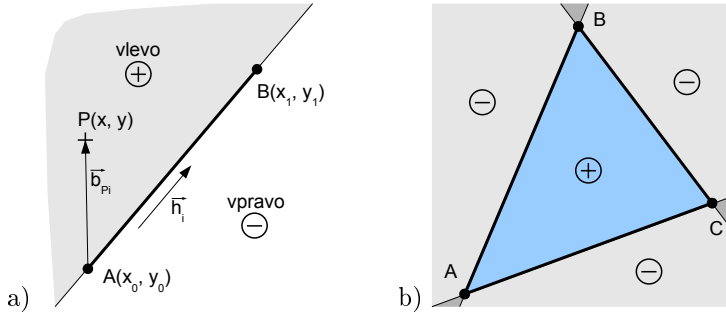
Základem Pinedova algoritmu jsou *Hranové funkce* $E_i(x, y)$. Jde o test polohy bodu $P(x, y)$ v rovině oblasti Ω vůči jedné její hraně e_i určené vrcholy oblasti $V_{i0}[x_{i0}, y_{i0}]$ a $V_{i1}[x_{i1}, y_{i1}]$. Vztah 5.4 pro výpočet hodnoty hranové funkce získáme vektorovým součinem 5.3 vektoru $\vec{h}_i$ 5.1 hrany e_i a vektoru $\vec{b}_{Pi}$ 5.2 z počátku hrany k testovanému bodu P (viz. obr. 5.7a).

$$\vec{h}_i = (x_{i1} - x_{i0}, y_{i1} - y_{i0}) = (\Delta x_i, \Delta y_i) \quad (5.1)$$

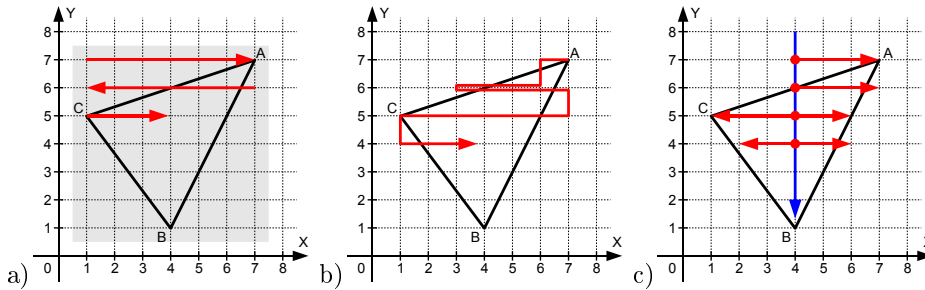
$$\vec{b}_{Pi} = (x - x_{i0}, y - y_{i0}) \quad (5.2)$$

$$E_i(x, y) = \vec{h}_i \times \vec{b}_{Pi} \quad (5.3)$$

$$E_i(x, y) = (x - x_{i0})\Delta y_i - (y - y_{i0})\Delta x_i \quad (5.4)$$



Obrázek 5.7: Schéma Pinedova algoritmu: a) hrana a její poloroviny, b) trojúhelníková oblast a její poloroviny.



Obrázek 5.8: Strategie procházení řádků pro Pinedův algoritmus: a) procházení opsaného obdélníka po řádcích, b) iterování po řádcích z maximálního vrcholu, c) procházení po řádcích podél plotu.

Je-li pro bod $P(x, y)$ hodnota všech hranových funkcí $E_i(x, y) \geq 0$, bod leží uvnitř nebo na hranici oblasti Ω . V opačném případě je mimo danou oblast.

Abychom nemuseli pro každý bod $P(x, y)$ kompletně vypočítávat všechny hranové funkce $E_i(x, y)$, je možné určit hodnotu hranových funkcí bodu $P(x, y)$ na základě hodnot odpovídajících hranových funkcí sousedního bodu $P(x \pm 1, y)$ (viz. vztah 5.8) nebo bodu $P(x, y \pm 1)$ (viz. vztah 5.9).

$$E_i(x + 1, y) = (x + 1 - x_{i0})\Delta y_i - (y - y_{i0})\Delta x_i \quad (5.5)$$

$$E_i(x + 1, y) = (x - x_{i0})\Delta y_i + \Delta y_i - (y - y_{i0})\Delta x_i \quad (5.6)$$

$$E_i(x + 1, y) = E_i(x, y) + \Delta y_i \quad (5.7)$$

$$E_i(x \pm 1, y) = E_i(x, y) \pm \Delta y_i \quad (5.8)$$

$$E_i(x, y \pm 1) = E_i(x, y) \pm \Delta x_i \quad (5.9)$$

Řešení vyplnění trojúhelníka (velmi časté při 3D zobrazení) je tak převedeno na testování hranových funkcí všech pixelů v okolí oblasti Ω . Pro každý pixel vypočteme hodnoty tří hranových funkcí. Výpočet hodnoty jedné hranové funkce na základě sousedního bodu reprezentuje jednu operaci sčítání celých čísel. Algoritmus je tedy velmi jednoduchý a snadno paralelizovatelný. Z těchto důvodů je velmi výhodné a jednoduché realizovat implementaci Pinedova algoritmu do hardware grafických jednotek.

Pro procházení okolí oblasti Ω je možné zvolit tři různé strategie:

- Procházení obdélníka opsaného oblasti Ω po řádcích (viz. obr. 5.8a). Zbytečně procházíme $\sim 1/2$ obdélníka, který je určitě prázdný.
- Procházení řádků od maximálního vrcholu s obratem a přechodem na další řádek při opuštění oblasti (viz. obr. 5.8b). Algoritmicky složitější postup.
- Procházení po řádcích podél svislého plotu, který spustíme ze středu opsaného obdélníka nebo z prostředního vrcholu (viz. obr. 5.8c). Každou stranu plotu můžeme procházet nezávisle – možnost paralelizace.

5.4 Semínkové vyplňování

Semínkové vyplňování je klasický algoritmus pro vyplňování rastrových oblastí. Vyplňovaná oblast Ω je popsána v rastrové matici, nejčastěji přímo v obrazové paměti (framebufferu), spojitou množinou podobných (stejných) pixelů tvořících hranici nebo spojitou množinou podobných (stejných) pixelů tvořících oblast.

Nemáme k dispozici orientovaný seznam hraničních entit, takže není možné určit, co je uvnitř a co vně oblasti. Proto musí na počátku uživatel zadat startovací bod (semínko), které určí co je uvnitř oblasti (viz. obr. 5.9).

Základní myšlenkou *Semínkového vyplňování* je obarvování sousedních pixelů (4 nebo 8) od daného semínka. Obarvené pixely se rekurzivně stávají dalšími semínky. Pixely, které představují hranici oblasti nebo nemají vlastnosti oblasti se vyřazují.

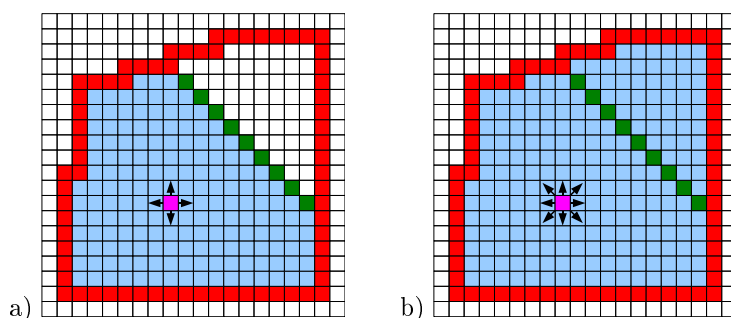


Podle způsobu rastrové definice oblasti nebo její hranice rozlišujeme následující čtyři varianty řízení (ukončování) procesu vyplňování:

- **Hraniční** – oblast je definována spojitou hranicí z pixelů dané barvy. Při vyplňování se pixely testují, jestli nemají barvu hranice.
- **Záplavové** – oblast je definována spojitou množinou vnitřních pixelů dané barvy. Při vyplňování se pixely testují, jestli mají barvu oblasti.
- **Měké** – oblast je definována spojitou hranicí z pixelů daného rozptylu barev. Při vyplňování se pixely testují, jestli nemají barvu uvnitř daného rozptylu.
- **Prahové** – oblast je definována spojitou množinou vnitřních pixelů jejichž barva leží v daném rozmezí okolo barvy semínka. Při vyplňování se pixely testují, jestli mají barvu uvnitř rozmezí okolo barvy semínka.

Podle způsobu obarvování okolí (výběru sousedů) od aktuálního semínka rozlišujeme dva typy vyplňování a zároveň dva druhy spojitosti hranice oblasti:

- **4-okolí** – každý pixel má 4 sousedy, po hraně (viz. obr. 5.9a). Při vyplňování testujeme a obarvujeme tyto 4 sousedy. Hranice oblasti je uzavřená, pokud všechny její hraniční pixely sousedí po hraně.
- **8-okolí** – každý pixel má 8 sousedů, po hraně a pře vrcholy (viz. obr. 5.9b). Při vyplňování testujeme a obarvujeme těchto 8 sousedů. Hranice oblasti je uzavřená, pokud všechny její hraniční pixely sousedí po hraně nebo přes vrchol.



Obrázek 5.9: Druhy oblastí a vyplňování pro semínkové vyplňování: a) vyplňování typu 4-okolí; b) vyplňování typu 8-okolí. Červená hranice oblasti je typu 4-okolí, zelená čára je typu 8-okolí.

Při vyplňování oblasti typu 4-okolí můžeme bez obav použít vyplňování typu 4-okolí i 8-okolí. Při vyplňování oblasti typu 8-okolí však musíme použít vyplňování typu 4-okolí, jinak nám obsah oblasti vyteče mimo hranice (viz. obr. 5.9).

Semínkový vyplňovací algoritmus je možné velmi jednoduše implementovat prostřednictvím rekurze. Zde však hrozí nebezpečí přetečení zásobníku při vyplňování rozsáhlejších oblastí. Proto je vhodnější implementovat algoritmus s ukládáním nových semínek do vlastního paměťového zásobníku. Paměťové nároky však mohou být velké, podle velikosti oblasti.

Obecný postup vyplňování dané rastrové oblasti Ω semínkovým algoritmem s vlastním zásobníkem pro semínko $S[x, y]$ je pak následující:

1. Testování sousedních pixelů aktuálního semínka (4 nebo 8 podle typu vyplňování), jestli jsou ještě neobarvené a patří do dané oblasti Ω .
2. Obarvení vyhovujících sousedních pixelů a jejich vložení do zásobníku.
3. Získání nového semínka z vrcholu zásobníku a návrat k bodu 1.
4. Konec, pokud je zásobník prázdný.

5.4.1 Řádkové semínkové vyplňování

Shrnutí

Vyplňování rovinných oblastí má pro interaktivní zobrazení 3D polygonálních modelů stejný význam jako rasterizace pro vektorové čárové objekty. Proto potřebujeme pro rastrové zobrazení plošných oblastí (polygonů, mnohoúhelníků) efektivní algoritmy s možností hardwarové implementace.

Dané plošné oblasti jsou popsány většinou svou hranicí a je potřeba vyřešit jejich vyplnění, vybarvení vnitřních bodů. Hranice oblastí mohou být definovány vektorově (množinou vektorových entit) nebo rastrově (pixely v rastrové matici).

Základním algoritmem pro vyplňování vektorových oblastí je *Řádkové vyplňování*, které prochází a vyplňuje danou oblast celkově po řádcích. Modifikací Řádkového vyplňování je *Inverzní řádkové vyplňování*, které s inverzí pixelů po řádcích vyplňuje jednotlivé hrany do pravého okraje.

Pro vyplňování konvexních vektorových oblastí, nejčastěji trojúhelníků, je určen *Pinedův algoritmus*, který pro hrany oblasti definuje *hranové funkce*. Všechny vnitřní body oblasti pak mají pro všechny hranové funkce kladnou hodnotu.

Pro vyplňování rastrových oblastí je určen algoritmus *Semínkového vyplňování*, který od zadaného prvního pixelu (semínka) rekurzivně obarvuje své sousedy, až po dosažení hranice dané oblasti. Podle počtu sousedů rozlišujeme vyplňování typu 4-okolí nebo 8-okolí. Stejně tak je rozlišována spojitost hranice oblastí.



Kontrolní otázky

1. Jaká je souvislost vyplňování 2D oblastí s zobrazování 3D polygonálních modelů?
2. Jakým způsobem jsou popsány vyplňované oblasti?
3. Jakým způsobem mohou být definovány hranice vyplňované oblasti?
4. Jaký je princip Řádkového vyplňování?
5. Jakým způsobem se můžeme vypořádat s lichým počtem průsečíků řádku při Řádkovém vyplňování?
6. Jakým způsobem můžeme optimalizovat Řádkový vyplňovací algoritmus?
7. Jaký je princip Inverzního řádkového vyplňování?
8. Jaké jsou nevýhody (slabosti) Inverzního řádkového vyplňování?
9. Jaký je principiální rozdíl mezi Řádkovým vyplňováním a Inverzním řádkovým vyplňováním?
10. Jaký je princip Pinedova vyplňovacího algoritmu?
11. Odvoďte hranovou funkci pro Pinedův vyplňovací algoritmu?
12. Jaké máme k dispozici strategie procházení řádků u Pinedova vyplňovacího algoritmu?
13. Jaký je princip Semínkového vyplňování?
14. Jaké typy obarvování okolí se semínka znáte?
15. Jaké druhy spojitosti rastrové hranice oblasti znáte?



16. Jaký je princip Řádkového semínkového vyplňování?
17. Jaký je přínos Řádkového semínkového vyplňování proti klasickému Semínkovému vyplňování?
18. Jakým způsobem může být popsána rastrová oblast pro Semínkové vyplňování?

Kapitola 6

Ořezávání vektorových 2D objektů



1:00



Cílem této kapitoly je seznámit se s hlavními algoritmy a metodami používanými pro ořezávání vektorových objektů hranicemi zobrazovacího okna. Konkrétně se budeme zabývat ořezáváním úseček a polygonů definovaných po částech lineárně (mnohoúhelníky).

V počítačové grafice je velmi časté, že množství informace obsažené v zobrazovaných datech (obrazech, kresbách, modelech atd.) je větší než schopnosti současných zobrazovacích zařízení (monitorů, tiskáren, atd.). To se může týkat barevné hloubky (počtu zobrazovaných barev) nebo rozlišení zobrazované informace.

Abychom byli schopni zobrazovat a prohlížet si detaily dat s vysokým rozlišením, potřebujeme mít možnost měnit měřítko zobrazení (zoom) a posouvat výřezem obrazové plochy. Pokud však budeme chtít zobrazit pouze část zobrazovaných dat (výřez), potřebujeme umět vybrat (oříznout) jejich viditelnou část, jinak se budeme zbytečně zabývat tím, co stejně není vidět.

Jestliže zobrazujeme rastrovou informaci, stačí pouze oříznout vlastní definiční rastrovou matici. Pokud však zobrazujeme vektorovou informaci, mohou být některé entity zcela mimo nebo zcela uvnitř zobrazovaného výřezu nebo mohou protínat hranice výřezu.

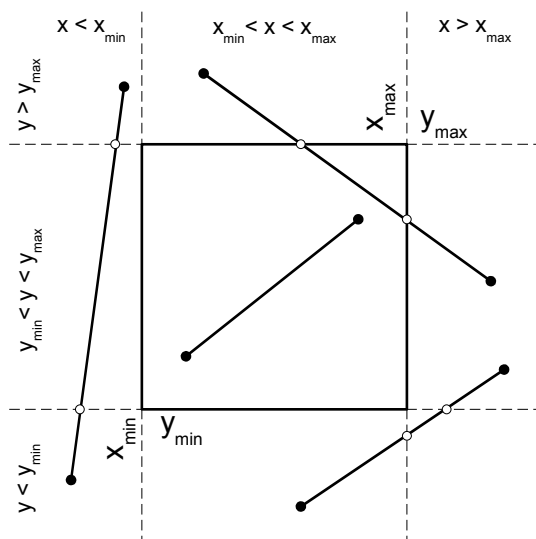
Ořezávání vektorových entit je proces výběru zobrazovaných vektorových entit vůči aktuálnímu rozměru a poloze zobrazovacího okna. Entity, které leží zcela mimo okno, budou z dalšího zpracování vyloučeny. Entity, které hranice okna protínají, budou rozděleny (oříznuty) na části ležící vůči oknu zcela uvnitř a zcela vně. Výsledkem procesu ořezávání jsou entity stejného typu (úsečky, oblouky, polygony, atd.), jako původní entity před ořezáním.



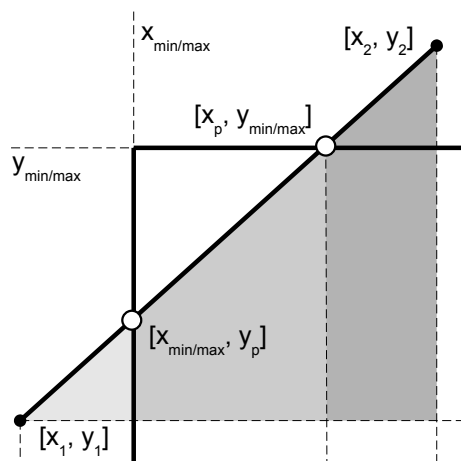
Nejprimitivnější způsob ořezávání vektorových entit je na úrovni rastrovaných pixelů. U všech zobrazovaných entit bez výběru provedeme rasterizaci. Teprve souřadnice získaných pixelů se porovnávají se souřadnicemi zobrazovacího okna a vykreslují se pouze vnitřní pixely. Tento postup je samozřejmě velmi neefektivní a prakticky se nepoužívá.

Pro popis dalších sofistikovanějších metod ořezávání vektorových entit se omezíme na úsečky a polygony definované po částech lineární hranicí (mnohoúhelníky). Další druhy složitějších vektorových entit jsou potom převedeny na kombinaci úseček nebo mnohoúhelníků.

Zobrazovací okno je obecně definováno polygonem s obecnou (křivkovou) hranicí. Pro další výklad předpokládáme konvexní okno ve tvaru obdélníku se stranami rovnoběžnými se souřadnými osami.



Obrázek 6.1: Primitivní ořezání úsečky.

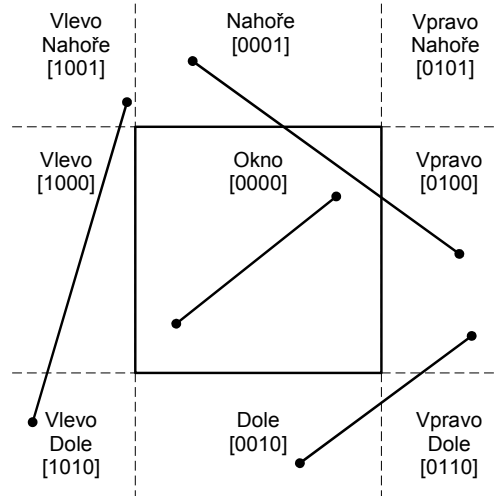


Obrázek 6.2: Určování průsečíků úsečky s hranicí okna.

6.1 Ořezávání úseček

Úsečka je nejjednodušší "čárová" (topologicky jednorozměrná) vektorová entita, definovaná souřadnicemi koncových bodů (viz. kapitola 4.1, viz. obr. 4.1). Při zobrazení dané úsečky potřebujeme nejprve testovat její polohu vůči zobrazovacímu oknu. Leží-li úsečka zcela mimo okno, je vyloučena z dalšího zpracování. Leží-li úsečka celá uvnitř okna, je vykreslena. Protíná-li úsečka hranice okna, je potřeba ji na hranicích okna rozdělit na části, které budou znova testovány.

Primitivním způsobem testování polohy úsečky vůči zobrazovacímu oknu je testování polohy jejích koncových bodů (viz. obr. 6.1). Pokud leží oba koncové body dané úsečky uvnitř okna, leží celá úsečka také uvnitř okna. Pokud leží oba koncové body úsečky na vnější straně stejné hranice okna, leží celá úsečka také vně okna. Pro zbývající případy je potřeba analyzovat všechny průsečíky s hranicemi okna, rozdělit úsečku na části podle získaných průsečíků a opakovaně tyto vytvořené části testovat.



Obrázek 6.3: Ořezání úsečky algoritmem Cohen-Sutherland, kódování vrcholů.

Výpočet průsečíků dané úsečky s hranicemi okna je velmi jednoduchý. Z podobnosti trojúhelníků (viz. rovnice 6.1 a obr. 6.2) odvodíme vztahy pro výpočet průsečíku $[x_p, y_p]$ dosazením polohy hranice okna $x_p = x_{min/max}$ (viz. rovnice 6.2) pro svislé nebo $y_p = y_{min/max}$ pro vodorovné hranice (viz. rovnice 6.3).

$$\frac{x_2 - x_1}{y_2 - y_1} = \frac{x_p - x_1}{y_p - y_1} \quad (6.1)$$

$$x_p = x_{min/max} \rightarrow y_p = \frac{(x_{min/max} - x_1)(y_2 - y_1)}{x_2 - x_1} + y_1 \quad (6.2)$$

$$y_p = y_{min/max} \rightarrow x_p = \frac{(x_2 - x_1)(y_{min/max} - y_1)}{y_2 - y_1} + x_1 \quad (6.3)$$

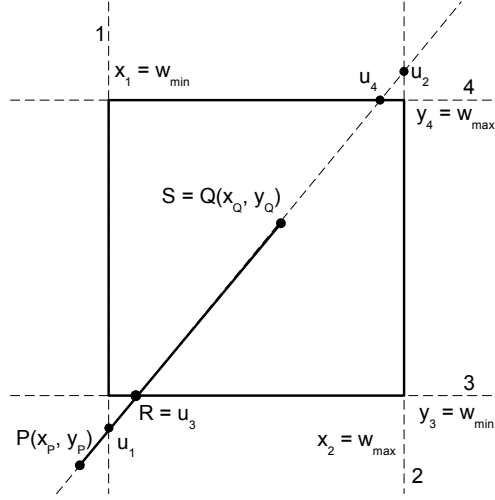
6.1.1 Cohen-Sutherland

Cohen-Sutherlandův algoritmus pro ořezání úseček je rozšířením primitivního způsobu testování polohy úsečky vůči zobrazovacímu oknu. Koncové body úsečky P_1, P_2 jsou označeny binárními 4-bitovými kódy C_1, C_2 (viz. obr. 6.3). Rovina okna je rozdělena na devět podoblastí, kterým odpovídají příslušné kódy. Sestavení kódů je podle schématu: [Vlevo, Vpravo, Dole, Nahoře]. Podle polohy příslušného koncového bodu úsečky v některé z těchto podoblastí, jsou odpovídajícím způsobem nastaveny bity jeho kódu.

Podle hodnot binární kombinace kódů koncových bodů můžeme určit potenciální polohu úsečky vůči oknu:

- C_1 or $C_2 = 0 \rightarrow$ celá úsečka leží uvnitř okna.
- C_1 and $C_2 \neq 0 \rightarrow$ celá úsečka leží mimo okno.
- C_1 and $C_2 = 0 \rightarrow$ úsečka potenciálně protíná hranice okna.

Pokud úsečka potenciálně protíná hranice okna, vypočteme příslušné průsečíky (viz. vztahy 6.2 a 6.3). Podle nich úsečku rozdělíme na části, které znovu testujeme na polohu vůči oknu. Průsečíky počítáme pouze pro ty hranice okna, kterým odpovídá nastavený bit v hodnotě binárního výrazu: C_1 or C_2 .



Obrázek 6.4: Ořezání úsečky algoritmem Liang-Barsky.

6.1.2 Liang-Barsky

Dalším možností pro ořezání úseček je algoritmus Liang-Barsky. Tento algoritmus pracuje v parametrickém prostoru. Ořezávaná úsečka je dána svými koncovými body $P[x_P, y_P]$ a $Q[x_Q, y_Q]$ (viz. obr. 6.4). Parametricky je úsečka popsána rovnicemi 6.4 a 6.5.

$$x = x_P + (x_Q - x_P) \cdot u = x_P + \Delta x \cdot u \quad (6.4)$$

$$y = y_P + (y_Q - y_P) \cdot u = y_P + \Delta y \cdot u \quad (6.5)$$

Na celé úsečce je parametr $u \in \langle 0, 1 \rangle$. Při ořezání hledáme část úsečky ležící uvnitř daného okna. V algoritmu Liang-Barsky to znamená, že hledáme hodnoty parametrů pro koncové body části úsečky ležící uvnitř okna. Pro část úsečky uvnitř okna platí nerovnice 6.6 a 6.7 (viz. obr. 6.4). Tyto nerovnice je možné přepsat do tvaru nerovnice 6.8. Parametry p_k a q_k jsou definovány v rovnicích 6.9 až 6.12. Pro průsečíky přímky dané úsečky s hranicemi okna platí vztah 6.13.

$$x_{wmin} \leq x_P + \Delta x \cdot u \leq x_{wmax} \quad (6.6)$$

$$y_{wmin} \leq y_P + \Delta y \cdot u \leq y_{wmax} \quad (6.7)$$

$$u \leq \frac{q_k}{p_k}, \quad k = 1, 2, 3, 4 \quad (6.8)$$

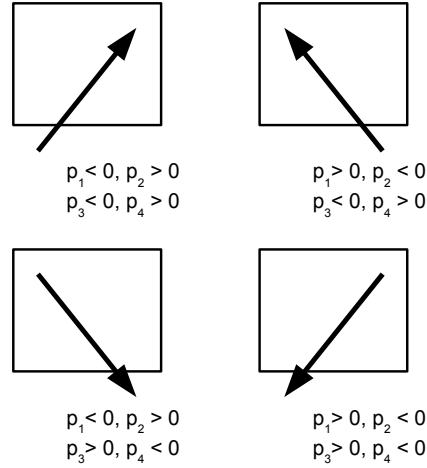
$$p_1 = -\Delta x, \quad q_1 = x_P - x_{wmin} \quad (6.9)$$

$$p_2 = \Delta x, \quad q_2 = x_{wmax} - x_P \quad (6.10)$$

$$p_3 = -\Delta y, \quad q_3 = y_P - y_{wmin} \quad (6.11)$$

$$p_4 = \Delta y, \quad q_4 = y_{wmax} - y_P \quad (6.12)$$

$$u_k = \frac{q_k}{p_k}, \quad k = 1, 2, 3, 4 \quad (6.13)$$



Obrázek 6.5: Orientace úsečky vůči oknu při ořezávání algoritmem Liang-Barsky.

Podle hodnot parametrů p_k a q_k můžeme určovat polohu úsečky vůči oknu. Je-li parametr $p_k = 0$, pak je úsečka rovnoběžná s odpovídající hranicí okna. Je-li navíc pro stejný index k hodnota parametru $q_k < 0$, leží celá úsečka mimo okno a není potřeba se s ní dále zabývat.

Znaménko parametru p_k určuje směr úsečky vůči hranicím okna k (viz. obr. 6.5). To nám pomůže určit koncové body R, S dané úsečky po oříznutí oknem. Parametr u_R počátečního bodu R vybereme z parametrů u_k vypočítaných (viz. vztah 6.13) pro hranice k , jejichž parametr $p_k < 0$. Parametr u_S koncového bodu S vybereme z parametrů u_k vypočítaných (viz. vztah 6.13) pro hranice k , jejichž parametr $p_k > 0$. Uvnitř okna při tom musí stále platit $0 \leq u_{R,S} \leq 1$.

Z předchozích skutečností můžeme pak na závěr zobecnit vztahy (viz. 6.14 a 6.15) pro výpočet parametrů u koncových bodů R, S části dané úsečky ležící uvnitř okna (viz. obr. 6.4).

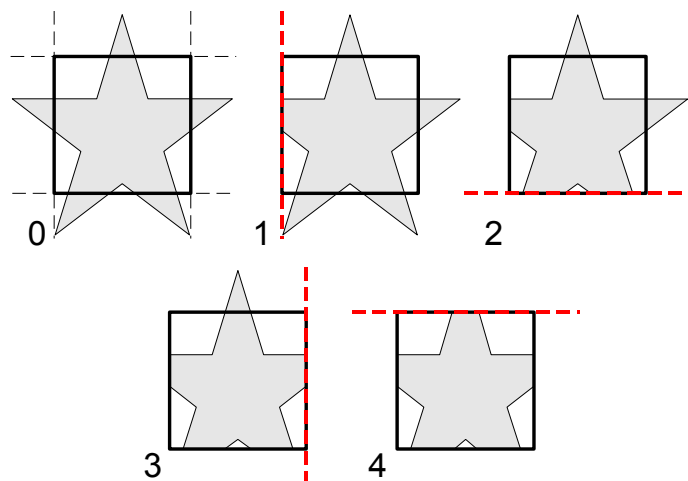
$$u_R = \max (u_k (p_k < 0), 0) \quad (6.14)$$

$$u_S = \min (u_k (p_k > 0), 1) \quad (6.15)$$

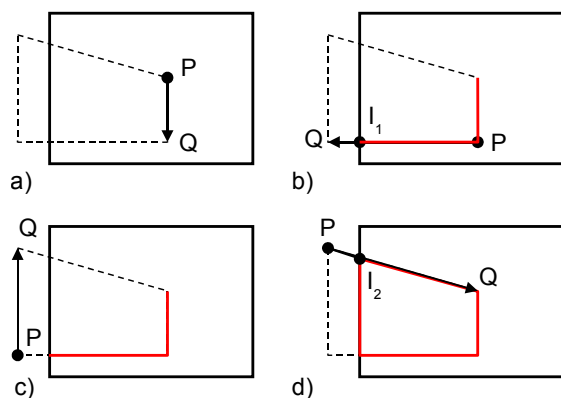
6.2 Ořezávání polygonů

Polygony jsou, podobně jako úsečky, nejjednodušší rovinné uzavřené vyplněné objekty. Definujeme je jako mnohoúhelníky s po částech lineární hranicí. Při zobrazování polygonu opět potřebujeme testovat jeho polohu vůči zobrazovacímu oknu. Leží-li zcela mimo okno, je vyloučen z dalšího zpracování. Leží-li celý uvnitř okna, je vykreslen, obrysem nebo s vyplněním. Protíná-li však hranice okna, je potřeba ho nejprve oříznout o hranice okna a potom teprve vnitřní část vykreslit.

Jako nejjednodušší a přímočarý způsob oříznutí polygonu se jeví prosté oříznutí jeho hranic (úseček) zobrazovacím oknem. Tím bychom však dostali pouze hromadu ořezaných úseček. Ztratili bychom informaci o části polygonu ležící uvnitř okna, jako o uzavřené rovinné oblasti. Musíme proto použít algoritmy specializované na oříznutí polygonů, které jako výsledek produkují opět polygony ležící uvnitř zobrazovacího okna.



Obrázek 6.6: Postup ořezání polygonu algoritmem Sutherland-Hodgman.



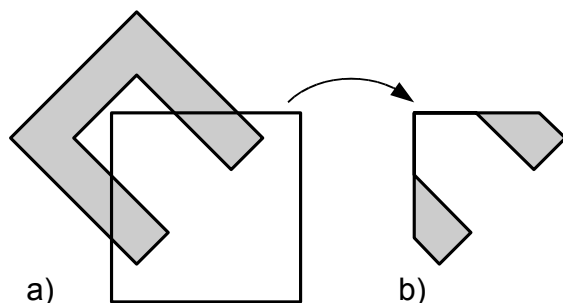
Obrázek 6.7: Postup ořezání polygonu jednou stranou okna algoritmem Sutherland-Hodgman: a) vnitřní úsečka, b) úsečka protínající stranu směrem ven, c) vnější úsečka, d) úsečka protínající stranu směrem dovnitř, přibude spojnice průsečíků

6.2.1 Sutherland-Hodgman

Algoritmus Sutherland-Hodgman provádí ořezání polygonů obdélníkovým oknem se stranami rovnoběžnými se osami X a Y . Komplexní úkol ořezání polygonu celým oknem je zjednodušen na ořezání pouze jednou stranou okna (přímkou procházející příslušnou stranou okna). Po oříznutí jednou stranou okna provedeme oříznutí druhou, třetí a čtvrtou stranou. Tak postupně po jednodušších krocích dosáhneme oříznutí celého polygonu celým oknem (viz. obr. 6.6).

Algoritmus je možné implementovat také tím způsobem, že ořezáváme pouze jednou stranou okna (nebo přímo souřadnou osou) a postupně otáčíme a posouváme ořezávaný polygon. Otočení polygonu o 90° lze snadno realizovat výměnou souřadnic ($x' = y$, $y' = -x$).

Pro výpočet průsečíků hranic polygonu s hranicemi okna použijeme stejný postup jako pro výpočet průsečíků úseček s hranicemi obdélníkového okna (viz. vztahy 6.3 a 6.2).



Obrázek 6.8: Vznik překrývajících se hran při ořezání polygonu algoritmem Sutherland-Hodgman: a) původní polygon před ořezáním, b) ořezaný polygon s překryvem hran přes roh okna

Při ořezání polygonu jednou stranou okna postupujeme orientovaně po jednotlivých jeho hranicích (úsečkách) (viz. obr. 6.7). Každou úsečku testujeme, jestli leží uvnitř/vně okna nebo protíná ořezávací stranu. Pokud úsečka leží celá uvnitř okna, vložíme ji do výsledného seznamu úseček (viz. obr. 6.7a). Pokud úsečka stranu protíná, pak ji ořízneme a zbylou vnitřní část opět vložíme do výsledného seznamu úseček. Při tom si pamatujeme získaný průsečík úsečky se stranou okna, kde úsečka opouští okno (protíná stranu okna směrem ven) (viz. obr. 6.7b). Pokud úsečka leží celá vně okna, neděláme nic (viz. obr. 6.7c). Následující úsečku, která protíná stranu okna, opět ořízneme. Do výsledného seznamu úseček vložíme úsečku tvořenou spojnicí předchozího průsečíku se stranou okna a průsečíku aktuálně zpracovávané úsečky. Jako další vložíme do výsledného seznamu úseček zbylou vnitřní část aktuálně zpracovávané úsečky (viz. obr. 6.7d).

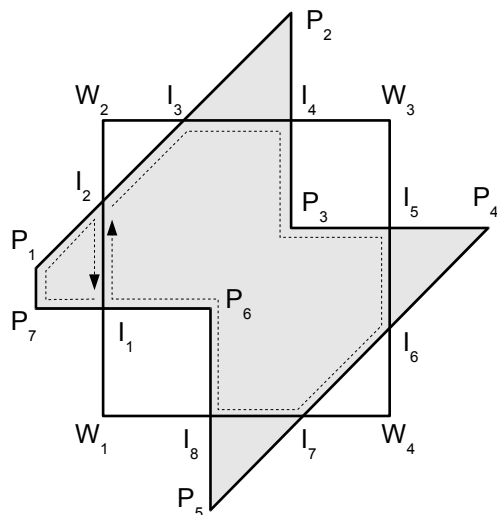
Algoritmus Sutherland-Hodgman je relativně jednoduchý a snadnou implementovatelný. Jeho hlavní nevýhodou je generování překrývajících se hran ořezaného polygonu podél hranic a rohů okna (viz. obr. 6.8). Ořezávaný polygon při tom může být definován pouze jednou uzavřenou smyčkou hraničních úseček. Je tedy komplikovaná realizace polygonů s vnitřními otvory.

6.2.2 Weiler-Atherton

Pro obecné ořezání polygonů i s vnitřními otvory (vnitřní smyčky mají opačnou orientaci) obecným oknem definovaným polygonem je určen algoritmus Weiler-Atherton. Jeho základem je nalezení všech průsečíků mezi dvěma množinami úseček v rovině, ořezávaným polygonem a ořezávacím oknem.

Z vrcholů obou vstupních polygonů a získaných průsečíků sestavíme několik pomocných seznamů. Jejich vhodným procházením můžeme podle potřeby získat vnitřní nebo vnější části polygonu ořezaného oknem. Pomocné seznamy jsou tyto:

- P – seznam vrcholů včetně průsečíků při cestě po ořezávaném polygonu
- W – seznam vrcholů včetně průsečíků při cestě po ořezávacím okně
- I – seznam průsečíků ve kterých hraniční úsečky polygonu vstupují do okna
- O – seznam průsečíků ve kterých hraniční úsečky polygonu vystupují z okna
- C – seznam ořezaných polygonů uvnitř okna
- R – seznam odřezků polygonů vně okna



Obrázek 6.9: Postup ořezání polygonu algoritmem Weiler-Atherton

Postup procházení pomocných seznamů pro získání částí ořezávaných polygonů uvnitř okna je následující (výsledek ukládáme do seznamu C):

- Začneme na seznamu P na prvním vrcholu ze seznamu I
- Jsme-li v seznamu P na vrcholu, který se nachází v seznamu I, pokračujeme v procházení po vrcholech seznamu P
- Jsme-li v seznamu P na vrcholu, který se nachází v seznamu O, pokračujeme v procházení po vrcholech seznamu W
- Jsme-li v seznamu W na vrcholu, který se nachází v seznamu O, pokračujeme v procházení po vrcholech seznamu W
- Jsme-li v seznamu W na vrcholu, který se nachází v seznamu I, pokračujeme v procházení po vrcholech seznamu P
- Pokračujeme dokud neuzavřeme obrazec a neprojdeme celý seznam P

Postup procházení pomocných seznamů pro získání částí ořezávaných polygonů vně okna je následující (výsledek ukládáme do seznamu R):

- Začneme na seznamu P na prvním vrcholu ze seznamu O
- Jsme-li v seznamu P na vrcholu, který se nachází v seznamu O, pokračujeme v procházení po vrcholech seznamu P
- Jsme-li v seznamu P na vrcholu, který se nachází v seznamu I, pokračujeme v procházení po vrcholech seznamu W, v opačném směru
- Jsme-li v seznamu W na vrcholu, který se nachází v seznamu I, pokračujeme v procházení po vrcholech seznamu W, v opačném směru
- Jsme-li v seznamu W na vrcholu, který se nachází v seznamu O, pokračujeme v procházení po vrcholech seznamu P
- Pokračujeme dokud neuzavřeme obrazec a neprojdeme celý seznam P

Příklad pomocných seznamů a výsledků zpracování pro obrázek 6.9 je následující:

- $P - P_1, I_2, I_3, P_2, I_4, P_3, I_5, P_5, I_6, I_7, P_5, I_8, P_6, I_1, P_7, P_1$
- $W - W_1, I_1, I_2, W_2, I_3, I_4, W_3, I_5, I_6, W_4, I_7, I_8, W_1$
- $I - I_2, I_4, I_6, I_8$
- $O - I_1, I_3, I_5, I_7$
- $C - I_2, I_3, I_4, P_3, I_5, I_6, I_7, I_8, P_6, I_1, I_2$
- $R_1 - I_1, P_7, P_1, I_2, I_1$
- $R_2 - I_3, P_2, I_4, I_3$
- $R_3 - I_5, P_4, I_6, I_5$
- $R_4 - I_7, P_5, I_8, I_7$

Shrnutí

Ořezávání vektorových objektů ve 2D je úzce spojeno s jejich zobrazováním. Pokud používáme maximálně efektivní metody pro jejich rasterizaci nebo vyplňování, pak můžeme takto získanou výhodu rychlosti zobrazení zcela ztratit nevhodným vykreslováním objektů, které ve skutečnosti na obrazovce vůbec nejsou vidět, protože si zrovna prohlížíme jinou část zobrazovaných dat nebo jejich malý detail.

Proto musíme před vlastním zobrazením (rasterizací, vyplňováním) vektorových dat provést jejich oříznutí pohledovým oknem. To znamená odstranit všechny objekty, které leží zcela mimo okno nebo oříznout ty, které okno protínají. Po tomto kroku dostaneme pouze ty objekty a jejich části, které leží uvnitř aktuálního pohledového okna. Pro jednoduchost jsme se zabývali ořezáním pouze úseček a polygonů s po částech lineární hranicí (mnohoúhelníky).

Při ořezání úseček je základním úkolem určení jejich polohy vůči oknu. Nejjednodušším způsobem je určení polohy koncových bodů úsečky vůči oknu. Pokročilejší způsob používá algoritmus Cohen-Sutherland, který označí každý koncový bod 4 bitovým kódem. Binární kombinací těchto kódů obou koncových bodů úsečky je pak možné lépe a jednodušeji určit polohu úsečky vůči oknu. Úsečky protínající hranice okna jsou rozděleny a znova testovány. Další možností pro ořezání úseček je algoritmus Liang-Barsky, který testuje polohu úsečky vůči oknu a hledá průsečíky v parametrickém prostoru ořezávané úsečky.

Při ořezání polygonů nestačí pouze ořezat hraniční úsečky, protože bychom ztratili spojitost hranic polygonu. Musíme tedy použít specializované algoritmy, které na výstupu mají opět uzavřené orientované polygony ležící uvnitř okna. Jedním z nich je algoritmus Sutherland-Hodgman, který ořezává daný polygon postupně vždy jen jednou stranou okna (proloženou přímkou). Další možností je implementovat ořezání polygonu jednou souřadnou osou a polygon příslušně postupně otáčet po 90° přehazováním souřadnic vrcholů. Algoritmus Weiler-Atherton je schopen provádět obecné ořezání polygonu včetně vnitřních otvorů obecným oknem definovaným polygonem. Jeho základem je několik pomocných seznamů vrcholů polygonu a okna včetně průsečíků. Jejich vhodným průchodem získáme části polygonu uvnitř nebo vně okna.



Kontrolní otázky

1. Proč je důležité ořezávat polygonální objekty zobrazovacím oknem před jejich vykreslením?
2. Jak můžeme snadno získat průsečík úsečky s hranicí obdélníkového okna?
3. Jaký je princip algoritmu Cohen-Sutherland pro ořezání úseček?
4. Jakým způsobem algoritmus Cohen-Sutherland pro ořezání úseček určuje polohu úsečky vůči oknu?
5. Jakým způsobem určuje algoritmus Cohen-Sutherland pro ořezání úseček průsečíky úsečky s hranicemi okna?
6. Jaký je princip algoritmu Liang-Barsky pro ořezání úseček?
7. Jakým způsobem algoritmus Liang-Barsky pro ořezání úseček určuje polohu úsečky vůči oknu?
8. Jakým způsobem určuje algoritmus Liang-Barsky pro ořezání úseček průsečíky úsečky s hranicemi okna?
9. Je možné ořezávat polygony prostým ořezáním jejich hraničních úseček?
10. Co se stane, když ořežeme polygon prostým ořezáním jeho hraničních úseček?
11. Jaký je princip algoritmu Sutherland-Hodgman pro ořezání polygonů?
12. Jaké má algoritmus Sutherland-Hodgman pro ořezání polygonů omezení a nevýhody?
13. Jaký je princip algoritmu Weiler-Atherton pro ořezání polygonů?
14. Jaké druhy polygonů je schopen zpracovat algoritmus Weiler-Atherton pro ořezání polygonů?



Kapitola 7

Geometrické transformace



1:00



Cílem této kapitoly je seznámit se s hlavními principy transformací vektorových objektů ve 2D a 3D prostoru, včetně realizace projekcí. Konkrétně se budeme zabývat maticovou realizací afinních transformací spolu s paralelní a perspektivní projekcí.

Geometrické transformace vektorových objektů jsou jednou z nejčastěji používaných operací v současné počítačové grafice.

Objekty popsané ve vektorové grafice mohou být definovány mnoha různými způsoby (viz. kapitola ???). Vždy je však jejich popis založen na uzlových bodech, vrcholech. Při jejich zobrazování, vytváření a upravování je potřeba velmi často provádět (řádově 1000 – 1000 000 x) jejich: posunutí, otočení, zmenšení/zvětšení nebo zkosení. Těmto operacím se pak souhrnně říká *geometrické transformace* a jsou jedním z nejdůležitějších a nejpoužívanějších nástrojů moderní počítačové grafiky. Prvním krokem pro akceleraci počítačové grafiky proto většinou bývá hardwarová implementace právě geometrických transformací.

Geometrické transformace můžeme chápat jako změnu pozice vrcholů v aktuálním souřadnicovém systému nebo jako změnu souřadnicového systému.

Lineární transformace je zobrazení f z jednoho vektorového prostoru do druhého $V \rightarrow W$, které zachovává lineární kombinace. Pro libovolný vektor $x(x_1, x_2, \dots, x_n)$ a skalár $\alpha_1, \alpha_2, \dots, \alpha_n$ platí $f(\alpha_1 x_1, \alpha_2 x_2, \dots, \alpha_n x_n) = \alpha_1 f(x_1), \alpha_2 f(x_2), \dots, \alpha_n f(x_n)$. Všechny základní geometrické transformace jsou lineární.



7.1 Homogenní souřadnice

Souřadnice bodu v rovině je $P(x, y)$. Pro jeho posunutí na pozici $P'(x', y')$ musíme k aktuálním souřadnicím přičíst vektor posunutí $T(d_x, d_y)$. Nové souřadnice pak získáme podle vztahů 7.1. Maticový zápis najdeme ve vztahu 7.2. Podobně je tomu, pokud chceme daný bod vůči počátku souřadnic: otočit, změnit měřítko nebo zkosit (viz. vztahy 7.3 – 7.5). V tom případě R , S a S_H jsou transformační matice pro rotaci, měřítko a zkosení.

$$x' = x + dx, \quad y' = y + dy \quad (7.1)$$

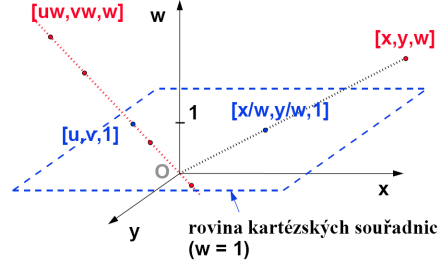
$$P' = P + T \quad (7.2)$$

$$P' = P \cdot R \quad (7.3)$$

$$P' = P \cdot S \quad (7.4)$$

$$P' = P \cdot S_H \quad (7.5)$$

Naneštěstí je posun bodu vyjádřen jinou operací (součet, viz. vztah 7.2) než rotace, měřítko a zkosení (násobení, viz. vztahy 7.3 – 7.5). To je velmi nevýhodné,



Obrázek 7.1: Geometrická představa homogenních souřadnic ve 2D.

protože nemůžeme se všemi těmito transformacemi pracovat jednotně, jako se součinem matic a vektorů. Proto je naší snahou převést všechny transformace včetně posunutí na jednotnou formu.

Řešením tohoto problému jsou *homogenní souřadnice bodu* (viz. obr. 7.1). Homogenní souřadnice jsou zároveň jednotnou reprezentací afinních transformací. Umožňují nám vyjádřit všechny druhy základních transformací (včetně posunutí) jednou transformační maticí a aplikovat je násobením matic a vektorů.

Homogenní souřadnice bodu ve 3D s kartézskými souřadnicemi $[x, y, z]$ je uspořádaná čtveřice $[X, Y, Z, w]$ pro kterou platí $x = X/w$, $y = Y/w$, $z = Z/w$. Bod je svými homogenními souřadnicemi určen jednoznačně. Souřadnici w nazýváme *váhou bodu*. Hodnota váhy je většinou $w = 1$, v případech lineárních transformací.

DEF

7.2 Základní geometrické transformace ve 2D

7.2.1 Posunutí

Posunutí bodu v rovině s homogenními souřadnicemi $P(x, y, 1)$ s vektorem posunutí $T(d_x, d_y)$ je dáno vztahem 7.6 (viz. obr. 7.2a). Maticový zápis je pak ve vztahu 7.7 a 7.8. Tvar transformační matice T posunutí a její inverzní formy T^{-1} (posunutí zpátky, opačným směrem) najdeme ve vztahu 7.9 a 7.10

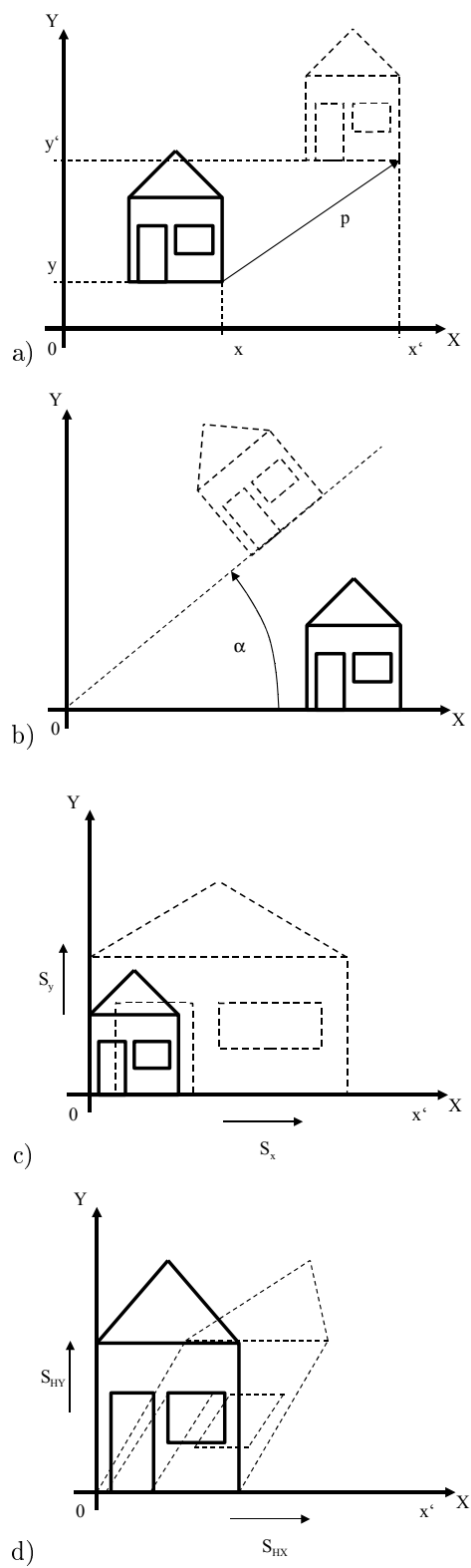
$$x' = x + d_x, \quad y' = y + d_y \quad (7.6)$$

$$[x', y', 1] = [x, y, 1] \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ d_x & d_y & 1 \end{bmatrix} \quad (7.7)$$

$$P' = P \cdot T \quad (7.8)$$

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ d_x & d_y & 1 \end{bmatrix} \quad (7.9)$$

$$T^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -d_x & -d_y & 1 \end{bmatrix} \quad (7.10)$$



Obrázek 7.2: Schéma základních lineárních geometrických transformací:
a) posun, b) rotace, c) měřítko, d) sklonění.

7.2.2 Otáčení

Otočení bodu v rovině s homogenními souřadnicemi $P(x, y, 1)$ o úhel α se středem otáčení v počátku souřadného systému je dáno vztahem 7.11 (viz. obr. 7.2b). Kladný směr otáčení je proti směru hodinových ručiček, od osy X k ose Y, podle pravidla apravé ruky. Maticový zápis je pak ve vztahu 7.12 a 7.13. Tvar transformační matice R rotace a její inverzní formy R^{-1} (otočení zpět, opačně) najdeme ve vztahu 7.14 a 7.15

$$x' = x \cdot \cos \alpha - y \cdot \sin \alpha, \quad y' = x \cdot \sin \alpha + y \cdot \cos \alpha \quad (7.11)$$

$$[x', y', 1] = [x, y, 1] \cdot \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.12)$$

$$P' = P \cdot R \quad (7.13)$$

$$R = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.14)$$

$$R^{-1} = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.15)$$

7.2.3 Změna měřítka

Změna měřítka bodu v rovině s homogenními souřadnicemi $P(x, y, 1)$ a faktory změny měřítka S_x a S_y je dána vztahem 7.16 (viz. obr. 7.2c). Maticový zápis je pak ve vztahu 7.17 a 7.18. Tvar transformační matice S změny měřítka a její inverzní formy S^{-1} (inverzní změna měřítka) najdeme ve vztahu 7.19 a 7.20

Je-li faktor změny měřítka $S_{x,y} > 1$, dochází ke zvětšení. Je-li $0 < S_{x,y} < 1$, dochází ke zmenšení. Je-li $S_{x,y} < 0$, dochází k převrácení (zrcadlení).

$$x' = x \cdot S_x, \quad y' = y \cdot S_y \quad (7.16)$$

$$[x', y', 1] = [x, y, 1] \cdot \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.17)$$

$$P' = P \cdot S \quad (7.18)$$

$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.19)$$

$$S^{-1} = \begin{bmatrix} 1/S_x & 0 & 0 \\ 0 & 1/S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.20)$$

7.2.4 Zkosení

Zkosení bodu v rovině s homogenními souřadnicemi $P(x, y, 1)$ a faktory zkosení S_{hx} a S_{hy} je dána vztahem 7.21 (viz. obr. 7.2d). Maticový zápis je pak ve vztahu 7.22 a 7.23. Tvar transformační matice S_H zkosení a její inverzní formy S_H^{-1} (opačné zkosení) najdeme ve vztahu 7.24 a 7.25

$$x' = x + S_{hx} \cdot y, \quad y' = y + S_{hy} \cdot x \quad (7.21)$$

$$[x', y', 1] = [x, y, 1] \cdot \begin{bmatrix} 1 & S_{hx} & 0 \\ S_{hy} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.22)$$

$$P' = P \cdot S_H \quad (7.23)$$

$$S_H = \begin{bmatrix} 1 & S_{hx} & 0 \\ S_{hy} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.24)$$

$$S_H^{-1} = \begin{bmatrix} 1 & -S_{hx} & 0 \\ -S_{hy} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.25)$$

7.3 Transformace ve 3D

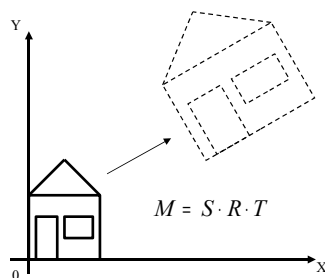
Geometrické transformace ve 3D jsou logickým rozšířením 2D transformací. Transformované body jsou popsány homogenními 3D souřadnicemi $P(x, y, z, w)$. Maticový zápis jednotlivých základních transformací je stejný pro 2D i 3D (viz. vztahy 7.2 až 7.5). Transformační matice T pro posun ve 3D je pouze rozšířením dimenze 2D matice (viz. vztah 7.26). Totéž se týká 3D transformační matice S pro změnu měřítka (viz. vztah 7.27).

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ d_x & d_y & d_z & 1 \end{bmatrix} \quad (7.26)$$

$$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.27)$$

Poněkud jiná situace nastává u rotační transformace ve 3D. Stejně jako ve 2D probíhá rotace okolo počátku souřadného systému. Máme však různé transformační matice R_x , R_y , R_z pro rotaci okolo souřadných os X , Y a Z (viz. vztahy 7.28 – 7.30).

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & \sin \alpha & 0 \\ 0 & -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.28)$$



Obrázek 7.3: Skládání transformací, změna měřítka, rotace a posun.

$$R_y = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.29)$$

$$R_z = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 & 0 \\ -\sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.30)$$

Podobně jako u rotační transformace ve 3D je transformace zkosení ve 3D rozdělena na tři různé operace, podle směrů, ve kterých zkosení probíhá. Máme potom tři transformační matice S_{HYZ} , S_{HXZ} , S_{HXY} pro zkosení ve směrech YZ , XZ a XY (viz. vztahy 7.31 – 7.33).

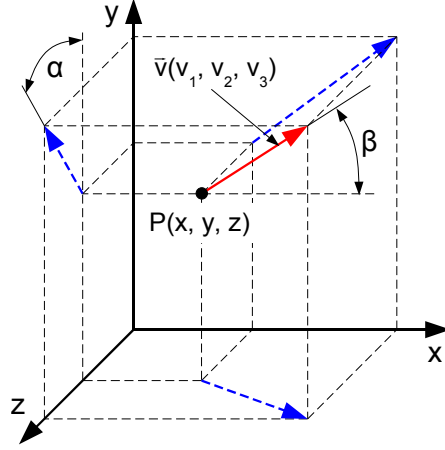
$$S_{HYZ} = \begin{bmatrix} 1 & S_{hy} & S_{hz} & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.31)$$

$$S_{HXZ} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ S_{hx} & 1 & S_{hz} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.32)$$

$$S_{HXY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ S_{hx} & S_{hy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.33)$$

7.4 Skládání transformací

Při praktickém nasazení transformací si však většinou nevystačíme se základními transformacemi (viz. kapitola 7.2). Naštěstí se dá každá požadovaná obecná lineární transformace rozložit na posloupnost základních transformací. Obecná transformace se pak dá vyjádřit jednou transformační maticí. Tuto *obecnou* transformační matici získáme *násobením* dílčích základních transformačních matic (viz. obr. 7.3). *Toto násobení matic musí být z pravé strany a v přesném pořadí provádění jednotlivých transformací.*



Obrázek 7.4: Schéma rotace okolo obecné osy.

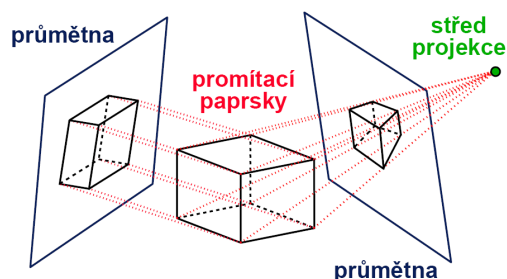
7.4.1 Rotace ve 3D kolem obecné osy

Dobrým příkladem obecné transformace může být rotace o daný úhel ω ve 3D kolem obecné osy (viz. obr. 7.4). Osa rotace je definována směrovým vektorem $\vec{v}(v_1, v_2, v_3)$ různým od některé z os souřadného systému a bodem $P(x, y, z)$ na ní ležícím, který je různý od počátku souřadného systému. Protože rotace neprobíhá kolem některé ze souřadných os, není možné přímo použít některou z variant základní 3D rotace (viz. vztahy 7.28 – 7.30). Požadovanou transformaci je proto potřeba rozložit na posloupnost základních transformací (viz. vztah 7.34). Postup nalezení této posloupnosti základních transformací je následující:

- Posunutí osy rotace do počátku souřadného systému. K tomu použijeme opačný vektor pozice bodu na ose rotace.
- Otočení posunuté osy rotace do jedné z souřadných rovin (např. XY). K tomu využijeme rotaci okolo jedné ze souřadných os (X) o úhel α odpovídající směrovému kosinu průmětu vektoru osy do kolmé souřadné roviny (YZ).
- Otočení sklopené osy rotace do jedné ze souřadných os (X). K tomu využijeme rotaci okolo jedné ze souřadných os (Z) o úhel β odpovídající směrovému kosinu vektorem osy a osy do které sklápíme (XZ).
- Provedení požadované rotace o daný úhel ω kolem příslušné souřadné osy (X). K tomu využijeme standardní 3D rotaci okolo příslušné souřadné osy.
- Vrácení osy rotace do původní polohy. K tomu provedeme násobení inverzními transformačními maticemi předchozích transformací v opačném pořadí.

$$M = T \cdot R_X \cdot R_Z \cdot R_{X(\omega)} \cdot R_Z^{-1} \cdot R_X^{-1} \cdot T^{-1} \quad (7.34)$$

Existují způsoby, jak rozložit rotaci okolo obecné osy procházející počátkem na dílčí rotace okolo souřadných os X, Y, Z . Získáme tak přímo komplexní rotační transformační matici, která příslušnou transformaci realizuje. Ještě efektivnější způsob práce s rotačními transformacemi je použití *kvaterniony*.



Obrázek 7.5: Paralelní (vlevo) a perspektivní (vpravo) projekce.

7.5 Projekce

Možnost definovat vektorové a rastrové objekty, provádět s nimi požadované operace, atd. je nutnou součástí Počítačové grafiky (viz. kapitola ??), nikoli však postačující. Pro praktické použití je další nutnou součástí také grafické zobrazení získaných výsledků.

Pracujeme-li s 2D objekty, stačí jejich převedení na odpovídající reprezentaci podle požadavků výstupního zařízení (viz. kapitola ??). Pracujeme-li však s 3D objekty, musíme je nejprve převést do 2D, protože prakticky všechna zatím dostupná zobrazovací zařízení pracují ve 2D.

Aplikací 3D geometrických transformací na vytvořené 3D objekty zajistíme jejich požadovanou orientaci a polohu vůči pozorovateli (kameře) nebo pozorovateli vůči nim. Stále však budeme mít 3D objekty. Pro jejich převedení do 2D musíme ještě provést jejich projekci (promítnutí). Při tom samozřejmě dochází k nevratné ztrátě geometrické informace. Z hlediska zobrazení je však výsledek stejný jako při pohledu na původní 3D objekty.

Projekce je transformace realizující redukci dimenze 3D prostoru na 2D prostor při zachování parametrů použitého zobrazení. Projekce provádí redukci dimenze prostoru v definovaném směru (směru pohledu), nejčastěji ve směru osy $-Z$. Pracuje se s vrcholy projektovaných objektů a zachovává se druh jejich reprezentace (rastrová, vektorová).

DEF

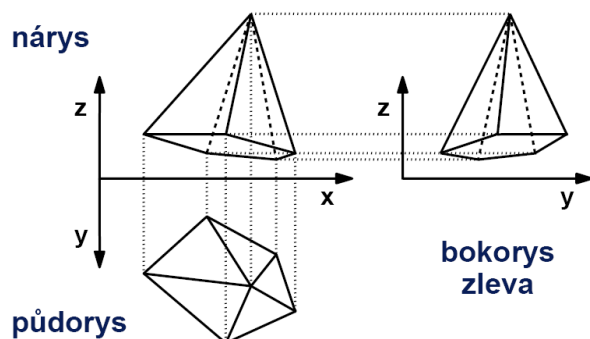
Geometrickou ilustrací pro 3D projekci je promítání jednotlivých vrcholů objektů na průmětnu (promítací plochu) prostřednictvím paprsků (viz. obr. 7.5). Podle způsobu vrhání paprsků rozlišujeme dva hlavní druhy projekce:

- Paralelní (rovnoběžnou) projekci.
- Perspektivní (středovou) projekci.

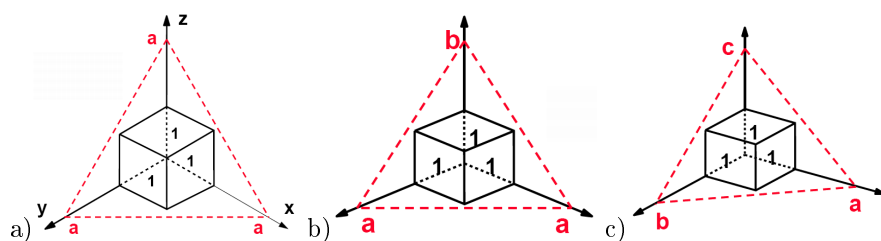
Předpokládáme zároveň, že promítací plochou je rovina. V tom případě se úsečky promítají na úsečky a stačí proto promítat pouze vrcholy. Při promítání na *zakřivenou* průmětnu (panorama, rybí oko) se úsečky promítají na křivky a musíme proto jednotlivé entity dělit na menší části promítané samostatně.

7.5.1 Paralelní

Paralelní projekce je lineární projekce, která zobrazuje vrcholy promítaných objektů prostřednictvím rovnoběžných paprsků. Velikost průmětů objektů není závislá na jejich vzdálenosti od průmětny. Rovnoběžnost promítaných hran je v této projekci zachována.



Obrázek 7.6: Mongeova projekce.



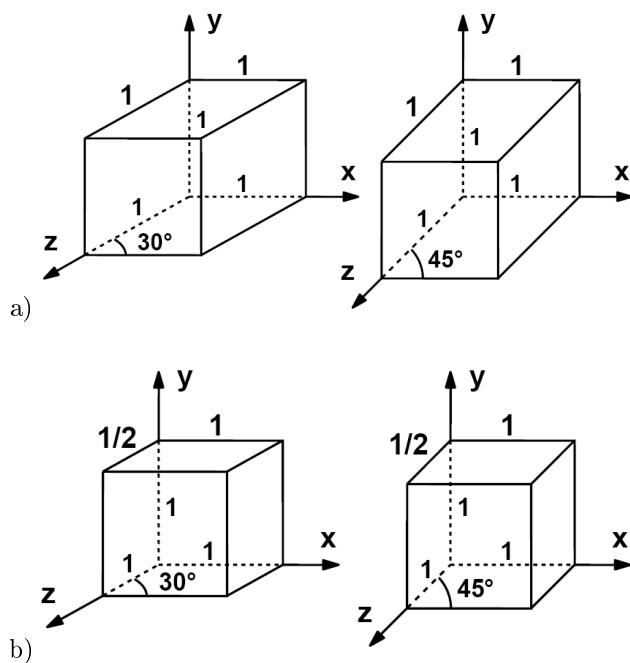
Obrázek 7.7: Axonometrická projekce: a) isometrie, b) dimetrie, c) trimetrie.

Podle úhlu dopadu promítacích paprsků na průmětnu rozlišujeme dva základní druhy paralelní projekce:

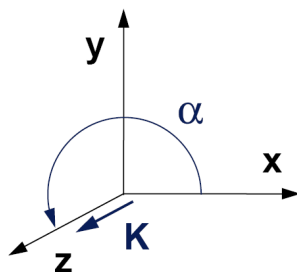
- Kolmá projekce – promítací paprsky jsou kolmé na průmětnu:
 - Mongeova projekce – zobrazení v základních pohledech (narys, půdorys, bokorys) získáno promítáním ve směrech souřadných os X , Y a Z (viz. obr. 7.6).
 - Axonometrická projekce – obecná kolmá projekce, isometrie, dimetrie, trimetrie (viz. obr. 7.7).
- Kosoúhlá projekce – úhel dopadu paprsků na průmětnu < 90 . Rovina XY je rovnoběžná s průmětnou, osa Z je sklopena pod úhlem.
 - Kavalírska projekce – všechny osy mají stejné měřítko (viz. obr. 7.8a).
 - Kabinetní projekce – osa Z je v měřítku $1/2$ vůči ostatním osám (viz. obr. 7.8b).

Praktickou realizací kolmé projekce je zanedbání (nastavení na 0) souřadnice promítaných vrcholů, které odpovídají ose promítání (nejčastěji osa Z). Zbylé dvě osy pak tvoří průmětnu (XY). Tuto operaci lze snadno realizovat prostřednictvím jednoduché transformační matice (viz. vztah 7.35).

$$P_{XY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.35)$$



Obrázek 7.8: Kosoúhlá projekce: a) kavalírska, b) kabinetní.



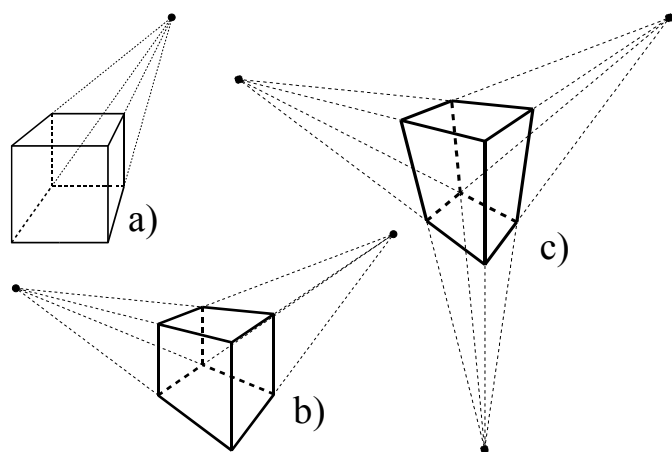
Obrázek 7.9: Kosoúhlá projekce, realizace.

Praktickou realizaci kosoúhlé projekce lze také snadno realizovat prostřednictvím jednoduché transformační matice (viz. vztah 7.36 a obr 7.9). Osy XY tvoří průmětnu, K je poměr zkrácení osy Z a α je úhel průmětu osy Z . Pro kavalířskou projekci je $K = 1$, pro kabinetní projekci je pak $K = 0.5$.

$$P_{XY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ K \cos \alpha & K \sin \alpha & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.36)$$

Pro získání základních pohledů v Mongeově projekci provedeme kolmou projekci v všech třech souřadných osách. Promítaný objekt při tom zůstává v základní souřadné poloze (viz. obr. 7.6).

Použití paralelní projekce je nejčastěji pro technické zobrazení v CAD/CAM systémech při tvorbě výkresové dokumentace (Mongeova projekce), při zobrazení 3D objektů (Axonometrie). Pro ilustrační technické 3D náčrty a schémata se pak používá kosoúhlá projekce.



Obrázek 7.10: Druhy perspektivní projekce: a) jednobodová, b) dvoubodová, c) tříbodová.

7.5.2 Perspektivní projekce

Perspektivní projekce je nelineární neafinní projekce, která zobrazuje vrcholy promítaných objektů prostřednictvím paprsků protínajících se v jednom bodě, ve středu projekce. Střed projekce je zároveň většinou místem pozice pozorovatele. Velikost průmětů objektů je nepřímoúměrně závislá na jejich vzdálenosti od průmětny. Čím je objekt blíže průmětně, tím je jeho obraz větší a naopak. Rovnoběžnost promítaných hran není v této projekci zachována.

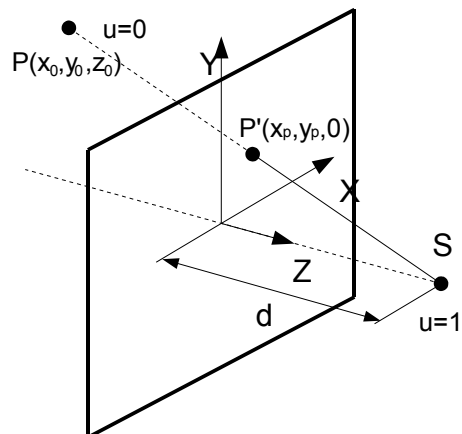
DEF

Perspektivní projekce odpovídá našemu lidskému vidění reálného světa. Proto se nejčastěji používá pro realističtější zobrazení ve virtuální realitě, architektuře, počítačových 3D hrách, atd.

Průmětna může mít obecně jakoukoli polohu vůči souřadnému systému zobrazovaných objektů. Pro zjednodušení však rozlišujeme tři následující případy:

- Jednobodová perspektiva – průmětna je rovnoběžná se dvěma souřadnými osami. Hraný objektů rovnoběžné se třetí osou se sbíhají do jednoho bodu – úběžníku (viz. obr. 7.10a).
- Dvoubodová perspektiva – průmětna je rovnoběžná s jednou souřadnou osou. Hraný objektů rovnoběžné se zbylými dvěma osami se sbíhají do dvou úběžníků (viz. obr. 7.10b).
- Tříbodová perspektiva – průmětna má vůči souřadnému systému obecnou polohu. Hraný objektů rovnoběžné se souřadnými osami se sbíhají do tří úběžníků (viz. obr. 7.10c).

Úběžníky představují "průsečíky rovnoběžek ležících v nekonečnu". V perspektivní projekci můžeme nalézt velké množství průsečíků. Prakticky však zobrazujeme pouze ty, ve kterých se protínají hrany rovnoběžné se souřadnými osami. Jejich význam ve vícebodové perspektivě je především v architektuře, kde zobrazujeme "hranaté" objekty orientované podél souřadného systému.



Obrázek 7.11: Schéma odvození perspektivní projekce.

Stejně jako paralelní projekci můžeme i perspektivní projekci vyjádřit prostřednictvím transformace s maticí P_{per} (viz. vztah 7.40 a 7.41). Její odvození vycházející z parametrického vyjádření promítacího paprsku najdeme ve vztazích 7.37 – 7.39 (viz. obr. 7.11). Transformační matici pro vícebodovou perspektivu pak najdeme ve vztahu 7.42.

$$x = x - xu, \quad y = y - yu, \quad z = z - (z + d)u \quad (7.37)$$

$$u \in \langle 0, 1 \rangle, \quad u = 0 \rightarrow (x, y, z), \quad u = 1 \rightarrow (0, 0, d) \quad (7.38)$$

$$z_P = 0 \rightarrow u = z/(z + d), \quad x_P = x \frac{1}{1 + z/d}, \quad y_P = y \frac{1}{1 + z/d} \quad (7.39)$$

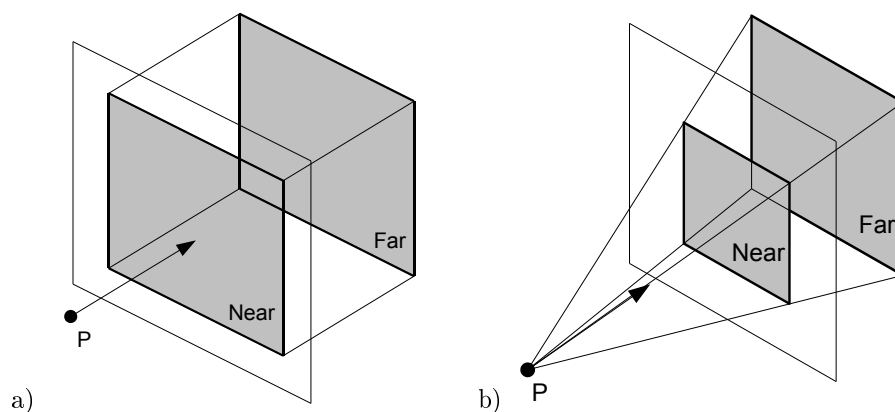
$$(x_P, y_P, z_P, w_P) = (x, y, z, w) \cdot P_{per}, \quad w_P = 1 + z/d \quad (7.40)$$

$$P_{per} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.41)$$

$$P_{per} = \begin{bmatrix} 1 & 0 & 0 & 1/d_x \\ 0 & 1 & 0 & 1/d_y \\ 0 & 0 & 1 & 1/d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7.42)$$

7.5.3 Pohledový objem

Stejně jako v rovině je i v prostoru zbytečné zobrazovat objekty, které nejsou vidět, protože leží mimo pohled nebo jsou příliš blízko nebo příliš daleko. V rovině provádíme oříznutí okrajem zobrazovacího okna (viz. kapitola ??). V prostoru provádíme oříznutí *pohledovým objemem* (viz. obr. 7.12).



Obrázek 7.12: Pohledový objem: a) paralelní projekce, b) perspektivní projekce.

Pohledový objem je část 3D prostoru, která obsahuje zobrazované (viditelné) objekty. Pohledový objem je ohraničen okraji okna protaženými do prostoru do tvaru hranolu (při paralelní projekci) nebo komolého jehlanu (při perspektivní projekci). V směru pohledu je objem uzavřen přední a zadní ořezávací stěnou.

Kromě odstranění objektů mimo pohled a tím urychlení zobrazení pomáhá oříznutí pohledovým objemem také odstranit singulární body (úběžníky, atd.), které vznikají především při projekci. Pro snadnější realizaci ořezání provedeme ještě před projekcí transformaci pohledového objemu na jednotkovou krychli, jednotkový jehlan.

Shrnutí

Geometrické transformace vektorových objektů jsou jednou z nejčastěji používaných a nejdůležitějších operací moderní počítačové grafiky. Proto bývají velmi často hardwarově akcelerovány. Jejich prostřednictvím můžeme měnit polohu, orientaci, velikost a sklon objektů v prostoru. Transformace jsou aplikovány na body, vrcholy definující geometrii objektů.

Z důvodu jednotné maticové reprezentace základních afinních transformací (posun, rotace, měřítko, zkosení) se zavádí *homogenní souřadnice*. Jde o rozšíření kartézských souřadnic $[x, y, z]$ o váhovou souřadnici w do tvaru $[x, y, z, w]$. Pro lineární transformace je $w = 1$.

Obecné transformace lze rozložit na posloupnost základních transformací. Obecnou transformační matici získáme násobením jednotlivých základních transformačních matic, z prava, v pořadí prováděných operací.

Speciálním druhem transformací jsou projekce, které zajišťují redukci dimenze 3D prostoru na 2D prostor pro potřeby zobrazení 3D objektů. projekci můžeme ilustrovat promítáním jednotlivých vrcholů objektů prostřednictvím paprsků na promítací plochu (průmětnu). Podle způsobu vrhání paprsků rozlišujeme: paralelní (rovnoběžné paprsky) a perspektivní projekci (paprsky do bodu).



Kontrolní otázky

1. Proč jsou geometrické transformace pro počítačovou grafiku tak důležité?
2. Jaký rozměr mají transformační matice ve 2D/3D a proč?
3. Proč mají transformační matice pro 3D rozměr 4×4 , když máme pouze tři kartézské souřadnice (x, y, z) ?
4. Jak vypadá 3D homogenní souřadnice pro bod a jak pro vektor?
5. Proč pro transformace zavádíme homogenní souřadnice, proč nám nestačí kartézské souřadnice?
6. Co se stane, bude-li váhová souřadnice homogenních souřadnic $w = 0$?
7. Jaké jsou základní 3D afinní transformace?
8. Jakým způsobem provádíme skládání transformací a k čemu je to dobré?
9. Co je to projekce a jaké základní druhy projekcí znáte?
10. Co je to paralelní projekce a jaké její základní druhy znáte?
11. Co je to perspektivní projekce a jaké její základní druhy znáte?
12. Jak prakticky realizujeme paralelní projekce?
13. Jak prakticky realizujeme perspektivní projekce?
14. Co je to pohledový objem a k čemu je to dobré?
15. Jaké je praktická oblast použití paralelní projekce?
16. Jaké je praktická oblast použití perspektivní projekce?
17. Je paralelní projekce lineární a jaké to má důsledky?
18. Je perspektivní projekce lineární a jaké to má důsledky?
19. Jak se chovají promítací paprsky v paralelní projekci?
20. Jak se chovají promítací paprsky v perspektivní projekci?
21. Jaká je velikost obrazu promítaných objektů v paralelní projekci a proč?
22. Jaká je velikost obrazu promítaných objektů v perspektivní projekci a proč?



Kapitola 8

2D křivky



1:00

Cílem této kapitoly je ...



8.1 Křivky v CG

Použití

Popis tvaru, intuitivní změna a zadávání

Interpolační X aproximační

Racionální X Neracionální

8.2 Spline křivky, spojitost křivek

8.3 Interpolační křivky

8.3.1 Fergusonova kubika

8.3.2 Cochanek/Bartels

8.3.3 Catmul-Room

8.4 Aproximační křivky

8.4.1 Beziérovky křivky

De-casteljau

8.4.2 NURBS křivky

Shrnutí

Kontrolní otázky



Kapitola 9

Datová reprezentace 3D objektů



1:00

Cílem této kapitoly je ...

Popis X zobrazení 3D dat

Popis objemu X povrchu

Převody mezi reprezentacemi

Použití reprezentací pro 3D zobrazení.



9.1 Požadavky na popis 3D dat

Manifold

9.2 CSG

Princip

Zobrazení

Vlastnosti

Šablonování, primitiva, parametrizace

9.3 B-rep

Draty

Polygony

Spline plochy

Způsoby uložení dat, okřídlená hrana, okřídlený vrchol

9.4 Voxel modely

9.5 Implicitní modely

9.6 LOD

9.7 Subdivision

9.8 3D spline plochy

Shrnutí metod, tab.

Shrnutí

Kontrolní otázky



Kapitola 10

Řešení viditelnosti 3D objektů



1:00

Cílem této kapitoly je ...



10.1 Základní myšlenka řešení viditelnosti

10.2 Vektorové algoritmy

10.2.1 Robertsův algoritmus

10.2.2 Metoda plovoucího horizontu

10.3 Rastrové algoritmy

10.3.1 Malířův algoritmus

10.3.2 Dělení obrazu

10.3.3 Z-buffer

10.3.4 Ray-casting

Shrnutí

Kontrolní otázky



Kapitola 11

Osvětlení s stínování 3D objektů



1:00

Cílem této kapitoly je ...

Objekty v reálném světě vidíme, protože na ně dopadá světlo, které se odráží na jejich povrchu a dopadá do našeho oka. Podle způsobu, jak a kam se světlo odráží vzniká prostorové (3D) vnímání objektů.

Pro popis způsobu odrazu světla na povrchu objektů používáme osvětlovací modely.



11.1 Osvětlovací modely

empirické, fyzikální

Pro zjednodušené (empirické) řešení osvětlovacího modelu rozlišujeme pouze difúzní a zrcadlový odraz světelného paprsku.

Difúzní odraz ...

Zrcadlový odraz ...

11.1.1 Lambertův osvětlovací model

11.1.2 Phongův osvětlovací model

11.2 Stínování

ne vrhání stínů

11.2.1 Konstantní stínování

11.2.2 Gourandovo stínování

11.2.3 Phongovo stínování

Shrnutí

Kontrolní otázky



Kapitola 12

Texturování 3D objektů



1:00



Cílem této kapitoly je seznámit se s podstatou a hlavními metodami texturování 3D objektů. Věnovat se budeme problematice mapování textur a hlavním způsobům použití textur pro popis povrchové struktury objektů.

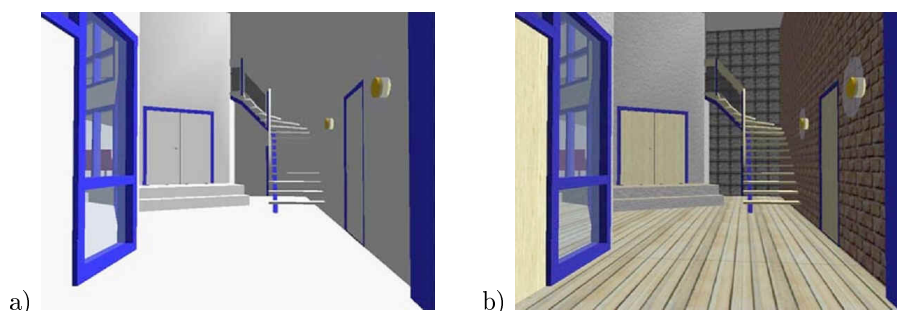
Objekty ve skutečném světě kolem nás nejsou "jen plastové koule, válce, hranoly atd." Reálné objekty mají specifickou povrchovou strukturu, barvu a optické vlastnosti, které mohou být pro každý povrchový bod více či méně rozdílné. Mnoho objektů má specifickou, charakteristickou povrchovou strukturu, texturu.

Textura je popis detailní struktury povrchu objektu, nezávislý na jeho geometrii. Jeden vzorek textury nazýváme *texel*. Proces nanášení textur na geometricky definovaný povrch objektu nazýváme *texturování*. Z hlediska vizuálního vnímání textura pomáhá vzájemně odlišovat a rozpoznávat jednotlivé objekty. Z hlediska zobrazování počítačových 3D modelů objektů umožňují textury dosáhnout řádově vyššího stupně realističnosti výsledného obrazu.



U skutečných objektů je vysoká úroveň jejich "struktury" (v celém objemu, ne jen na povrchu) dána vlastnostmi materiálů, ze kterých jsou vyrobeny. Dosáhnout podobného vizuálního efektu prostřednictvím detailního geometrického popisu by bylo velmi datově náročné. Například u polygonálních modelů by bylo potřeba pro každý strukturální prvek (list, stéblo trávy, změna barvy, šmouha nebo škrábanec na povrchu, atd.) vytvořit alespoň jeden polygon. Výsledné modely by se tak mohl blížit požadovaným realistickým parametrům, ale pro svou velikost by se s nimi nedalo pracovat (zobrazovat, upravovat, atd.). Proto je mnohem výhodnější modelovat reálné objekty geometricky jednodušeji a detailní povrchovou strukturu popsát texturou (viz. obr. 12.1).

Hodnoty texelů můžeme získat z tabulky nebo voláním funkce (procedury). Podle toho rozlišujeme textury:



Obrázek 12.1: Zobrazení scény interiéru bodovy: a) bez textury, b) s texturou.

- Datové – diskrétně navzorkované a uložené v paměťovém poli, tabulce. Vhodné pro rychlé vykreslování, dochází však k aliasu.
- Procedurální – matematicky definované výpočtem podle daných parametrů. Vhodné pro kvalitní spojitý výsledek bez aliasu, ale pomalejší.

Z hlediska datové dimenze můžeme textury rozdělit na:

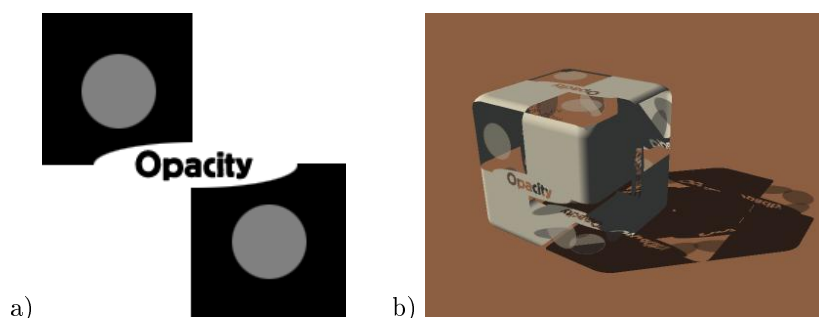
- 2D – popisují pouze povrch objektu.
- 3D – popisují prostorové rozložení vlastností v celém objemu objektu.
- 4D – jsou to 3D textury s časovou změnou, umožňují dynamické změny.

Textury mohou na povrchu objektu popisovat (mapovat) různé vlastnosti:

- Barva povrchu – nejčastější způsob použití textur (viz. obr. 12.1). Textura nanese na povrch objektu barevnou strukturu v podobě RGB informace.
- Světelné vlastnosti povrchu – *difuze a odrazivosti*. Mění charakter odrazu světla na povrchu objektu při výpočtu osvětlení (Phongův osvětlovací model, viz. kapitola ???). Simuluje lokálně ušpiněný (matný) nebo vyleštěný povrch, atd.
- Průhlednost – nerovnoměrně průhledné objekty, prolínání vlastností definovaných různými texturami přes sebe, oříznutí geometrie objektu nastavením úplné průhlednosti vybraným částem atd. Je tak možné nanesením vhodné textury na jednoduchou geometrii (např. obdelník, hranol, kouli atd.) modelovat složité objekty (listí, trávu, atd.) (viz. obr. 12.1).
- Modifikace normály – *Bump texture*. Provádí modifikaci normály podle hodnoty gradientu v textuře. Vzniká čistě optický efekt hrboлатého povrchu (viz. kapitola 12.2.1).
- Změna geometrie – *Displacement texture*. Provádí posun povrchových bodů ve směru normály podle hodnoty v textuře. Dochází ke skutečné lokální změně geometrie (viz. kapitola 12.2.2).
- *Hypertextura* – určuje optické vlastnosti nad povrchem objektu, vhodné pro zobrazení objektů typu ohně, vlasů, trávy, atd.
- Osvětlení – *Light texture*. Realizují off-line difuzní statické osvětlení povrchu.
- Zrcadlení okolí – *Environment texture*. Nanáší na lesklý povrch obraz okolí (viz. kapitola 12.2.3).

Na každý povrch objektu můžeme nanášet jednu texturu nebo i několik najednou. Vícenásobné nanášení textur pak nazýváme *Multitexturing*. Tento postup umožňuje na jednu definovaný povrch nanášet komplexní strukturní informaci (barvu, osvětlení, deformace povrchu, vlastnosti osvětlení, atd.) a vzájemně ji vhodně míchat.

Používáme-li pro texturování objektů datovou texturu (většinou), tak během rasterizace provádíme vzorkování diskretní informace rastrové (diskretní) textury. Mapované souřadnice textury jsou při tom reálné, ale souřadnice diskretní textury jsou celočíselné. Musíme tedy při zobrazování texturovaných objektů vhodně interpolovat hodnoty textury pro reálné souřadnice:



Obrázek 12.2: Použití textury s průhledností: a) použitá textura s průhledností, b) výsledná scéna s použitím průhledné textury na krychli.

- *Bez interpolace* – zaokrouhlíme souřadnice v prostoru textury na nejbližší celočíselné hodnoty. Jde tedy o výběr nejbližší hodnoty. Je to nejrychlejší a nejjednodušší postup.
- *Bilineární interpolace* – provádíme lineární interpolaci z nejbližších čtyřech hodnot. Jde o základní interpolaci, požadují se minimální výpočty a zachovává se spojitost obrazových dat.
- *Polynomiální interpolace* – provádí se interpolace z širšího okolí, polynomelem vyššího stupně (nejčastěji kubickým). Je již náročnější na výpočty, ale velmi dobře vyhlazuje obrazová data.

12.1 Mapování textur

Máme tedy pro daný objekt texturu, která detailně popisuje strukturu a vlastnosti jeho povrchu. Teď už zbývá pouze vhodně nanést texturu na povrch objektu. Tomuto procesu se říká *mapování textur*. Můžeme si ho představit, jako nalepení tapety na povrch tělesa (viz. obr. 12.3). Je-li povrch tělesa popsán plochou rozvinutelnou do roviny (např. válec, hranol), můžeme nanést texturu bez jejího zkreslení. U zborcených ploch však dojde ke zkreslení textur (např. koule). Je-li povrch objektu popsán více plochami, pak většinou nastává problém s navazováním textur v místech přechodů, podobně jako u tapetování.

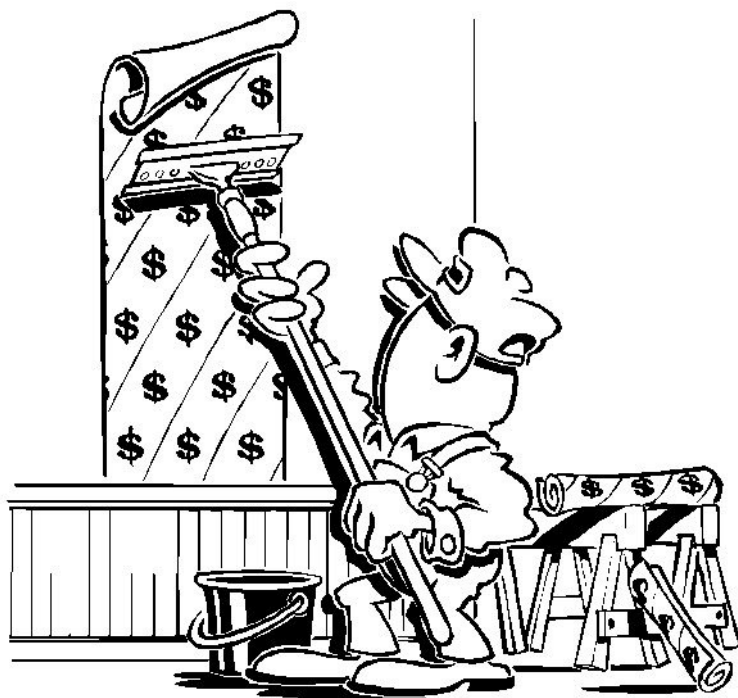
Obecně je textura popsána vhodnou funkcí $T(u, v)$, která souřadnicím v rovině (pro 2D textury, v prostoru pro 3D textury) přiřazuje hodnoty mapované vlastnosti (např. barvy) (viz. rovnice 12.1). Pro datové textury je funkce $T(u, v)$ navazována do podoby paměťové tabulky. Implementací funkce $T(u, v)$ formou procedury dostaneme procedurální texturu.

Pro nanesení textury na povrch objektu musíme ještě definovat funkci $M(x, y, z)$, která souřadnicím na povrchu objektu přiřazuje souřadnice textury (viz. rovnice 12.2). Funkci $M(x, y, z)$ potom nazýváme *inverzní mapovací funkce*.

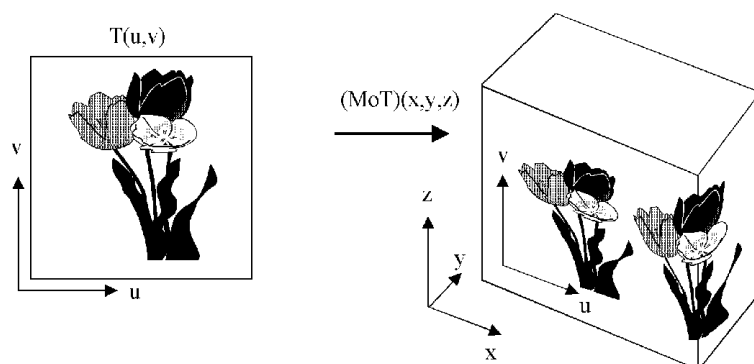
$$T(u, v) : D_T \longrightarrow H_T, \quad D_T \subset \mathbb{R}^2 - \text{prostor textury} \quad (12.1)$$

$$M(x, y, z) : D_M \longrightarrow D_T, \quad D_M \subset \mathbb{R}^3 - \text{prostor objektu} \quad (12.2)$$

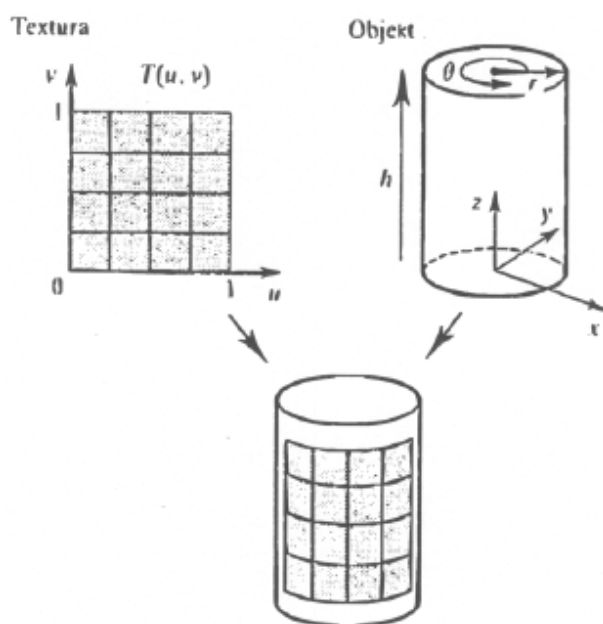
Během zobrazování objektu máme k dispozici souřadnice $[x, y, z]$ na povrchu objektu a hledáme k nim odpovídající mapované vlastnosti. Toto zobrazení je dáno spojením funkcí $T(M(x, y, z))$ (viz. obr. 12.4).



Obrázek 12.3: Ilustrace mapování textur jako tapetování.



Obrázek 12.4: Mapování textur.



Obrázek 12.5: Inverzní mapování na válec.

12.1.1 Inverzní mapovací funkce

Je-li povrch objektu popsán jedinou analytickou plochou, pak můžeme jako inverzní mapovací funkci $M(x, y, z)$ použít inverzní funkci k parametrizaci plochy, pokud taková existuje. Jako příklad jsou dále uvedeny inverzní mapovací funkce pro válec a kouli.

Pro válcovou plochu (viz. obr. 12.5) uvažujeme s umístěním podstavy v počátku souřadné soustavy. Body na válcové ploše popíšeme v cylindrických souřadnicích podle vztahů 12.3 až 12.5. Z nich odvodíme inverzní mapovací funkci $T(M(x, y, z))$, která je dána vztahy 12.6 až 12.8.

$$x = R \cos \theta, \quad \theta \in \langle 0, 2\pi \rangle \quad (12.3)$$

$$y = R \sin \theta \quad (12.4)$$

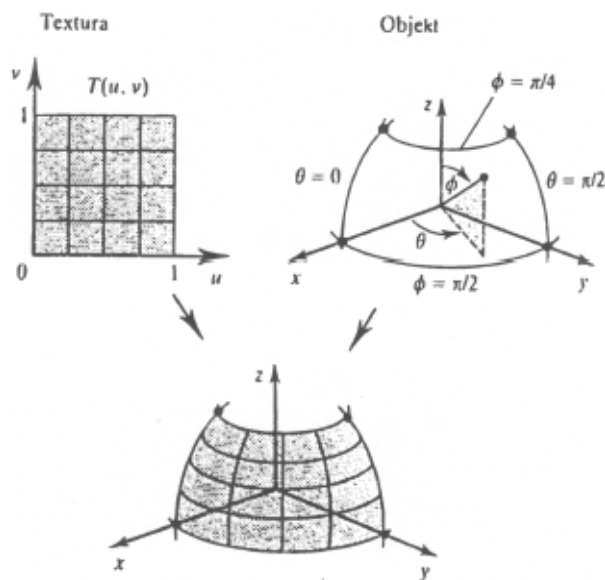
$$z \in \langle 0, h \rangle \quad (12.5)$$

$$u = \frac{1}{2\pi} \arccos \frac{x}{R}, \quad \text{pro } y \leq 0 \quad (12.6)$$

$$u = 1 - \frac{1}{2\pi} \arccos \frac{x}{R}, \quad \text{pro } y > 0 \quad (12.7)$$

$$v = \frac{z}{h} \quad (12.8)$$

Pro kulovou plochu (viz. obr. 12.6) postupujeme obdobně. Kouli uvažujeme se středem v počátku souřadné soustavy. Body na kulové ploše popíšeme v cylindrických souřadnicích podle vztahů 12.9 až 12.11. Z nich opět odvodíme inverzní mapovací funkci $T(M(x, y, z))$, která je dána vztahy 12.12 až 12.14.



Obrázek 12.6: Inverzní mapování na kouli.

$$x = R \cos \theta \cos \phi, \quad \theta \in \langle 0, 2\pi \rangle \quad (12.9)$$

$$y = R \cos \theta \sin \phi, \quad \phi \in \langle -\pi/2, \pi/2 \rangle \quad (12.10)$$

$$z = R \sin \theta \quad (12.11)$$

$$u = \frac{1}{2\pi} \arccos \frac{x}{\sqrt{x^2 + y^2}}, \quad \text{pro } y \leq 0 \quad (12.12)$$

$$u = 1 - \frac{1}{2\pi} \arccos \frac{x}{\sqrt{x^2 + y^2}}, \quad \text{pro } y > 0 \quad (12.13)$$

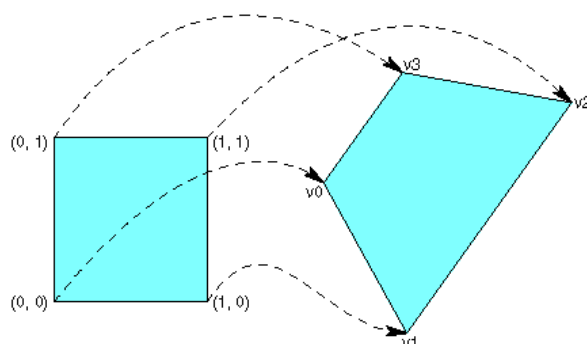
$$v = 0.5 + \frac{1}{\pi} \arcsin \frac{z}{R} \quad (12.14)$$

12.1.2 Texturovací souřadnice

Ve většině případů se dnes pracuje s 3D modely objektů popsanými Spline plochami nebo sítí polygonů (trojúhelníků). Jejich geometrie má většinou složitou topologii

Pro popis geometrie texturovaných objektů se nejčastěji používají 3D spline plochy nebo polygonální (trojúhelníkové) sítě (B-rep reprezentace, viz. kapitola ???). Topologie těchto objektů bývá většinou složitá, takže nelze získat vhodnou inverzní mapovací funkci $T(M(x, y, z))$. Proto je vhodnější použít *texturovací souřadnice*. Ty pro každý povrchový vrchol (např. uzly polygonální sítě) objektu definují odpovídající (u, v) souřadnice textury (viz. obr. 12.7). Hodnoty texturovacích souřadnic mezi definovanými hodnotami ve vrcholech jsou lineárně interpolovány (přes jednotlivé polygony).

Inverzní mapovací funkci je možné označit jako analytický způsob mapování textury na povrch objektu, kdy pro souřadnici na povrchu objektu přímo vypočítáme hodnotu textury. Texturovací souřadnice jsou pak po částech (polygonech) aproximovanou diskretizací mapování textury v jednotlivých vrcholech objektu.



Obrázek 12.7: Mapování textur s použitím texturovacích souřadnic.

Tak dokážeme snadno a jednoznačně mapovat texturu na povrch libovolného objektu i v případech, kdy není možné získat inverzní mapovací funkci. Problém se tím však pouze posouvá z oblasti vlastního mapování do oblasti získání vhodných hodnot texturovacích souřadnic pro jednotlivé vrcholy objektu.

U geometricky a topologicky jednodušších objektů je výhodné nastavit hodnoty texturovacích souřadnic ručně. Dosáhneme tak nejkvalitnějšího výsledku, protože můžeme detailně kontrolovat a *ladit* kvalitu mapování: minimalizovat deformaci textur, zajistit jejich správné navazování, atd.

V případě složitějších objektů (nerozvinutelný, rozvětvený, atd.) již není možné kontrolovat texturovací souřadnice ručně. Potom je vhodné použít nějakou automatickou metodu, např. *promítání textur* (viz. kapitola 12.1.3).

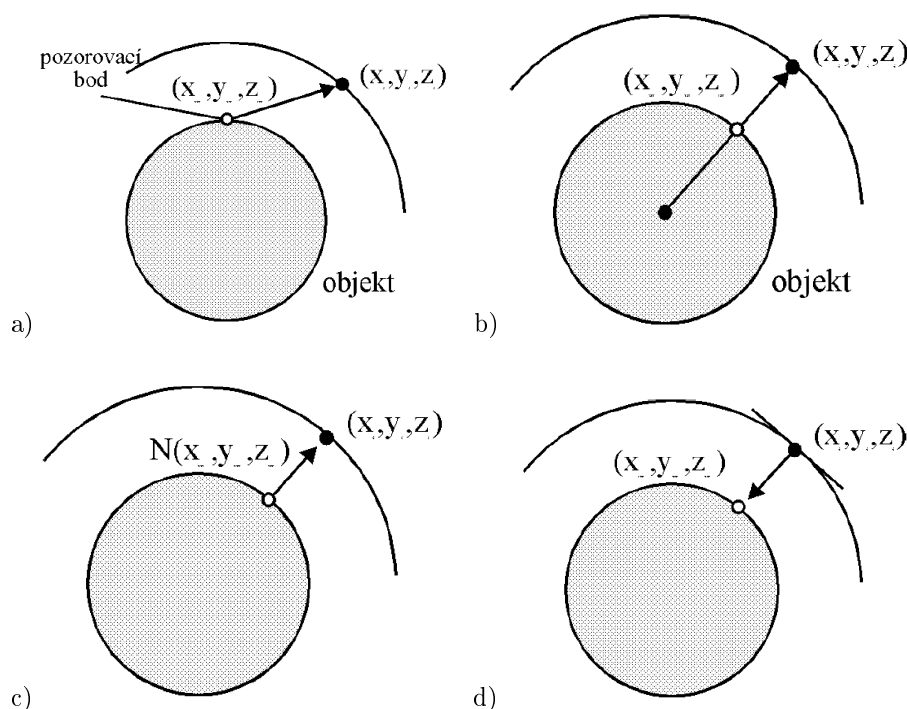
12.1.3 Promítání textur

Promítání textur je způsob jak automaticky získat hodnoty texturovacích souřadnic. Dosáhneme tak v mnoha případech dostatečně kvalitní a navazující mapování textur i na geometricky složitějších objektech (viz. obr. 12.9).

Místo nanášení textury přímo na požadovaný objekt, nanese textury s využitím inverzní mapovací funkce na pomocný objekt (rovinu, kouli, hranol, válec, atd.), který je topologicky jednoduchý, lze pro něj nalézt inverzní mapovací funkci a svým charakterem blízký texturovanému objektu. Pomocný objekt je opsaný texturovanému objektu. Mezi povrchem texturovaného a pomocného objektu je potřeba nalézt vhodné zobrazení. Jeho prostřednictvím pak přeneseme texturovací souřadnice nanášené textury přes pomocný objekt na povrchové vrcholy texturovaného objektu.

Hlavní způsoby zobrazení mezi povrchem texturovaného a pomocného objektu jsou následující:

- Promítnutí odraženým paprskem od pozorovatele přes texturovaný objekt na pomocný objekt (viz. obr. 12.8a).
- Promítnutí ze středu texturovaného na pomocný objekt (viz. obr. 12.8b).
- Promítnutí normálou texturovaného objektu na pomocný objekt (viz. obr. 12.8c).
- Promítnutí normálou pomocného objektu na texturovaný objekt (viz. obr. 12.8d).



Obrázek 12.8: Způsoby zobrazení povrchů při promítání textur: a) odrazem od pozorovatele z texturovaného na pomocný objekt, b) ze středu texturovaného objektu, c) normálou texturovaného objektu, d) normálou pomocného objektu.

12.1.4 Mapování 3D textur

Analogií pro mapování 2D textur je tapetování. Pro mapování 3D textur je vhodnou analogií vyřezání (frézování) objektu z jednoho bloku materiálu (polotovaru).

Při mapování 3D textur nepotřebujeme složitě hledat inverzní mapovací funkci nebo generovat texturovací souřadnice. Textura je definována jako jednotková krychle v prostoru u, v, w . Pro získání hodnoty textury v bodě $[x, y, z]$ provádíme *pouze* transformaci (posun, rotace, měřítko) prostoru objektu na prostor textury.

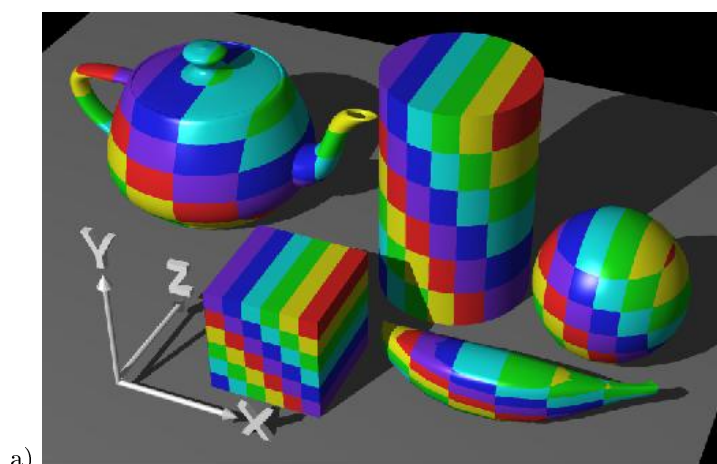
Aplikací 3D textur odstraníme problém kvality mapování a navazování 2D textur. Objekty s 3D texturou mohou mít bez problému velmi složitou prostorovou strukturu, jako je dřevo, mramor, atd. (vir. obr. 12.10).

Hlavní nevýhodou 3D textur je jejich paměťová náročnost, při uložení jejich definičních dat. V tom nám může pomoci komprese textur, která je dnes často podporována na úrovni hardware grafických karet. Další možností je uložit data textury z nižším rozlišením a přesnější hodnoty se dopočítávají interpolací, která je také často podporována na úrovni hardware grafických karet.

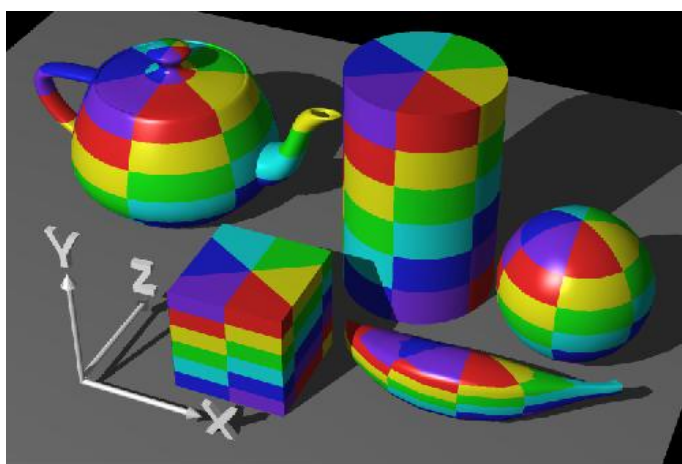
12.2 Aplikace textur

12.2.1 Bump mapping

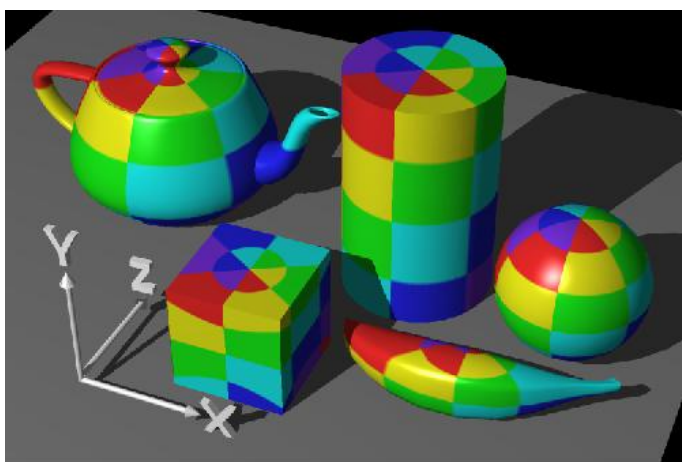
Bump mapping je způsob aplikace textury na povrch objektu, který u hladké geometrie vytváří efekt *hrbolatého* povrchu (vir. obr. 12.11). Jde o čistě optický efekt, který nemá vliv na vlastní geometrii objektu. Obrysy objektu tedy zůstávají nezměněny. Mapování textury je prováděno některým ze *standardních* způsobů (viz.



a)

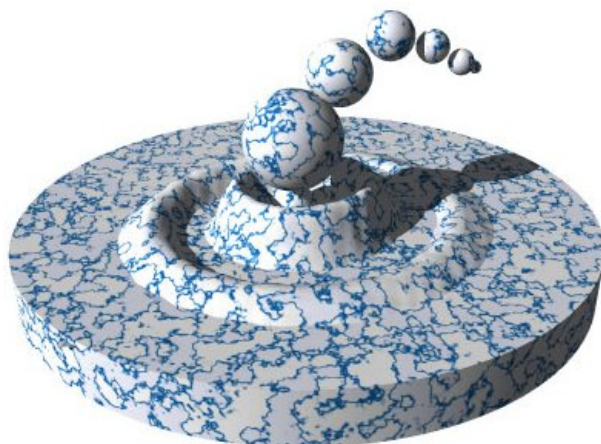


b)



c)

Obrázek 12.9: Příklady mapování textury promítáním: a) promítání z roviny, b) promítání z válce, c) promítání z koule.



Obrázek 12.10: Mapování 3D textury.

kapitola 12.1). Na povrch objektu jsou jako vlastnosti nanášeny požadované optické změny povrchu (gradienty). Efektu hrboлатého povrchu je dosahováno modifikací normál povrchu podle hodnot v textuře (vir. obr. 12.12). Bump mapping je vhodný pro simulaci drobných nerovností na povrchu objektu, kdy je výsledek dostačně realistický.

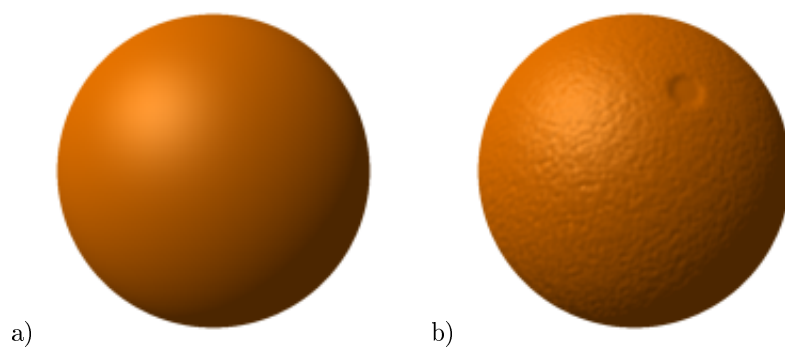
12.2.2 Displacement mapping

Displacement mapping je, podobně jako Bump mapping, způsob aplikace textury na povrch objektu, který vytváří efekt *hrboлатého* povrchu (vir. obr. 12.14). Na rozdíl od Bump mapping se však nejedná o čistě optický efekt. Dochází zde ke skutečnému posunu vrcholů objektu podle hodnot v textuře, ve směru normály povrchu. Obrysy objektu je tedy odpovídajícím způsobem změněn. Mapování textury je prováděno některým ze *standardních* způsobů (viz. kapitola 12.1). Displacement mapping je vhodný pro simulaci větších změn (oproti Bump mappingu) ve struktuře povrchu objektu.

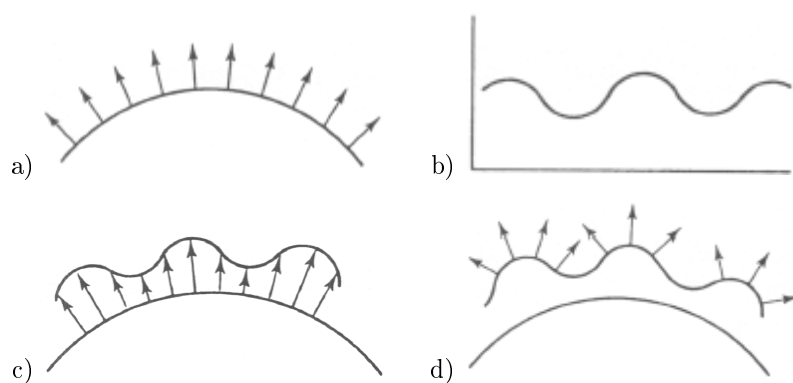
Vlastní geometrie objektu tedy zabezpečuje jeho základní tvar a rozměry. Displacement mapping pak zajišťuje detaily povrchu. Bump mapping se spolu s barevnou texturou nakonec postará o jemnou plastickou a barevnou strukturu povrchu (vir. obr. 12.15).

12.2.3 Environmental mapping

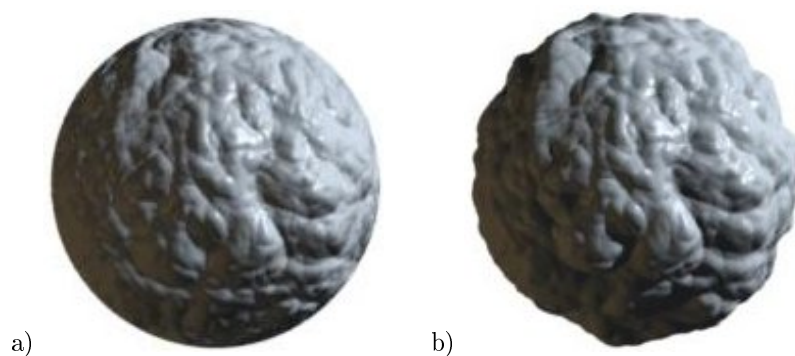
Environmental mapping je způsob aplikace textury na objekt, který na lesklém povrchu vytváří efekt *odrazu* (odlesku/zrcadlení) svého okolí (vir. obr. 12.16). Pokud zobrazovací metoda implicitně neřešení zrcadlení okolí na povrchu objektů (vir. kapitola ???), lze tímto způsobem příslušný efekt simulovat. Základem je textura, která představuje obraz okolí pro vybraný objekt. Tato textura je standardním způsobem s využitím inverzní mapovací funkce nanášena na obalové těleso (koule nebo krychli) (vir. obr. 12.17, 12.18). Z obalového tělesa je textura promítáním odraženého paprsku mapována na povrch zrcadlicího objektu (vir. kapitola 12.1.3).



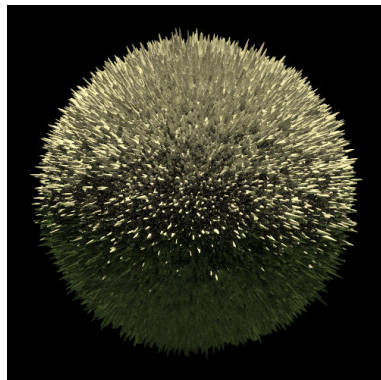
Obrázek 12.11: Výsledný efekt aplikace bump textury: a) povrch bez textury, b) povrch s bump texturou.



Obrázek 12.12: Způsob úpravy povrchu objektu při bump mappingu: a) původní povrch, b) profil textury (gradienty), c) úprava profilu povrchu (délky normál) podle textury, d) úprava směrů normál podle textury.



Obrázek 12.13: Srovnání výsledného efektu pro: a) bump mapping, b) displacement mapping.



Obrázek 12.14: Displacement mapping efekt na povrchu koule.

12.3 MIP mapping

MIP (Multum in Parvo) – mnoha v malém. Jde o způsob uložení jedné textury (jejího obrazu) v řadě velikostí do jedné datové matice (viz. obr. 12.19). Textura je při tom rozložena na tři matice pro R, G a B barevný kanál. Každý kanál je potom samostatně uložen do 1/4 datové matice dvojnásobné velikosti. Do zbylé nevyužitě 1/4 je rekurzivně uložena stejná textura poloviční velikosti (viz. obr. 12.20). Moderní 3D grafické karty většinou podporují přímé použití MIP textur na úrovni jejich hardware.

Podle vzdálenosti texturovaného objektu (velikosti jeho obrazu) je automaticky vypočtena hodnota textury (viz. obr. 12.19). Hodnota textury se bere přímo z textury odpovídajícího rozlišení nebo se vypočítá lineární interpolací ze dvou nejbližších rozlišení. Jde tedy o použití hodnoty textury v nejvhodnějším rozlišení podle velikosti obrazu texturovaného objektu a tím jeho vzorkovací frekvence.

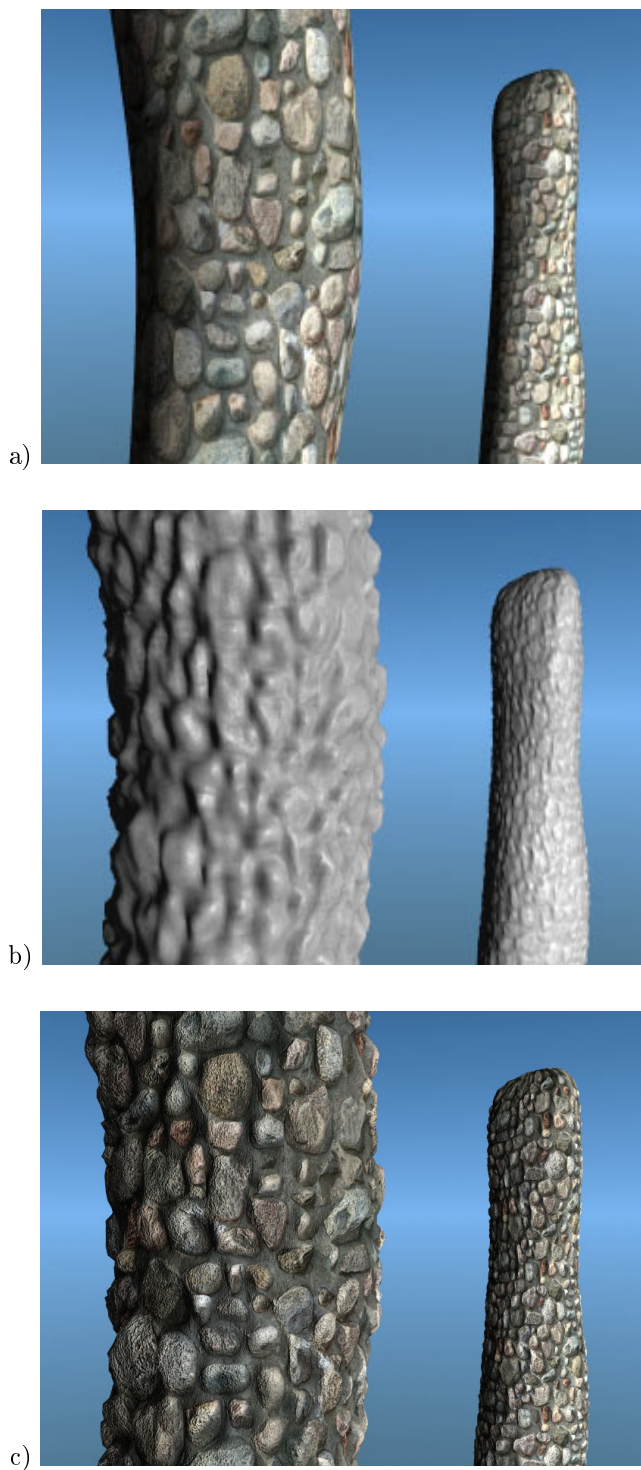
Použití MIP textur má dvojí význam:

- Zrychlení zobrazení vzdálenějších objektů použitím menších verzí textur.
- Odstranění aliasu textur při zobrazení texturovaných objektů v perspektivní projekci (viz. obr. 12.21). Předpokladem je vhodná filtrace při získávání menších rozlišení obrazu textury.

12.4 Perspektivní korekce textur

Při zobrazování texturovaných objektů v perspektivní projekci (viz. kapitola ???) dochází ke zkreslení mapování textur na jejich povrchu. Toto zkreslení se projevuje jako *prohnutí* textur uvnitř texturovaných polygonů (viz. obr. 12.23).

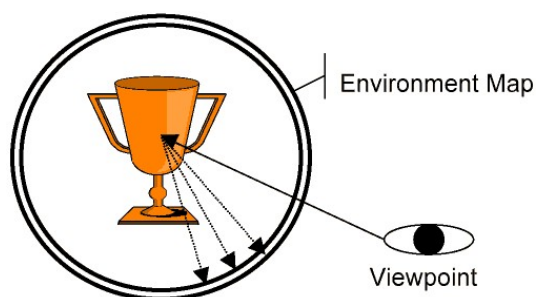
Ve vrcholech objektu mapováním získáme texturovací souřadnice (viz. kapitola 12.1.2). Přes jednotlivé polygony jsou texturovací souřadnice (u, v) lineárně interpolovány. V perspektivní projekci jsou však s hloubkou souřadnice vrcholů (x, y, z) přepočítány hyperbolicky. Z tohoto důvodu pak neodpovídají reálné souřadnice bodů (x, y, z) uvnitř polygonů (přepočítané hyperbolicky) jejich texturovacím souřadnicím (u, v) (interpolovaným lineárně) (viz. obr. 12.22). Jinými slovy perspektivní projektce není afinní transformace a proto nejsou zachovány relativní vzdálenosti.



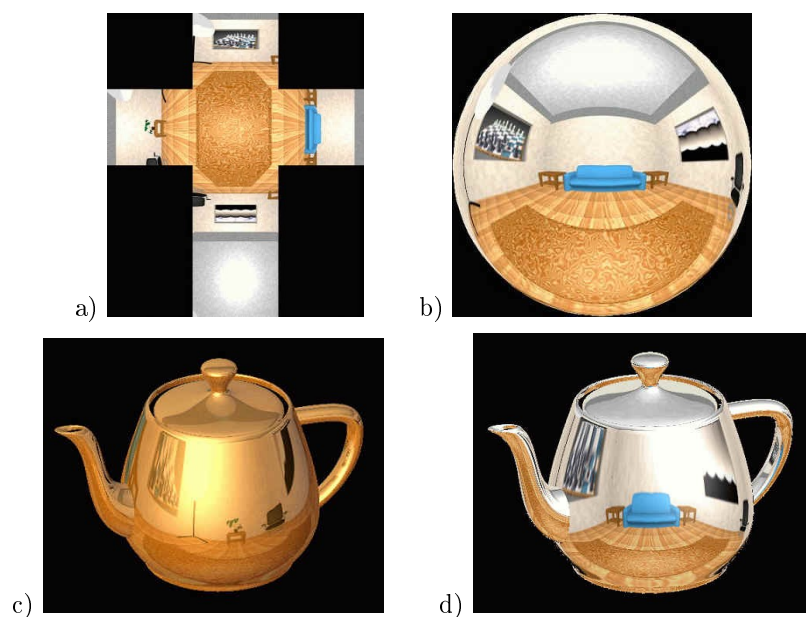
Obrázek 12.15: Aplikace displacement mappingu na povrch objektu: a) originální povrch s barevnou texturou, b) povrch s displacement texturou c) výsledný efekt aplikace barevné a displacement textury.



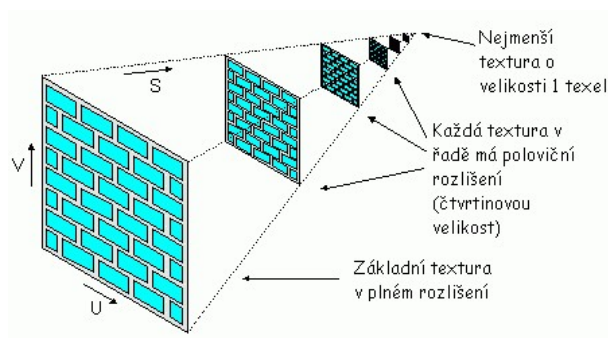
Obrázek 12.16: Environmental mapping efekt na povrchu objektu.



Obrázek 12.17: Schéma aplikace environmental mappingu.



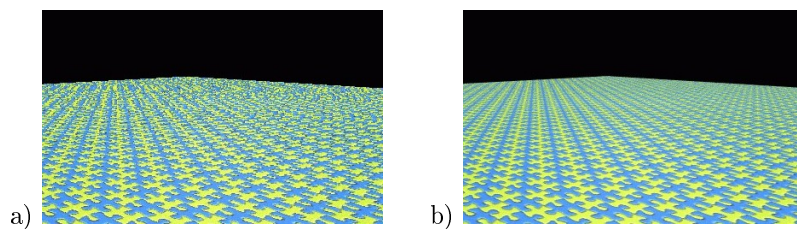
Obrázek 12.18: Aplikace environmental mappingu na povrch objektu: a) obraz okolí na obalové krychli, b) obraz okolí na obalové kouli, c) obraz okolí mapovaný z krychle na čajník, d) obraz okolí mapovaný z koule na čajník.



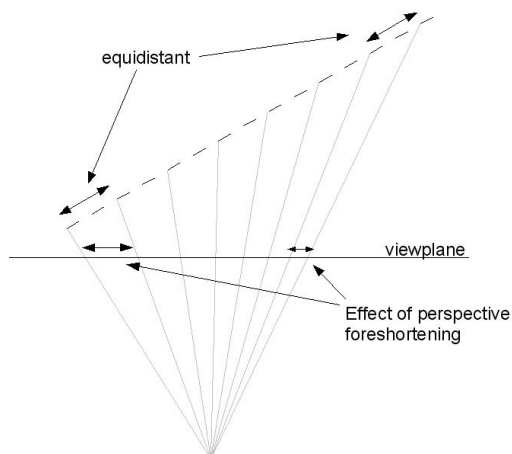
Obrázek 12.19: Schéma hierarchie MIP textury.



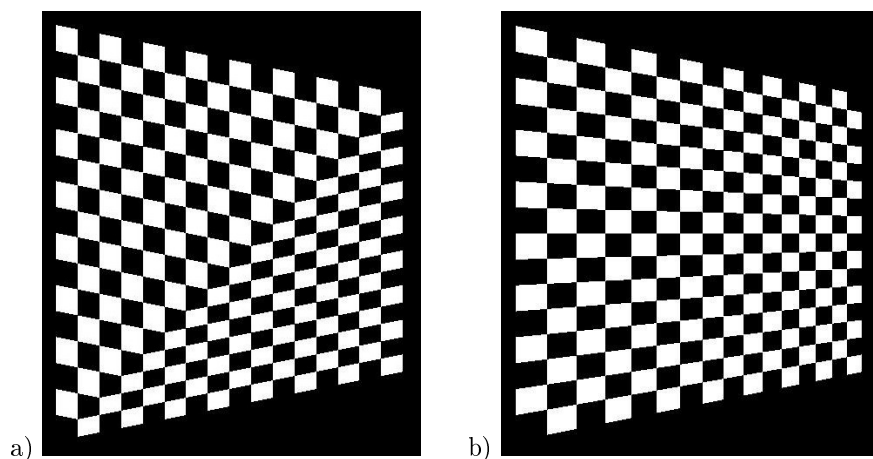
Obrázek 12.20: Zobrazení matice MIP textury.



Obrázek 12.21: Srovnání zobrazení textury: a) bez MIP mapingu, b) s aktivním MIP mapingem.



Obrázek 12.22: Zobrazení problematiky perspektivní korekce mapování textur.



Obrázek 12.23: Zobrazení textur v perspektivní projekci: a) s lineární interpolací texturovacích souřadnic, b) s hyperbolicou interpolací texturovacích souřadnic.

Toto zkusení je možné korigovat následujícími způsoby:

- Rozdělením každého texturovaného polygonu (taselace) na skupinu malých trojúhelníků a lineární interpolací texturovacích souřadnic do jejich vrcholů. Na malých trojúhelnících je chyba vznikající vlivem perspektivní projekce zanedbatelná (viz. obr. 12.23a).
- Hyperbolicou korekcí texturovacích souřadnic při jejich lineární interpolaci přes jednotlivé texturované polygony (během rasterizace polygonů). Tak posuneme texturovací souřadnice na správné místo i vzhledem k perspektivní projekci. Texturovací souřadnice jsou tak nejen lineárně interpolovány, ale ještě podrobeny perspektivnímu dělení vzdáleností (viz. obr. 12.23b).

Shrnutí

Textura je popis struktury na povrchu objektů. Její nanášení (mapování) na povrch objektů se nazývá: "Texturování". Vlastní textura může být definována datovou maticí nebo procedurou, ve 2D, 3D nebo 4D (čas) prostoru. Textura může na povrch objektu mapovat nejrůznější vlastnosti: RGB barvu, průhlednost, světelné vlastnosti, plastickou strukturu povrchu, osvětlení, zrcadlení okolí atd. Aplikací textur na povrch objektů dosáhneme výrazně vyšší úrovně realističnosti jejich zobrazení při zachování relativně jednoduché geometrie. Mapování textury na povrch objektu může být provedeno přímo prostřednictvím analytické inverzní mapovací funkce, po částech pomocí texturovacích souřadnic nebo nepřímo promítáním z jednoduchého pomocného objektu.



Kontrolní otázky

1. Co je to textura, texturování, texel?
2. Jakým způsobem může být textura definována a v jaké dimenzi?
3. Jaké vlastnosti mohou být texturou mapovány na povrch objektu?
4. Jakým způsobem provádíme interpolaci hodnot textury během rasterizace?
5. Co je to mapování textur na povrch objektu?
6. Jaké znáte způsoby mapování textur na povrch objektu?
7. Co je podstatou mapování textur prostřednictvím inverzní mapovací funkce a kdy je vhodné ji použít?
8. Jaký je princip mapování textur prostřednictvím texturovacích souřadnic a kdy je vhodné je použít?
9. Jak probíhá mapování textur promítáním a kdy je vhodné ho použít?
10. V čem spočívá mapování 3D textur a kdy je vhodné ho použít?
11. Který způsob mapování textur je nejvhodnější a proč?
12. Jakým způsobem je dosaženo efektu hrbolatého povrchu u Bump mappingu?
13. Jakým způsobem je dosaženo efektu hrbolatého povrchu u Displacement mappingu?
14. Jaký je rozdíl mezi Bump a Displacement mappingem?
15. Kdy je vhodnější použití Bump mappingu a kdy Displacement mappingu?
16. V čem je výhoda použití Bump mappingu a Displacement mappingu?
17. Jakého efektu dosáhneme aplikací Environmental mappingu?
18. Co znamená MIP mapping a jakým způsobem se aplikuje při texturování?
19. Co je přínosem techniky MIP mapping při texturování objektů?
20. Co je příčinou zkreslení textur při zobrazení texturovaných objektů v perspektivní projekci?
21. Jakým způsobem můžeme provést korekci zkreslení textur při zobrazení texturovaných objektů v perspektivní projekci?



Kapitola 13

Realistické zobrazení 3D objektů

Cílem této kapitoly je ...

13.1 Ray-tracing

13.2 Radiosita

13.3 Particle-tracing

Shrnutí

Kontrolní otázky



1:00



Kapitola 14

Antialiasing, obraz jako vícerozměrný signál



1:00

Cílem této kapitoly je ...



14.1 Alias

Co to je? Jak vzniká?

Časový alias.

Zubaté hrany

Alias textur

14.2 Antialiasing

Supersampling

Multisampling

MIP mapping

Shrnutí

Kontrolní otázky



Kapitola 15

Grafické formáty

Cílem této kapitoly je ...

15.1 Ztrátové formáty

15.2 Neztrátové formáty

15.3 Vektorové formáty

Shrnutí

Kontrolní otázky



1:00



Kapitola 16

Základy grafických 3D API



1:00

Cílem této kapitoly je ...

- Nejrozšířenější API pro 3D vektorovou grafiku.

- Multiplatformní

- HW akcelerace

- Stavový stroj

- Použité metody a algoritmy: transformace, osvětlení, textury, shadery, vektorové entity (vrcholy, body, uzečky, tri, quad, poly...)

- Pipeline zpracování příkazů a dat.



Shrnutí

Kontrolní otázky

